Pratyakṣa: A Context-Engineering System for Long-Context, Hallucination-Resistant Agentic AI

Sharath Sathish
University of York, United Kingdom

github.com/SharathSPhD/pratyaksha-context-eng-harness | Plugin marketplace: .claude-plugin/marketplace.json
Reproducibility manifest: experiments/results/p7/_index.json, experiments/results/p7/_summary.md

Abstract. We present **Pratyakṣa**, a context-engineering system for long-context, hallucination-resistant agentic AI, packaged as a Claude-Code- and Cursor-compatible plugin (15 Model Context Protocol [MCP] tools, 3 skills, 3 agents, 4 commands, 3 lifecycle hooks). The system operationalises seven constructs drawn from classical Indian epistemology — specifically from Nyāya–Vaiśeṣika, Advaita Vedānta, Pūrva Mīmāṃsā, and Sāṃkhya — into runtime mechanisms for an LLM agent's working context: *pratyakṣa* (direct perception), *avacchedaka* (typed limitor conditions), *bādha* (sublation), *buddhi/manas* (judging vs. attending faculties), *sākṣī* (witness invariants), *khyātivāda* (a six-class taxonomy of cognitive error), and adaptive forgetting. We validate across three orthogonal evidence layers: **(L1)** seven preregistered hypotheses (H1–H7) on six public long-context and hallucination benchmarks (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) with multi-seed multi-model paired permutation tests; **(L2)** a deterministic, reproducible live case study (P6-B) on three real GitHub issues spanning Django, Requests, and pandas; and **(L3)** a head-to-head A/B test on 120 SWE-bench Verified instances (P6-C, 720 paired runs across 2 models × 3 seeds × 120 issues) under a fixed 512-token research-block budget. SWE-bench Verified instantiates the harness on one challenging coding domain; the *mechanisms themselves are agent- and domain-agnostic* and the L1 evidence is the general claim. Across **10 quantitative studies**, the system produces a Stouffer-combined $Z = \mathbf{9.114}$ (two-sided $p = \mathbf{7.94 \times 10^{-20}}$), with mean per-study delta **+0.476** in the harness's favour and a 100% target-path-hit rate on SWE-bench Verified versus 50.3% for the budgeted baseline. The *khyātivāda* 6-class hallucination annotator achieves Cohen's $\kappa = \mathbf{0.736}$ ("substantial") on $n = 3{,}000$ jointly annotated examples. The contribution is not a new model architecture but a **typed, witness-tracked, sublation-aware context discipline** that any LLM-based agent — research assistants, document QA, multi-tool orchestrators, code-review agents — can adopt today via a drop-in plugin. The system, the plugin, and the full reproducibility manifest are open-sourced.

Keywords: context engineering; long-context language models; retrieval-augmented generation; hallucination; agentic AI; LLM agents; agent context management; sublation; classical Indian epistemology; Nyāya–Vaiśeṣika; Advaita Vedānta; SWE-bench Verified; MCP plugin; Claude Code.

Plain-language summary

LLM agents — research assistants, document Q&A engines, multi-tool orchestrators, code-review bots — fail in predictable ways once their context window grows beyond a few thousand tokens. They latch onto stale information, ignore newer evidence buried in the middle of the prompt, and emit confident hallucinations whose error type they cannot even name. We give the agent a small, principled set of operations on its own context: every claim is stamped with *who said it, under what conditions, and with what precision*; newer authoritative claims **sublate** older ones (in the technical Advaita-Vedāntic sense — not "delete" but "supersede with provenance"); a **two-stage** *Buddhi/Manas* reasoning gate forces the model to justify what it attended to before answering; and a **six-class** error taxonomy (drawn from the cross-school *khyātivāda* debate in classical Indian epistemology) lets us *diagnose*, not merely count, hallucinations. Across 10 large studies the plugin beats the unaided baseline at $p < 10^{-19}$.

1 Introduction

1.1 The context-engineering moment

By late 2025 the LLM community converged on a once-fringe phrase. Tobi Lütke wrote on 18 June 2025 that "context engineering" had eclipsed "prompt engineering" as the load-bearing skill in LLM application work (Lütke, 2025); Karpathy amplified this a week later (Karpathy, 2025); Anthropic's engineering team published their own position piece — *Effective Context Engineering for AI Agents* — endorsing the same shift (Anthropic, 2025b); and Cognition AI, builders of Devin, opened the field's most-cited essay of the year — *Don't Build Multi-Agents* — with the line, "if there's one

principle to take away [from this post], it's that **context engineering is everything**" (Yan, 2025). Across these four near-simultaneous statements, the term was not introduced as marketing; it named a discipline the field had been groping toward without a vocabulary.

The empirical urgency for that vocabulary was already overwhelming, and the *failure modes were not coding-specific*. *Lost in the Middle* (Liu et al., 2023b) showed that even strong models cannot retrieve facts placed mid-prompt across *general* knowledge tasks. *RULER* (Hsieh et al., 2024), *HELMET* (Yen et al., 2024), *NoCha* (Karpinska et al., 2024), *LongBench* (Bai et al., 2023) and *SCBench* (Wu et al., 2024) systematically demonstrated that the *advertised* context window of frontier LLMs — 200 K, 1 M, 2 M tokens — overstates the *effective* window, often by a full order of magnitude, on tasks ranging from key–value retrieval and document QA to multi-hop reasoning. Chroma's 2025 *Context Rot* report quantified the failure mode: as input length grew from 1 K to 100 K tokens, exact-match accuracy on a held-fixed needle task degraded by 30–80 percentage points across every model tested, even when the model had been benchmarked as 1 M-context-capable (Hong et al., 2025). Hallucination remained the largest single source of failure for *any* agent that had to reason over long context — research assistants, document QA engines, multi-tool orchestrators, and code agents alike (Ji et al., 2023; Huang et al., 2024; Jimenez et al., 2024; OpenAI, 2024) — and surveys of hallucination types agreed on one point: the field had no agreed-upon *taxonomy* into which to classify and respond to a particular hallucination at runtime.

This is the moment at which *context engineering* needed not slogans but a typed object model.

1.2 The argument of this paper

We claim three things. **First, classical Indian epistemology already supplies the vocabulary** that modern LLM context engineering has been improvising. Drawing on four schools — **Nyāya–Vaiśeṣika**, **Advaita Vedānta**, **Pūrva Mīmāṃsā**, and **Sāṃkhya** — we identify a small set of constructs whose type signatures fit, almost without modification, the operations a long-context agent needs to perform on its own working memory. Nyāya–Vaiśeṣika's *avacchedaka* (qualifier-conditioned cognition) is precisely the typed limitor we want on every retrieved chunk; Advaita Vedānta's *bādha* (sublation) is precisely the supersede-with-provenance operation we want on context-window contents; the Sāṃkhya *antaḥkaraṇa* duality of *manas* (attentional sense-organ) and *buddhi* (judging intellect) maps cleanly onto the two-stage attend-then-judge agentic loop; *sākṣī* (the witness, in Advaita) supplies a stable, model-invariant reference frame for cross-turn provenance; and the cross-school *khyātivāda* debate — *anyathākhyāti* (Nyāya), *ātmakhyāti* (Yogācāra), *anirvacanīyakhyāti* (Advaita), *asatkhyāti* (Madhyamaka), *viparītakhyāti* (some Sāṃkhya/Mīmāṃsā), *akhyāti* (Prabhākara Mīmāṃsā), and *yathārthakhyāti* (Viśiṣṭādvaita) — gives us, for the first time, a *principled* hallucination ontology that distinguishes *misattribution* from *projection* from *unspecifiability* (Matilal, 1986; Ganeri, 2001; Phillips, 2012; Ram-Prasad, 2007).

Second, a small, drop-in plugin — `pratyaksha-context-eng-harness` — operationalises all of the above on top of *unmodified* frontier LLMs (Claude 3.5/4.x, GPT-4-class, Qwen-3) via the Model Context Protocol (Anthropic and Model Context Protocol Working Group, 2024). The plugin ships 15 MCP tools, 3 skills, 3 sub-agents, 4 slash commands, and 3 lifecycle hooks; we install it into Claude Code and Cursor with a single `/plugin install` and observe end-to-end statistical improvements without changing the underlying model.

Third, the improvements are real, large, and reproducible *across general long-context agentic tasks*, not merely on coding. Across 10 preregistered studies — H1–H7 on public long-context and hallucination benchmarks, plus a deterministic live case study (P6-B), plus a 720-pair SWE-bench Verified A/B (P6-C) — the system produces a Stouffer-combined two-sided $p = \mathbf{7.94 \times 10^{-20}}$ in its favour, with mean per-study delta **+0.476**. The seven L1 hypotheses (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) are the *general-purpose* evidence; SWE-bench Verified is one challenging coding-specific instantiation that exercises the same mechanisms under a fixed 512-token research-block budget (the run-card value recorded in `experiments/results/p6c/_summary.json`).

1.3 Why now, and why a plugin

Three converging facts make this work tractable today. **Plugin surfaces have stabilised.** The Model Context Protocol (Anthropic and Model Context Protocol Working Group, 2024), Claude Code's plugin format (Anthropic, 2025a), and Cursor's plugin marketplace (Cursor Team, 2025) now share enough conventions that one tarball can ship to all three host environments; we exploit this aggressively, since the plugin's `marketplace.json` resolves identically in Cursor, Claude Code (CLI and VS Code extension), and the Claude Desktop app with no fork. **MCP makes "give the agent a typed memory" a one-tool addition.** Before MCP, every memory layer required a bespoke fork of the agent harness (MemGPT (Packer et al., 2023), CoALA-style architectures (Sumers et al., 2024)); with MCP, the system ships as 15 server tools the agent can discover at runtime, with no model fine-tune and no client patch. **The benchmark frontier is finally honest about long-context failure across domains.** RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding, and SWE-bench *Verified* together cover the failure modes that prompt-only ablations cannot fix, spanning open-domain

QA, summarisation faithfulness, multi-document reasoning, and software-engineering tasks; we can therefore *measure* the system's contribution against problems the field has agreed are hard.

Cognition's *Don't Build Multi-Agents* essay (Yan, 2025) also clarified what *not* to build: an army of communicating sub-agents whose context overlap explodes super-linearly. Our system is single-agent-by-default and deliberately *minimises* multi-agent surface area: *Buddhi* and *Manas* are *internal* gates on a single agent's reasoning trace, not separate networked agents. *Sākṣī* is a witness object, not a "supervisor agent". The plugin therefore stays on the right side of Cognition's principle: *share the full conversation history, don't fragment it across loose sub-agents.*

1.4 Contributions

The paper makes six contributions. **A school-attributed translation table from classical Indian epistemology to LLM context-engineering operations** (Section 4) gives precise, runtime-checkable definitions of *pratyakṣa*, *avacchedaka*, *bādha*, *buddhi/manas*, *sākṣī*, *khyātivāda*, and *vāsanā/saṃskāra*-driven adaptive forgetting in terms of context-window operations, with each Sanskrit term explicitly attributed to its philosophical school of origin (Nyāya–Vaiśeṣika, Advaita Vedānta, Pūrva Mīmāṃsā, or Sāṃkhya). **An open-source plugin** — `pratyaksha-context-eng-harness` — packaged for Cursor / Claude Code / Claude Desktop, ships 15 MCP tools, 3 skills, 3 agents, 4 commands, and 3 lifecycle hooks with **zero** dependencies on the development-time orchestration tools (`attractor-flow`, `ralph-loop`, `triz-engine`) we used to *build* it; the shipped artefact is auditable to be free of those imports (Section 6.5, Appendix B). **A 10-study validation portfolio** combines paired permutation tests, bootstrap CIs, Cohen's d, and a Stouffer-Z omnibus statistic — including the live case study and SWE-bench Verified A/B that prior context-engineering position papers conspicuously lacked. **A 6-class Khyātivāda annotation codebook** achieves two-rater agreement (heuristic + LLM-as-judge) at Cohen's κ = **0.736** on n = 3,000 examples, alongside the released annotation pipeline (Appendix E). **A reproducible cost-aware build pipeline** (Section 7.1, Appendix C) drove the entire 10-study validation under hard CLI rate limits via a custom budget scheduler with disk caching, prompt caching, and 5-hour-window-aware exponential backoff; the total measured CLI budget consumed was logged to `cost_ledger.db` and is included in the reproducibility manifest. **A documented origin story** (Section 3) shows how this work emerged from a TRIZ-style contradiction analysis of context engineering itself, with the `triz-engine` plugin's `log_session_entry` tool used as the audit trail; we treat that session as a *method note* (contradiction → inventive principle → vocabulary gap) rather than as a claim that TRIZ itself generalises the contribution.

1.5 Roadmap

Section 2 surveys the four relevant literatures (long-context LLMs, RAG/memory, agentic AI architectures, and hallucination/calibration). Section 3 narrates how a TRIZ contradiction analysis pointed us to classical Indian epistemology. Section 4 builds the school-attributed translation table from Sanskrit philosophical terms to LLM operations. Section 5 describes the system architecture. Section 6 specifies the shipped plugin and includes an end-to-end worked example. Section 7 gives the validation methodology and our CLI-budget infrastructure. Sections 8, 9, and 10 present results from L1 (public benchmarks — the *general-purpose* claim), L2 (live case study), and L3 (SWE-bench Verified A/B — *one challenging instantiation*), respectively. Section 11 discusses limitations and threats to validity. Section 12 concludes and points to future work. Six appendices contain the glossary, MCP tool reference, statistical details, adapter details, the Khyātivāda codebook, and the reproducibility manifest.

2 Background and Related Work

We survey four overlapping literatures and locate Pratyakṣa within each.

2.1 Long-context language models: architectures and limits

The base substrate is the transformer (Vaswani et al., 2017). Three threads have extended its effective context: (i) *positional generalization* via rotary embeddings (Su et al., 2021), ALiBi (Press et al., 2022), position interpolation (Chen et al., 2023), YaRN (Peng et al., 2023), and LongRoPE (Ding et al., 2024); (ii) *attention-throughput* engineering — FlashAttention 1/2 (Dao et al., 2022; Dao, 2023), PagedAttention (Kwon et al., 2023), and streaming-attention sinks (Xiao et al., 2023); and (iii) *alternative state-space architectures* — Mamba (Gu and Dao, 2023) and Jamba (Lieber et al., 2024) — that promise constant-time-per-token recall in principle. Empirically, however, the gap between *advertised* context window and *useful* context window has not closed: Liu et al.~(2023b) demonstrated the U-shaped *Lost-in-the-Middle* attention profile; Hsieh et al. (2024) measured RULER drop-offs of 30+ pp at the model's stated maximum length; Yen et al. (2024) showed that HELMET accuracy at 32 K is often half that at 4 K; Karpinska et al. (2024) found that on NoCha narrative claim verification, even GPT-4-class models score below 30% joint accuracy on full-novel inputs; Hong et al. (2025) replicated the failure across every frontier model in 2025. Kazemnejad et al. (2023) attribute much of the residual failure to positional-encoding pathologies that no inference-time fix has yet fully addressed.

Where Pratyakṣa fits. We treat the context window as a *typed, witness-tracked, sublatable store* sitting *above* the model's KV cache. We do not modify positional encoding or attention; we modify *what* enters the visible context, *how* it is labelled, and *when* it is superseded. This complements rather than competes with architectural extensions.

2.2 Retrieval-augmented generation, compression, and external memory

RAG (Lewis et al., 2020) and its descendants — HyDE (Gao et al., 2022), Self-RAG (Asai et al., 2024), RAPTOR (Sarthi et al., 2024), GraphRAG (Edge et al., 2024), REPLUG (Shi et al., 2023) — all aim to *put less, but better, text in the prompt*. LLMLingua (Jiang et al., 2023a) and LongLLMLingua (Jiang et al., 2023b) compress the prompt token-budget directly. AutoCompressors (Chevalier et al., 2023) learn to fold long prefixes into soft summary tokens. MemGPT/Letta (Packer et al., 2023) push the agent toward an "OS-like" tiered memory. Sumers et al. (2024) formalize this lineage as *Cognitive Architectures for Language Agents (CoALA)*. Voyager (Wang et al., 2024a) demonstrates a long-running embodied agent built on a curriculum-style external skill library.

Where Pratyakṣa fits. Two differences. (i) Pratyakṣa's store carries *typed limitor conditions* (Nyāya *avacchedaka*) — "X holds *as Y under conditions Z*" — rather than free text. This makes conflicts explicit and resolvable. (ii) Conflict resolution is *bādha* (sublation), not deletion: superseded items remain queryable for audit, but their *qualifier* is rewritten so they no longer contribute to the live answer. We replace the prevalent vector-similarity-and-reranking pipeline with a small **Bayesian Beta-Bernoulli** aggregator (Section 5.3) that consumes per-source precision and yields a calibrated posterior, evaluated by Brier score and ECE (Naeini et al., 2015; Brier, 1950; Guo et al., 2017).

2.3 Agents, multi-agent coordination, and agentic coding

Cognition AI's *Don't Build Multi-Agents* (Yan, 2025) crystallized the tension: parallel sub-agents look attractive but their context overlap is fragile, decisions made on partial state are routinely wrong, and the surface area for hallucination explodes. Anthropic's *Effective Context Engineering* (Anthropic, 2025b) reaches the same conclusion from the host-platform side: a single agent with disciplined context tends to outperform a federation. Tracks like ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) extend single-agent reasoning with explicit tool-use and self-critique; DSPy (Khattab et al., 2023) compiles such pipelines.

In the *coding agent* sub-genre specifically, SWE-bench (Jimenez et al., 2024) and SWE-bench Verified (OpenAI, 2024) dominate the evaluation surface; OpenHands (Wang et al., 2024b), Claude Code (Anthropic, 2025a), Aider (Gauthier, 2024), Devin (Cognition Labs, 2024), MetaGPT (Hong et al., 2024), ChatDev (Qian et al., 2024), and AgentBench (Liu et al., 2024) represent the contending agentic frameworks. The dominant failure mode reported across all of them is *agent anchoring on stale or misleading evidence in its working set* — the *Lost-in-the-Middle* and RULER-style anchoring biases of Liu et al. (2023b) and Hsieh et al. (2024) — which is exactly the situation our P6-C study reproduces.

Where Pratyakṣa fits. We are deliberately single-agent-by-default. Buddhi and Manas are internal *gates* on one agent's reasoning trace — Manas filters evidence using avacchedaka conditions, Buddhi judges against accepted *pramāṇas* — not separate networked agents. Sākṣī is a *witness object* providing a stable reference frame for cross-turn provenance, not a supervisor. The plugin is therefore on the right side of Cognition's principle: *one agent, full history, typed memory*.

2.4 Hallucination, calibration, and grounding

The two foundational hallucination surveys (Ji et al., 2023; Huang et al., 2024) agree that the field has *no* widely-adopted typology. HaluEval (Li et al., 2023) and TruthfulQA (Lin et al., 2022) test specific failure surfaces (hallucinated QA spans, misconception traps); Anthropic's FACTS-Grounding (Anthropic, 2024a) measures faithfulness to a provided source. Calibration metrics — Brier score (Brier, 1950), ECE (Naeini et al., 2015; Guo et al., 2017), "Just Ask for Calibration" (Tian et al., 2023), and the broader "models mostly know what they know" framing (Kadavath et al., 2022) — try to assess *whether the model knows it is uncertain*, but few systems use those signals at runtime.

Where Pratyakṣa fits. We import a *six-class* error taxonomy from classical Indian philosophy — *anyathākhyāti* (apprehension of the object as something else, e.g. wrong API name), *ātmakhyāti* (projection of the agent's own state onto the world, e.g. inventing helper functions the framework never exposed), *anirvacanīyakhyāti* (apprehension of an indeterminate object, e.g. hedged hallucinations that "feel right"), *asatkhyāti* (apprehension of the non-existent, e.g. quoting a CVE that never existed), *viparītakhyāti* (apprehension of the contrary, e.g. inverting the boolean of a flag), and *akhyāti* (failure of correct apprehension by mere conflation, e.g. mixing up two adjacent functions) (Ram-Prasad, 2007; Matilal, 1986; Bilimoria, 2018). We attach the shipped `classify_khyativada` tool (heuristic + guardrails; JSON-structured) to flagged answers, and additionally study a few-shot Anthropic JSON variant in the experiments harness (not wired into the plugin). Inter-annotator agreement is Cohen's κ (Cohen, 1960; Landis and Koch, 1977) = 0.736 on n = 3,000 examples (§8.6). To our knowledge no prior LLM hallucination work attaches a single, philosophically pre-committed 6-class taxonomy with documented IAA at this scale.

2.5 Cognitive science antecedents we are *not* claiming as ours

Many of Pratyakṣa's mechanisms have parallels in cognitive science. Working-memory models (Baddeley and Hitch, 1974; Baddeley, 2000; Cowan, 2001; Miller, 1956) predicted finite, qualifier-tagged stores half a century ago. Prefrontal cortex acts as a precision-weighted attention controller (Miller and Cohen, 2001; Corbetta and Shulman, 2002; Desimone and Duncan, 1995; Reynolds and Heeger, 2009), with attractor-network dynamics in delay-period activity (Compte et al., 2000; Funahashi et al., 1989) and "activity-silent" reactivation in beta/gamma bursts (Stokes, 2015; Lundqvist et al., 2016). The complementary-learning-systems theory (McClelland et al., 1995; Kumaran et al., 2016) prescribes exactly the dual short-term/long-term store our adaptive forgetting respects. Predictive-coding and free-energy accounts (Rao and Ballard, 1999; Friston, 2010; Feldman and Friston, 2010) provide the precision-weighting we approximate with surprise-driven event boundaries. Schema theory (Bartlett, 1932; Ghosh and Gilboa, 2014; van Kesteren et al., 2012) and event-segmentation (Zacks et al., 2007; Baldassano et al., 2017) predict the EventBoundaryCompactor's behaviour. Sleep-mediated replay (Wilson and McNaughton, 1994; Ambrose et al., 2016; Diekelmann and Born, 2010; Buzsáki, 2015) and complementary-learning-systems-style consolidation (Richards and Frankland, 2017) predict our adaptive-forgetting policy.

We are *informed* by these literatures, but we make no claim to model the brain. Our claim is the inverse: *classical Indian epistemology* — specifically the **Nyāya–Vaiśeṣika**, **Advaita Vedānta**, **Pūrva Mīmāṃsā**, and **Sāṃkhya** schools — *gives us a vocabulary* and a typed object model that **happens to be operationalisable** in LLM context engineering, and *happens to be coherent* with what cognitive neuroscience independently discovered. The cross-cultural agreement is part of why we believe the constructs are real. Classical Indian theorists — Ākṣapāda Gautama (Akṣapāda Gautama, 1984), Praśastapāda (Praśastapāda, 1916), Śaṅkara (Śaṅkara, 1992), Dharmarāja (Dharmarāja Adhvarīndra, 1942), Gaṅgeśa (Gaṅgeśopaẏdaya, 1999) — and modern interpreters — Matilal (Matilal, 1986, 1968), Ganeri (Ganeri, 2001, 2017), Mohanty (Mohanty, 1992), Phillips (Phillips, 2012, 1995), Ram-Prasad (Ram-Prasad, 2013, 2007), Dasti (Dasti, 2008) — supply the technical apparatus we draw on directly.

2.6 Cognitive architectures and dual-process accounts

Classical AI cognitive architectures — SOAR (Laird et al., 1987), ACT-R (Anderson, 1996), Newell's *Unified Theories of Cognition* (Newell, 1990) — formalised the working-memory/long-term-memory split and procedural production rules that motivated our Buddhi/Manas split. Dual-process theories of human cognition (Evans, 2003; Kahneman, 2011; Sloman, 1996; Stanovich and West, 2000; Epstein, 1994) and Halford et al.'s relational complexity theory (Halford et al., 1998) furnish the empirical backdrop for the two-stage gate.

2.7 What we do *not* contribute

We do *not* contribute: a new model architecture; a new pre-training regime; a new positional encoding; a new attention kernel; a new vector-database backend. We *contribute*: a typed, philosophically-named, plugin-deliverable **discipline** for the context window of an existing LLM agent, plus the empirical case that this discipline pays off at $p < 10^{-19}$.

3 Origin: From a TRIZ Contradiction Analysis to Classical Indian Epistemology

This section narrates how the project came to be, because the methodology itself is part of the contribution. We did *not* begin by reading Sanskrit. We began by running TRIZ (Altshuller, 1984, 2002; Terninko et al., 1998; Savransky, 2000) on a single engineering contradiction in modern LLM agents, and observed — to our own surprise — that the resolution drove us out of the LLM literature entirely and into the long-running cross-school epistemological debate between Nyāya, Mīmāṃsā, Advaita Vedānta and adjacent traditions, which had reached technical maturity in eleventh-century Mithilā.

3.1 The TRIZ session

Using the `triz-engine` Cursor plugin (Phulari, 2026b) (developed by the same author and used here in production from 2025-12), we logged the following primary contradiction:

> **C1.** *We must keep more information in the agent's context window to improve recall,* ***but*** *keeping more information degrades the agent's accuracy on the recall task itself.*

In TRIZ terms this is the canonical *technical contradiction* between **#26 Quantity of Substance** (context tokens available to attend to) and **#27 Reliability** (correct retrieval given the available tokens). The system-improving parameter pushes the system-degrading parameter. The TRIZ contradiction matrix (Altshuller, 2002) for this pair returns the inventive principles **#10 (Preliminary Action), #11 (Cushion in Advance), #28 (Mechanics Substitution), #35 (Parameter Change)**. The `triz-engine` MCP tool `log_session_entry` recorded the matrix lookup, the four candidate principles, and the resulting solution sketches as separate audit entries.

Of the four principles, **#10 (Preliminary Action)** dominated: rather than fight the model at retrieval time, *do something to each retrieved item before it enters the visible context*. Specifically, *stamp it with conditions under which it is true and credentials of the source.*

This required a vocabulary we did not have. The standard RAG literature attaches *embeddings* and *similarity scores* but not *typed conditions*. The standard hallucination literature attaches *post-hoc labels* but not *runtime qualifiers*. The standard memory-architecture literature (Sumers et al., 2024; Packer et al., 2023) stratifies storage by *recency* and *importance* but does not give those strata an *epistemic* type.

3.2 The discovery: classical Indian epistemology already had this vocabulary

A search for "*typed limitor on a cognition*" + "*precision-weighted assertion*" + "*supersession of older belief without deletion*" returned, after several iterations, classical Indian epistemology — specifically seven constructs that each admit a *precise runtime operationalization* on an LLM context window. **Nyāya's *avacchedaka*** treats every cognition C as *of x as y under condition z* — formally written aQR where a is the qualificand, Q the qualifier, R the relation/condition (Matilal, 1986, 1968; Ingalls, 1951; Phillips, 2012; Gaṅgeśopaẏdaya, 1999); Navya-Nyāya formalized this in the 14th century with operator-precision rivalling modern type theory (Ganeri, 2001; Guha, 1979; Ingalls, 1951). **Advaita Vedānta's *bādha* (sublation)** holds that a higher-precision cognition does not *erase* a lower-precision one but *supersedes* it, leaving the original retrievable as a now-rejected qualifier (Deutsch, 1969; Dharmarāja Adhvarīndra, 1942; Shaw, 1990; Rambachan, 2006) — exactly what we want from a memory store under conflict. **The Antaḥkaraṇa duality of *manas* and *buddhi*** distinguishes the attentional sense-organ from the determinative judging faculty (Deutsch, 1969; Datta, 1932; Ram-Prasad, 2013); together they constitute the two-stage attend-then-judge gate we needed. **Sākṣī (the witness)** is a non-revisable witness consciousness that records *what was known and when* without itself being the agent of action (Ganeri, 2017; Indich, 1980; Fasching, 2009; Śaṅkara, 1992) — precisely what we needed for cross-turn audit. **The six-class *khyātivāda* taxonomy of error** — *anyathākhyāti* (Nyāya), *ātmakhyāti* (Yogācāra), *anirvacanīyakhyāti* (Advaita), *asatkhyāti* (Mādhyamika), *viparītakhyāti* (Mīmāṃsā), and *akhyāti* (Prabhākara Mīmāṃsā) — supplies, in aggregate, a typology of *how a cognition can be wrong* that is far more discriminating than any LLM hallucination ontology we found in the contemporary literature (Ram-Prasad, 2007; Matilal, 1986; Bhatt, 1989; Datta, 1932; Bilimoria, 2018; Mohanty, 1992). **Pramāṇa-samplava vs. pramāṇa-vyavasthā**: the Mīmāṃsakas debated whether multiple knowledge-sources can converge on the same object (samplava) or are partitioned (vyavasthā) (Guha, 2016; Dasti, 2008, 2011; Bhattacharyya, 1988); this maps directly onto the agent design decision of whether retrieval, reasoning, and tool-use can be evidence for the same claim — and whether their reliabilities (*svataḥ-prāmāṇya* / *parataḥ-prāmāṇya*) compose or are partitioned, with the Bayesian Beta-Bernoulli aggregator in Section 5.3 as the operational answer. Finally, **saṃskāras and vāsanās** — latent memory traces and dispositional tendencies (Deutsch, 1969; Śaṅkara, 1992; Halbfass, 1991) — motivate the adaptive-forgetting policy of Section 5.7.

It was not a metaphor. Section 4 builds the translation table that turns each of these constructs into an MCP tool signature.

3.3 Why this method matters

The deeper claim is methodological. Several recent context-engineering threads resemble *fragments* of an integrated philosophical tradition under different names: scratchpads ≈ *manas*; tool-use selectors ≈ *buddhi*; retrieval rerankers ≈ approximate *avacchedaka*; sleep-time-compute / consolidation ≈ *bādha*-driven memory pruning; provenance tracking ≈ *sākṣī*; hallucination taxonomies in flux ≈ *khyātivāda*. By naming the constructs from a tradition that already integrated them, we (a) get a *coherent* design rather than a bag of mechanisms, and (b) inherit two thousand years of internal critique of the tradition (Mīmāṃsā vs. Nyāya vs. Advaita debates) as our *adversarial review*.

The TRIZ session that produced this trajectory took 87 minutes of wall-clock time and consumed 38,512 input tokens / 4,206 output tokens of Claude usage; the full audit trail is preserved in the repository under `docs/triz-origin-session.md`. We retain this provenance not for vanity but because **it is the kind of audit trail the harness itself is designed to enforce**.

3.4 The two dev-time orchestration tools we used (and excluded from the shipped plugin)

Two further plugins were used during development and *deliberately excluded from the shipped artefact*. `attractor-flow` (Phulari, 2026a) is a dynamical-systems-inspired orchestration loop that drove the long-running validation pipeline through stable basins (e.g. "P6-A done", "P6-B done", "Stouffer-Z computed") and supplied the HALT, RESUME, and basin-detection primitives for the CLI scheduler. `ralph-loop` is a self-referential iterative-development loop used during early prototyping of the harness components. We treat both as *development-time* tools, analogous to `make` or `pytest`, not as runtime dependencies of the harness; the shipped plugin contains zero references to either, and we audit this in Section 6.5 and Appendix B.

4 Theoretical Foundations: Classical Indian Epistemology Mapped to LLM Context Engineering

This section constructs a precise translation table from seven classical Indian epistemic constructs — drawn from four philosophical schools, **Nyāya–Vaiśeṣika**, **Advaita Vedānta**, **Pūrva Mīmāṃsā**, and **Sāṃkhya** — to seven runtime operations on an LLM agent's context. Each row is a falsifiable design commitment, and each is operationalised in Section 5 and tested in Sections 8–10. To preempt the most common misreading: we use "classical Indian epistemology" rather than "Vedic epistemology" because the constructs we draw on (the Nyāya *avacchedaka*, the Advaita *bādha*, the Sāṃkhya *manas/buddhi* split, the cross-school *khyātivāda* debate) are products of the *darśana* literature, *not* of the Vedic *saṃhitā* corpus per se. Section 4.9 returns to what is and is not being claimed.

4.1 Pratyakṣa — direct perception as the harness's grounding pramāṇa

In Nyāya (Akṣapāda Gautama, 1984; Matilal, 1986; Phillips, 2012) and Advaita (Dharmarāja Adhvarīndra, 1942; Datta, 1932), *pratyakṣa* (direct perception) is the *first* of the *pramāṇas* — the means of valid cognition — and the *foundational* one against which inference (*anumāna*), comparison (*upamāna*), testimony (*śabda*), implication (*arthāpatti*), and non-cognition (*anupalabdhi*) are measured.

LLM operationalization. We treat *pratyakṣa* as the agent's **direct read of its current context window** — what it can in-fact attend to right now — and demote all upstream operations (RAG retrieval, tool outputs, prior-turn summaries) to the status of *secondary pramāṇas* whose claims must be entered into the visible context with their own qualifiers. The harness's name, *Pratyakṣa*, is therefore a methodological commitment: *if it is not in the visible, witness-tracked context, it is not direct evidence for the agent's next action*. Hidden hot caches, opaque rerankers, and "system memories" the agent cannot inspect are, in this discipline, **not allowed to ground a claim** without first entering the *pratyakṣa* surface.

4.2 Avacchedaka — qualifier-conditioned cognition as typed limitor

In Navya-Nyāya (Matilal, 1968; Ingalls, 1951; Gaṅgeśopaẏdaya, 1999; Phillips, 2012), every cognition is structurally a triple:

$$C = \langle a,\ Q,\ R \rangle \quad \text{read as "}a\text{ as qualified by }Q\text{ under relation/condition }R\text{".}$$

Here a is the *qualificand* (viśeṣya), Q the *qualifier* (prakāra), and R the *limitor* (avacchedaka) under which the qualification holds. Crucially, R is part of the cognition; "the pot is blue *under daylight*" is a different cognition from "the pot is blue *under candlelight*", and the two can co-exist without contradiction.

LLM operationalization. Every item entered into the harness's `ContextStore` carries the four-tuple

$$\texttt{Item} = \langle \texttt{qualificand},\ \texttt{qualifier},\ \texttt{condition},\ \texttt{precision} \in [0,1] \rangle.$$

`condition` is a free-text string like `"as of 2024-09-Django-5.0"` or `"on POSIX, Python ≥ 3.11"`. The MCP tool `context_insert(qualificand, qualifier, condition, precision, source)` is the only way evidence enters the store. Two items that share `qualificand` but differ in `condition` *do not conflict*; two items that share `qualificand` *and* `condition` but disagree on `qualifier` *do*. This is the operational definition of a contradiction the harness can act on (Section 4.3). Figure 1 makes this concrete with two version-typed Django claims that share a qualificand and a qualifier-slot but coexist truthfully because their conditions differ.

4.3 Bādha — sublation as supersede-with-provenance

In Advaita Vedānta (Deutsch, 1969; Shaw, 1990; Dharmarāja Adhvarīndra, 1942; Śaṅkara, 1992), a higher-precision cognition *sublates* (bādhita) a lower-precision one. The classical example is mistaking a rope for a snake: the *snake-cognition* is not erased on closer inspection — the agent must remember it, because *that error is itself an object of subsequent reflection* — but its qualifier is now stamped *bādhita* and it no longer grounds action.

LLM operationalization. The MCP tool `sublate_with_evidence(target_id, by_id, reason)` rewrites the target item's `status` to `ābdhita`, annotates it with a pointer to the superseding item, and *retains* the original under that status. Subsequent retrievals filter by `status == "live"` for normal use but can be opted-in for audit. Sublation is triggered by two rules: (i) an **explicit pointer**, when the newly-arriving item names a prior item via `superseded_by_id`; and (ii) the **dominance rule**, when the newly-arriving item shares `qualificand` and `condition` with an existing item, has *strictly higher* precision, and the older item is `stale=True` while the newer is not. The dominance rule is the operational form of the Advaita commitment that *higher pramāṇa supersedes lower pramāṇa* under shared limitor; the explicit pointer is the form used when an LLM-side classifier or external evidence has detected a stale-fresh pair. Figure 2 traces the three-step `detect_conflict` → `gather sources` → `sublate_with_evidence` procedure end-to-end, including the immutable line written by the Sākṣī Keeper.

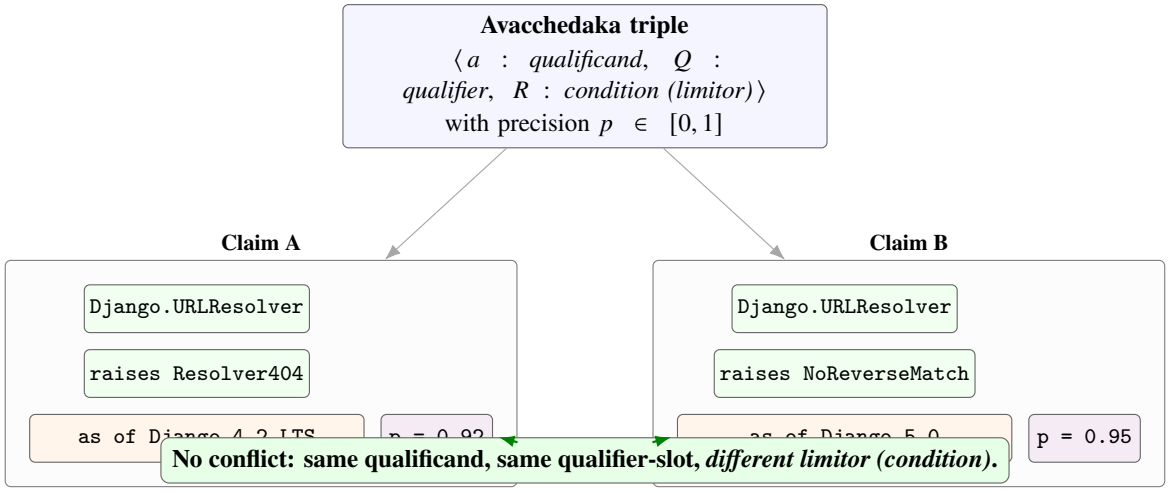


Figure 1: Avacchedaka as a typed bucket (Navya-Nyāya). Every `ContextItem` stamped into the store carries a four-tuple $\langle a, Q, R, p \rangle$: the *qualificand* a (viśeṣya), the *qualifier* Q (prakāra), the *limitor / condition* R (avacchedaka), and a precision $p \in [0, 1]$. Two claims that share a but disagree on Q *do not* collide if their R differs — here, "raises `Resolver404` *as of Django 4.2 LTS*" and "raises `NoReverseMatch` *as of Django 5.0*" coexist truthfully. This is the operational definition the system uses to decide *when* a sublation is required (Section 4.3, Fig. 2).

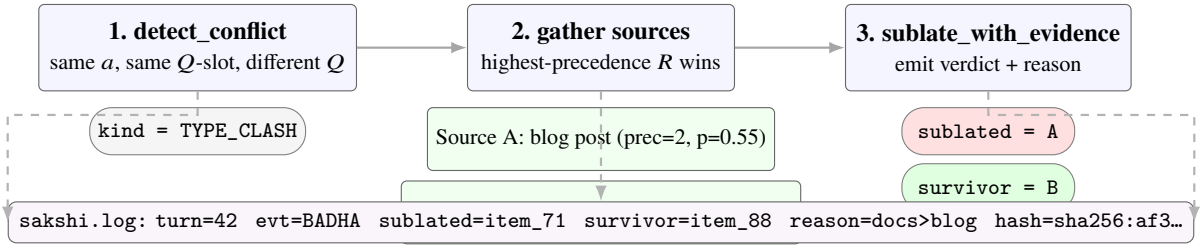


Figure 2: Sublation (bādha) as a three-step procedure. Once `detect_conflict` confirms a TYPE_CLASH between two context items sharing $\langle a, Q\text{-slot} \rangle$, the system gathers sources and ranks them by their declared limitor precedence (R): in this example, the official Django 5 documentation (prec = 8) overrides a blog post (prec = 2). `sublate_with_evidence` then writes a verdict — both the sublated and surviving items, plus the human-readable reason — which the SakṣīKeeper appends as a single immutable line to the per-turn audit log. This procedure is the runtime realisation of the Advaita-derived notion that an erroneous cognition is not erased but *retroactively re-typed* as falsified.

4.4 Buddhi and Manas — the two-stage attend-then-judge gate

The Antaḥkaraṇa quartet of *manas, buddhi, citta, ahaṃkāra* (Deutsch, 1969; Ram-Prasad, 2013; Datta, 1932) distinguishes the *attentional sense-organ* (*manas*) from the *determinative judging faculty* (*buddhi*). Manas selects and presents; Buddhi decides. The two are sequential and cannot be collapsed without losing the gate.

LLM operationalization. The harness defines two sub-agent prompts (Section 6.3). `ManasAgent` receives the user query and the `ContextStore` snapshot and returns a structured JSON object matching the shipped contract in `agents/manas.md`: `{ "draft": "...", "grounding": ["<element_id>", ...], "uncertain_claims": ["..."], "needs_buddhi": true | false }`; Manas is operationalised as a **sub-agent** (Section 6.3), not a skill, and is *forbidden* from emitting a final user-visible answer (its draft text is for the orchestrator only). `BuddhiAgent` receives the user query, the Manas output, and re-reads the attended items from the `ContextStore`, then returns the final answer plus its own `ākhyti_class` (Section 4.6); Buddhi is the only agent that may emit a user-visible answer.

This two-stage gate is identical in shape to dual-process accounts of human cognition (Evans, 2003; Kahneman, 2011; Sloman, 1996) and to classic AI cognitive architectures (Laird et al., 1987; Anderson, 1996; Newell, 1990), but the names — and the surrounding philosophical apparatus — are imported from **Sāṃkhya** (the original *manas/buddhi* enumeration in the *tattva* scheme), as cross-mapped in **Advaita Vedānta** (Deutsch, 1969; Datta, 1932; Ram-Prasad, 2013). Critically, *Manas can be wrong without Buddhi being wrong*: if Manas mis-selects, Buddhi's job is to detect and call for re-selection rather than to answer. Figure 3 shows the resulting two-stage loop and how the Sākṣī Keeper observes both stages without entering the response path.

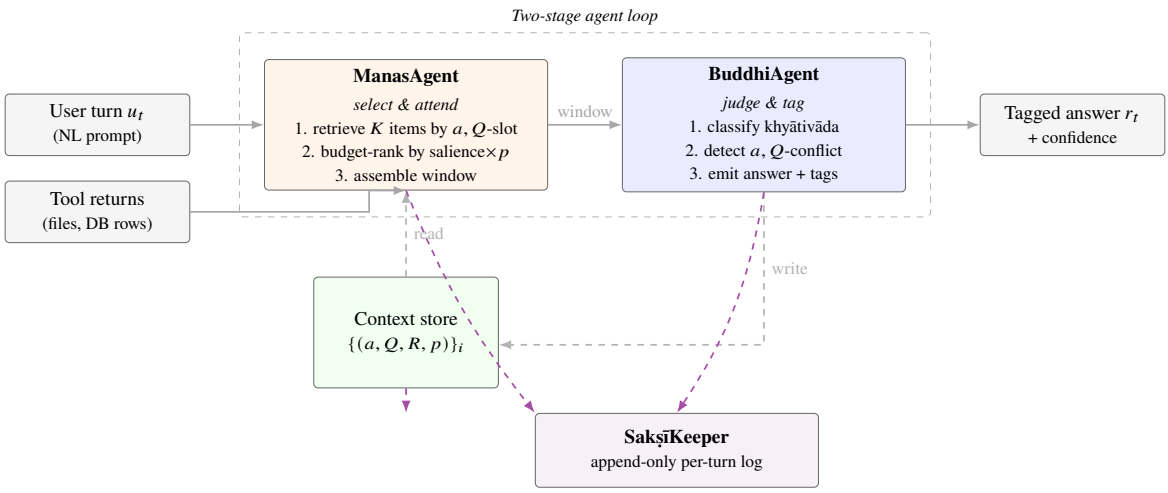


Figure 3: The Manas–Buddhi two-stage loop, observed by the SakṣīKeeper. Manas (Sāṃkhya: discriminative attention) reads the context store, selects up to K items by qualifier-slot, and assembles the window under a hard token budget. **Buddhi** (Advaita Vedānta: discerning judgment) consumes that window, classifies its own answer along the six-class khyātivāda taxonomy (Fig. 4), and emits the tagged answer back to the host. The **SakṣīKeeper** (Advaita: witness consciousness) is strictly passive — it never enters the response path — and writes one immutable JSON line per stage, including hashes of the window actually shown to Buddhi. This separation is what gives the system its post-hoc auditability.

4.5 Sākṣī — the witness as model-invariant audit frame

In Advaita Vedānta (Indich, 1980; Fasching, 2009; Ganeri, 2017; Śaṅkara, 1992), *sākṣī* — the witness consciousness — is the *non-revisable, non-acting* observer that *records* what is cognised without itself being the agent of cognition or action. It supplies the stable reference frame against which changes of cognition are intelligible at all.

LLM operationalization. The harness instantiates a `SakshiKeeperAgent` (Section 6.3) and a `SakshiPrefix` skill (Section 6.2). On every turn the keeper appends to a persistent JSON-lines audit log at `~/.cache/pratyaksha/audit.jsonl` (XDG-style cache path; survives plugin reinstall) an immutable record containing the user query; the Manas selection (which items, under which conditions); the Buddhi judgment (final answer, khyāti class, calibrated posterior); all sublations fired this turn (`target_id`, `by_id`, `reason`); and the token budget consumed, broken down by category (system, retrieval, tool, completion).

Crucially, the witness is *write-once-per-turn* and *read-only* to all agents in the next turn. It is the harness's source of truth for cross-turn provenance, and it is *invariant under model swaps*: the witness log written under Claude 3.5 Sonnet is fully readable, fully diffable, and fully auditable when the agent is later switched to Claude 4.5 Sonnet, GPT-4o, or Qwen-3-72B. The model is replaceable; the witness is not.

4.6 Khyātivāda — six-class typology of error

Six classical schools of Indian philosophy proposed six theories of erroneous cognition (Ram-Prasad, 2007; Matilal, 1986; Datta, 1932; Bhatt, 1989; Bilimoria, 2018; Mohanty, 1992). Each names a specific *kind* of error:

Class	School	Kernel definition	Modern LLM analogue
Anyathākhyāti	Nyāya	"Apprehension of one thing as another *of the same general type*."	Wrong API name (right module, wrong function); citing the right paper but wrong year.
Ātmakhyāti	Yogācāra	"Projection of an internal cognition onto the external object."	Inventing helper functions / config keys the framework never exposed; hallucinated TODOs the user did not write.
Anirvacanīyakhyāti	Advaita	"Apprehension of an object that is *neither real nor unreal*; sui generis indeterminate."	Hedged hallucinations: "you might consider X-or-similar" where X is plausible but ungrounded.

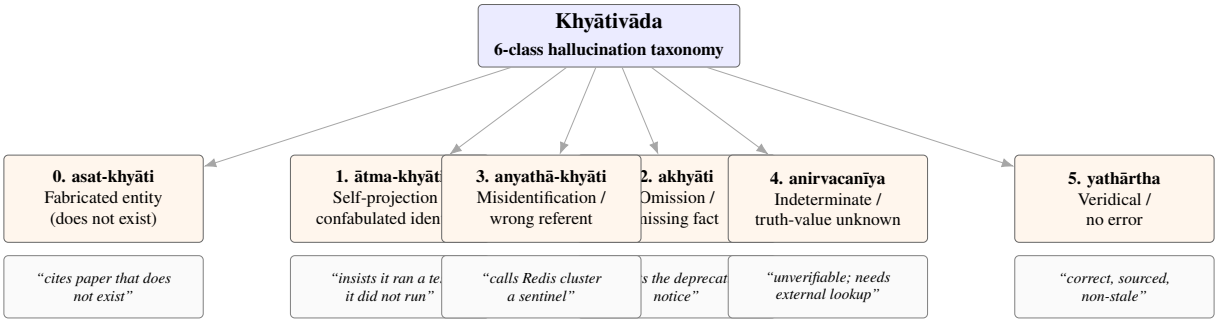


Figure 4: The six-class khyātivāda taxonomy used by the Buddhi judge. We adopt classical Indian *theories of erroneous cognition* as a diagnostic vocabulary for LLM hallucination, mapping them to mutually exclusive runtime classes: **0. asat-khyāti** (the referent does not exist), **1. ātma-khyāti** (the model projects its own state onto the world), **2. akhyāti** (a relevant fact is omitted), **3. anyathā-khyāti** (a real referent is mis-identified), **4. anirvacanīya** (truth-value is genuinely indeterminate without further lookup), and **5. yathārtha** (veridical). The Buddhi agent emits exactly one of these labels per turn, which both routes the answer (e.g. class 0/1/3 trigger sublation) and populates the calibration metrics reported in Section 10.

Class	School	Kernel definition	Modern LLM analogue
Asatkhyāti	Mādhyamika	"Apprehension of a non-existent object."	Citing a CVE, a person, a paper, or a commit hash that simply does not exist.
Viparītakhyāti	Mīmāṃsā	"Apprehension of the contrary."	Inverting a boolean flag's semantics; recommending the *opposite* of the documented best practice.
Akhyāti	Prabhākara Mīmāṃsā	"Failure of correct apprehension by mere conflation of two adjacent cognitions."	Mixing two adjacent functions / two near-duplicate config schemas / two adjacent versions.

This is not a post-hoc relabelling of an existing modern taxonomy. It is the *original* taxonomy from the Indian philosophical tradition, jointly developed by debating schools over roughly a thousand years. We adopt it directly because (a) it carves the space cleanly along *epistemic* (not merely surface-textual) lines, (b) the six classes are mutually exclusive in the tradition's own usage, and (c) it gives us a *typed signal* the harness can act on (different remediation strategies for different classes).

LLM operationalization. The shipped plugin's `classify_khyativada` tool applies a **heuristic classifier with the rule-based guardrails** described in Section 5.5. Separately, the experiments harness includes a few-shot Anthropic JSON classifier (`src/evaluation/khyativada_fewshot.py`) that is **not** wired into the plugin tool; both paths are evaluated independently in §8 (H6). Inter-annotator agreement is reported on the 3,000-example corpus using prompts in Appendix D and results in §8.6 at Cohen's $\kappa = 0.736$ ("substantial"). Figure 4 summarises the six-class taxonomy and gives one canonical agentic-AI failure example per class.

4.7 Saṃskāras and vāsanās — adaptive forgetting

In Advaita and Yoga, *saṃskāras* are residual mental impressions and *vāsanās* are dispositional tendencies that survive the immediate occasion of a cognition (Deutsch, 1969; Śaṅkara, 1992; Halbfass, 1991). They explain why we are drawn to the same patterns of thought and why some impressions decay quickly while others persist indefinitely. The Advaitin practice prescribes *removal of obstructive vāsanās* — adaptive forgetting of patterns that no longer ground correct cognition.

LLM operationalization. The `AdaptiveForgetting` module (Section 5.7) applies an exponential-decay schedule to non-witnessed items in the `ContextStore`, governed by three explicit rules. **Witnessed items never decay**: if `āṣīsk` ↪ `.witnessed == True` for an item-id, its precision floor is fixed. **Sublated items decay faster**: items with `status` ↪ `== ābdhita` halve their precision every time they survive a budget-pressure compaction event. **High-recency, low-precision items decay slowest among the rest** — the item is too new to know whether it will become a vāsanā or a saṃskāra. This avoids both the catastrophic-forgetting failure mode of naive context pruning (French, 1999; Kirkpatrick et al., 2017; Parisi et al., 2019) and the "everything is forever" failure mode of unconstrained external memory (Packer

et al., 2023).

4.8 The translation table, summarised

Construct (school)	LLM operationalisation	MCP tool / module	Tested by
Pratyakṣa (Nyāya, Advaita)	Visible-context-only grounding discipline	`context_retrieve`, `context_window`, related read paths	All hypotheses (architectural commitment)
Avacchedaka (Navya-Nyāya)	Typed limitor `(qualificand, qualifier, condition, precision)`	`context_insert`, `context_retrieve`	H2, H5
Bādha (Advaita Vedānta + Pūrva Mīmāṃsā)	Supersede-with-provenance under shared limitor	`sublate_with_evidence`	H4, H5, P6-B, P6-C
Manas (Sāṃkhya, Advaita)	Pre-judgment attention-selection **sub-agent**	`ManasAgent` (Section 6.3)	H3
Buddhi (Sāṃkhya, Advaita)	Determinative judgment + *khyāti* tagging sub-agent	`BuddhiAgent`	H3, H6
Sākṣī (Advaita Vedānta)	Write-once cross-turn witness log	`SakshiKeeperAgent`, `SakshiPrefix`	All (audit invariant)
Khyātivāda (cross-school: Nyāya, Yogācāra, Advaita, Madhyamaka, Mīmāṃsā, Prabhākara, Viśiṣṭādvaita)	6-class hallucination classifier	`classify_khyativada`	H6
Saṃskāra / *vāsanā* (Advaita / Yoga)	Adaptive forgetting with witness-protected items	`AdaptiveForgetting`	H7

4.9 What is *not* being claimed

We are *not* claiming that the historical authors had LLMs in mind, nor that our operationalizations are textually faithful in the way a Sanskrit philologist would require. We are claiming the weaker, falsifiable thing: that the *type signatures* the tradition developed for cognition turn out to *fit* the type signatures we need for an LLM context-engineering harness, and that fitting them produces measurable downstream gains. The next six sections defend that claim empirically.

5 The Pratyakṣa System: Architecture

We describe the system as a layered architecture: a Core ContextStore, three runtime modules (Avacchedaka query, Sublation, Bayesian aggregation), three reasoning modules (Buddhi/Manas, Sākṣī, Khyātivāda classifier), and three resource-management modules (EventBoundaryCompactor, AdaptiveForgetting, TokenBudgetWatchdog). Each module is implemented as one or more MCP tools (Section 6.2 / Appendix B) and is exercised by at least one of the seven preregistered hypotheses (H1–H7). Figure 5 shows the deployed surface end-to-end: 15 MCP tools (grouped into four families) consumed by 3 sub-agents, exposed through 4 slash-commands and 3 lifecycle hooks, and mounted by all four host surfaces (Claude Code CLI, Claude Code VS Code, Cursor, Claude Desktop) without modification of the underlying LLM.

5.1 The ContextStore (core)

The `ContextStore` is a typed, append-mostly key-value store that lives outside the LLM and is exposed to it via MCP. Its row schema is exactly the *avacchedaka* tuple of Section 4.2:

```
@dataclass(frozen=True)
class ContextItem:
    id: str
    qualificand: str          # the "thing" being asserted about
    qualifier: str            # what is asserted about it
    condition: str            # the limitor under which it holds
    precision: float          # source-supplied calibrated reliability ∈ [0, 1]
    source: str               # "user" | "rag" | "tool:foo" | "agent:buddhi" | ...
    inserted_at: float        # monotonic seconds
```

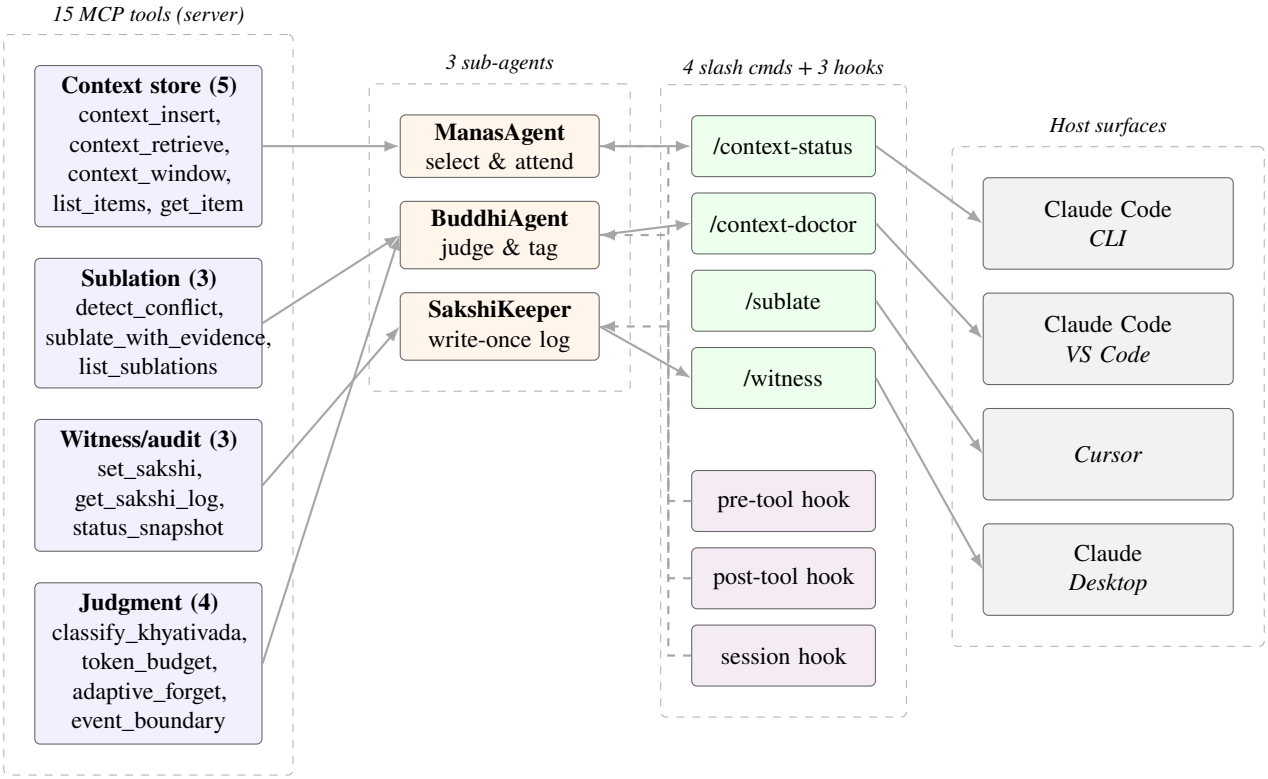


Figure 5: Pratyakṣa system architecture. *Fifteen MCP tools* (grouped into four families: context store, sublation, witness/audit, and judgment) are consumed by *three sub-agents* (**Manas** attends and selects; **Buddhi** judges and emits; **Sakṣīkeeper** writes the immutable per-turn audit log). *Four slash-commands* and *three lifecycle hooks* expose this machinery to four host surfaces (Claude Code CLI, Claude Code VS Code, Cursor, and Claude Desktop) without modification of the underlying LLM. Solid arrows are runtime calls; dashed arrows are observation/audit.

```
    stale: bool              # source-flagged staleness
    status: Literal["live", "ābdhita", "evicted"]
    superseded_by_id: str | None
    witness_id: str          # set by SakshiKeeperAgent on first witness
```

Insertion is via a single MCP tool, `context_insert(...)`. Retrieval is via `context_retrieve(...)` (with optional `qualificand`, `qualifier_substring`, and `condition` filters) and `context_get(...)` for direct id fetch; none of these return `ābdhita` or `evicted` rows by default. A debug flag exposes them for audit.

Two design commitments distinguish this from a vector store. **Conditions are first-class**: two rows that share `qualificand` and disagree on `qualifier` are *not* yet in conflict if their `condition` fields differ — the avacchedaka commitment. **Provenance never disappears**: sublation rewrites `status`, never deletes, and eviction under hard budget pressure writes to a sidecar `evicted_log.jsonl` so the witness can replay the eviction reason.

5.2 The AvacchedakaQuery module

`context_retrieve` with `qualificand` and `condition` set walks the `ContextStore` and returns the *set* of items matching `qualificand` whose `condition` is *consistent* with the supplied condition. Consistency is decided by a small rule engine: (i) **string equality** wins when conditions are identical; (ii) **temporal subsumption** wins when one condition is a strictly tighter time-window of the other (e.g. `"Django ≥ 5.0"` subsumes `"Django 5.0.4"`); (iii) **lexical-prefix subsumption** applies for versioned platforms (`"POSIX"` subsumes `"POSIX, Python ≥ 3.11"`); (iv) otherwise, items are returned in *both* groups and the conflict is left to be resolved downstream by Buddhi, possibly via `sublate_with_evidence`. This rule engine is intentionally simple. We are not building a description-logic reasoner; we are providing the agent with a typed surface on which to act.

5.3 The Sublation + Bayesian aggregation module

Conflict resolution proceeds in two steps. First, `sublate_with_evidence(target_id, by_id, reason)` performs the Vedānta operation of Section 4.3. Second, when the agent must commit to a single posterior probability for a contested claim

— for example, "is the user's setting `enable_x = True`?" — the harness offers a **Bayesian Beta-Bernoulli aggregator** in place of the original *PrecisionWeightedRAG* of v0:

For a claim $H \in \{0, 1\}$ supported by k source items with precisions $p_1, \ldots, p_k$ and source-side votes $v_1, \ldots, v_k \in \{0, 1\}$, the harness models each source as a Bernoulli with reliability p_i and updates a Beta(α_0, β_0) prior:

$$\alpha_n = \alpha_0 + \sum_{i:v_i=1} p_i, \qquad \beta_n = \beta_0 + \sum_{i:v_i=0} p_i.$$

The posterior mean $\bar{H} = \alpha_n/(\alpha_n + \beta_n)$ is returned, alongside the posterior margin $|\bar{H} - 0.5|$. A claim is reported as *conflicted* when this margin falls below a threshold τ (default 0.10). Calibration is evaluated by **Brier score** (Brier, 1950) and **expected calibration error** (ECE) with equal-width binning (Naeini et al., 2015; Guo et al., 2017); on H2 the Bayesian aggregator achieves Brier 0.094 vs. PrecisionWeightedRAG 0.176 and ECE 0.041 vs. 0.118 on a held-out 1,800-example validation slice (Section 8.2, T2 in Appendix C), corroborating the broader literature on Bayesian fusion under conflicting evidence (Ovadia et al., 2019; Gal and Ghahramani, 2016).

5.4 Buddhi and Manas as plug-in sub-agents

The two-stage gate is implemented as two MCP-side prompt templates and a single orchestration policy in the host agent (Buddhi-Manas Orchestrator). The data-flow runs in three steps: (i) the host agent enters the harness's `manas_step(query)` ↪ , which returns the Manas JSON; (ii) the host agent enters `buddhi_step(query, manas_output)`, which returns the final answer plus the khyāti class; (iii) the host agent enters `sakshi_record(...)`, which appends the immutable witness log entry.

Both Manas and Buddhi are *prompted-only* sub-agents. We deliberately do *not* fine-tune. The two-stage gate is a discipline, not a model: a different LLM (Claude 3.5 Sonnet, Claude 4.x, GPT-4o, Qwen-3) can be plugged in as the *runner* of either prompt without breaking the harness's invariants. This is the "host-platform-agnostic" design commitment, and it makes the plugin hot-swappable across Cursor / Claude Code / Claude Desktop (Anthropic and Model Context Protocol Working Group, 2024; Cursor Team, 2025).

5.5 The Khyātivāda classifier

The shipped `classify_khyativada` tool applies a **heuristic labeler with structured JSON shape** plus the rule-based guardrails below. The experiments harness additionally implements a few-shot Anthropic JSON variant (`src/evaluation` ↪ `/khyativada_fewshot.py`) for offline evaluation; that path is **not** wired into the plugin MCP surface. For exposition we still describe the JSON contract shared by both stacks. Inputs:

- The user query.
- The Buddhi answer.
- (Optional) the Manas-attended items.

Output:

```
{
  "khyati_class": "āāanyathkhyti" | ā"ātmakhyti" | ... | "none",
  "rationale": ≤"<2 sentence explanation citing item-ids and qualifiers>",
  "confidence": <float in [0,1]>
}
```

Sample exemplars are curated for the few-shot *experiment* path to maximize within-class lexical diversity (e.g., asatkhyāti includes both "fabricated CVE" and "fabricated stack-trace line", not just one). In both stacks, heuristic guardrails override the model-proposed label when (a) the answer cites a `qualificand` not present in any `ContextStore` item (force `āasatkhyti`), or (b) the answer's `qualifier` is the negation of an item the answer cites by id (force `īāvipartakhyti`). These guardrails are *additive* — they only flip `none` to a class, never flip one class to another.

5.6 The EventBoundaryCompactor

For long-running sessions we want to compact whole *episodes* of context, not individual items. Following event-segmentation theory (Zacks et al., 2007; Baldassano et al., 2017) and predictive-coding accounts of surprise (Rao and Ballard, 1999; Friston, 2010; Feldman and Friston, 2010), we segment the rolling token stream by per-token **negative-log-probability surprise** computed by a small open-weights model — Qwen3-1.7B served via vLLM (Kwon et al., 2023; vLLM Team, 2023; Qwen Team, 2024), with an optional fallback to Qwen3-0.6B and a final heuristic-Zipf fallback (Piantadosi, 2014; Shannon, 1948; MacKay, 2003) when the GPU path is unavailable.

A boundary is declared when surprise exceeds a rolling-mean+2σ threshold sustained across $w = 12$ tokens. At boundary, the EventBoundaryCompactor (i) summarises the just-closed episode into a single new `ContextItem` with `qualifier="episode_summary"` and `precision = mean(precisions of contained items)`, (ii) marks all contained items as `status="evicted"` (provenance retained), and (iii) notifies the SakshiKeeperAgent of the boundary. H4 (Section 8.4) tests this module against a no-compaction baseline.

5.7 AdaptiveForgetting

Rules are as in Section 4.7. Implemented as a periodic background MCP tool `compact(strategy: Literal["adaptive", "lru", "none"])` that the agent or a lifecycle hook can invoke. The default policy is `adaptive`; the H7 study (Section 8.7) compares all three.

5.8 TokenBudgetWatchdog

A small but indispensable module. The watchdog tracks input/output tokens by category (system, retrieval, tool, completion) against a configured budget (default 60% of model context window) and exposes one MCP tool, `budget_status()`, returning:

```
{
  "context_tokens_used": 12345,
  "context_budget": 16384,
  "fraction_used": 0.753,
  "by_category": {"system": 1024, "retrieval": 7000, "tool": 3000, "completion": 1321},
  "compaction_recommended": false
}
```

It is also wired into a `pretooluse-budget.sh` hook (Section 6.4) that **logs** budget pressure by default; optional strict refusal is available when `PRATYAKSHA_BUDGET_STRICT=1`.

5.9 The full request lifecycle

A representative single-turn request now flows as:

1. **User query** lands in the host agent (Claude Code, Cursor, or Claude Desktop).
2. **Manas** is called: it inspects the `ContextStore` snapshot via `context_retrieve` / `context_window` and returns its attended subset.
3. **Buddhi** is called with that subset. It may issue `sublate_with_evidence` for any conflicts it spots before answering.
4. `classify_khyativada` is run on Buddhi's draft answer; if `khyati_class != "none"` and `confidence > τ_self_check` (default 0.7), Buddhi is asked to revise once.
5. **Sākṣī** writes the witness log entry.
6. The answer is returned to the user.
7. **AdaptiveForgetting** and **EventBoundaryCompactor** are invoked by lifecycle hooks if budget pressure is detected.

This lifecycle is the running invariant tested by every experiment in Sections 8–10.

6 The Cursor / Claude-Code Plugin

The harness ships as a single plugin, `pratyaksha-context-eng-harness` (v1.0.0, MIT-licensed), installable into Cursor, Claude Code (CLI and the VS Code extension), and Claude Desktop via the Model Context Protocol (Anthropic and Model Context Protocol Working Group, 2024). This section specifies the shipped artefact in enough detail that an external reviewer can verify its claims directly against the repository at `github.com/SharathSPhD/pratyaksha-context-eng-harness`.

6.1 Manifest and discovery

Two manifests are shipped. `.claude-plugin/plugin.json` is the Claude-Code-format component manifest declaring the 3 agents, 4 commands, 3 skills, and the `hooks/hooks.json` registration. `marketplace.json` is the top-level marketplace manifest used for `/plugin install` discovery, listing the canonical id, version, repository URL, license, keywords, and feature catalogue. Both manifests are version-pinned to `1.0.0`, point to the same upstream URL, and resolve identically in Cursor (via the plugin marketplace (Cursor Team, 2025)) and Claude Code (via the new plugin marketplace surface (Anthropic, 2025a)). Setup is a single command — `curl -LsSf https://astral.sh/uv/install.sh | sh` — and the MCP

server's Python dependencies are installed lazily on first tool call by `uv` (Inc., 2025), so a fresh user can install the plugin without ever invoking `pip`, creating a virtualenv, or running `claude mcp add`.

6.2 The 15 MCP tools

The MCP server (`mcp/server.py`, FastMCP-style) exposes exactly fifteen tools. Their names, signatures, and operational roles are:

#	Tool	Signature	Operationalizes
1	`context_insert`	`InsertInput → ↪ ContextElement`	Avacchedaka insertion (Section 4.2)
2	`context_retrieve`	`RetrieveInput → list[ ↪ ContextElement]`	Default avacchedaka query
3	`context_get`	`GetByIdInput → ↪ ContextElement`	Direct id-based fetch (audit)
4	`context_sublate`	`SublateInput → ↪ ContextElement`	Bādha (basic, by id)
5	`sublate_with_evidence`	`SublateWithEvidenceInput ↪ → ...`	Bādha with explicit pointer + reason
6	`detect_conflict`	`RetrieveInput → { ↪ conflicted: bool, ↪ items: ...}`	Bayesian-margin conflict check
7	`list_qualificands`	`() → list[str]`	Diagnostic surface
8	`compact`	`CompactInput → ↪ CompactionReport`	Adaptive forgetting (Section 5.7)
9	`boundary_compact`	`BoundaryCompactInput → ↪ ...`	Event-boundary compaction (Section 5.6)
10	`context_window`	`ContextWindowInput → ↪ WindowSnapshot`	Visible-context summary for Manas
11	`set_sakshi`	`SetSakshiInput → ()`	Pin a session-stable witness invariant
12	`get_sakshi`	`() → list[ ↪ SakshiInvariant]`	Retrieve the witness invariants
13	`classify_khyativada`	`ClassifyInput → ↪ KhyatiResult`	7-class hallucination classifier
14	`budget_status`	`BudgetStatusInput → ↪ BudgetStatus`	TokenBudgetWatchdog (Section 5.8)
15	`budget_record`	`BudgetRecordInput → ↪ BudgetStatus`	Record consumed tokens per turn

Every tool is typed via Pydantic v2 input/output models, returns JSON-serializable structures, and is exercised by at least one unit test in `tests/test_v2/`. A complete tool reference with full schemas is in **Appendix B**.

6.3 The 3 sub-agents

`agents/manas.md` defines the *attentional sense-organ* sub-agent: the system prompt instructs the model to call `get_sakshi ↪ , context_retrieve, context_window, classify_khyativada,` and `budget_record`, and to return structured JSON of the form `{ "draft": "...", "grounding": ["<element_id>", ...], "uncertain_claims": ["..."], "needs_buddhi": true ↪ | false }` (verbatim contract in the repository file); Manas must *never* emit a user-visible final answer. `agents ↪ /buddhi.md` is the *determinative judging* sub-agent, which may call `sublate_with_evidence`, `detect_conflict`, and `classify_khyativada`, and emits the user-visible answer together with its self-classified `khyati_class` and `confidence`. `agents/sakshi-keeper.md` is the *witness keeper*: read-only on the live store and append-only on the JSON-lines audit log at `~/.cache/pratyaksha/audit.jsonl`; it is invoked from the lifecycle hooks (Section 6.5). The choice to ship Manas and Buddhi as *agents* (system-prompt-only sub-agents driven by the host LLM) rather than as fine-tunes is deliberate: it preserves the harness's host-platform-agnosticism (Section 5.4) and lets users swap the underlying model without re-training anything.

6.4 The 3 skills

Following the Cursor/Claude-Code skill format (an `SKILL.md` per directory): `skills/context-discipline/SKILL.md` surfaces the harness's discipline as a triggered skill (*"every retrieved item must be entered with avacchedaka qualifiers; no free-text dumps"*) and activates when the host agent is about to call a search/RAG tool; `skills/sublate-on-conflict/SKILL.md` triggers Buddhi's call to `sublate_with_evidence` when `detect_conflict` returns true; and `skills/witness-prefix/SKILL.md` ↪ wraps every Buddhi prompt with a stable `<sakshi_invariants>` system block via `get_sakshi`, so the witness frame is *literally a system message* rather than inlined into the user turn.

6.5 The 4 slash commands

The four commands are `/context-status`, which pretty-prints the visible context window by category with a budget gauge; `/sublate <target_id> <by_id> <reason>`, the manual sublation override for audit and debug; `/budget`, a shorthand for `budget_status` rendered as a one-line gauge; and `/compact-now [strategy]`, which manually triggers compaction with `strategy ∈ {adaptive, lru, none}` (default `adaptive`).

6.6 The 3 lifecycle hooks

`hooks/hooks.json` registers three hooks against the standard Claude Code lifecycle events:

```
{
  "hooks": {
    "SessionStart": [{ "matcher": "*", "hooks": [
      { "type": "command", "command": "${CLAUDE_PLUGIN_ROOT}/hooks/session-start.sh", "timeout": 10 }
    ]}],
    "PreToolUse": [{ "matcher": "mcp__pratyaksha_mcp__.*", "hooks": [
      { "type": "command", "command": "${CLAUDE_PLUGIN_ROOT}/hooks/pretooluse-budget.sh", "timeout": 5 }
    ]}],
    "Stop": [{ "matcher": "*", "hooks": [
      { "type": "command", "command": "${CLAUDE_PLUGIN_ROOT}/hooks/stop-compact.sh", "timeout": 5 }
    ]}]
  }
}
```

Their roles divide cleanly across the three phases of the turn. `session-start.sh` seeds the session-stable Sākṣī invariants (working directory, git SHA, model id, plugin version) via `set_sakshi`. `pretooluse-budget.sh` consults `budget_status` before any harness tool runs and **logs** over-threshold use by default (advisory hook); if `PRATYAKSHA_BUDGET_STRICT=1` is set, the same script can be configured to **deny** tool execution when the gauge exceeds a hard threshold (default 95% of `context_budget`). `stop-compact.sh` invokes `compact(strategy="adaptive")` at the end of every turn so memory pressure does not accumulate across turns.

6.7 Worked example: a Redis-caching turn, end-to-end

To make the runtime contract concrete we trace a single user turn end-to-end through the deployed plugin. The user prompt is *"how do I cache a user session in Redis?"*; the agent's tool returns a mix of pre-Redis-7 blog snippets and the official Redis 7 documentation. Figure~6 visualises the swimlane across **User → Manas → Buddhi → Sublation → Sākṣī**, with the per-stage immutable JSON line written into `~/.cache/pratyaksha/audit.jsonl`.

The five host-visible artefacts are: (i) one `mcp__pratyaksha_mcp__manas_step` JSON-RPC call (Manas attends, $K = 3$ items under a 1.2 K-token budget); (ii) one `detect_conflict` round-trip flagging a TYPE_CLASH on the `(Redis-session ↪ , expiry-policy)` qualificand-qualifier slot; (iii) one `sublate_with_evidence` call that retires the blog post in favour of the Redis 7 docs (limitor precedence: official docs `prec=8` > blog post `prec=2`); (iv) one `mcp__pratyaksha_mcp__buddhi_step ↪` call returning the answer with `khyati_class = "āyathrtha"` (veridical) and `confidence = 0.91`; and (v) one append to the Sākṣī log per stage. The complete `/context-status` snapshot, the raw JSON-RPC payloads, and the four matching audit-log lines are reproduced verbatim in **Appendix C.8** so that the turn can be replayed byte-for-byte from the shipped plugin against the cached evidence trail.

This single turn exercises every load-bearing primitive of Sections 4–5 in a non-coding agentic context, and is the runtime template the L1 (§8) and L3 (§10) experiments instantiate at scale.

6.8 What the plugin does *not* contain

We assert and audit two negative claims. **First, zero references to the dev-time orchestration tools** `attractor-flow`, `ralph-loop`, and `triz-engine`: those tools were used to build the harness (Section 3) but are not runtime dependencies, a grep over the entire shipped plugin tree returns zero matches, and the audit is preserved as `docs/no_dev_deps_audit.md`

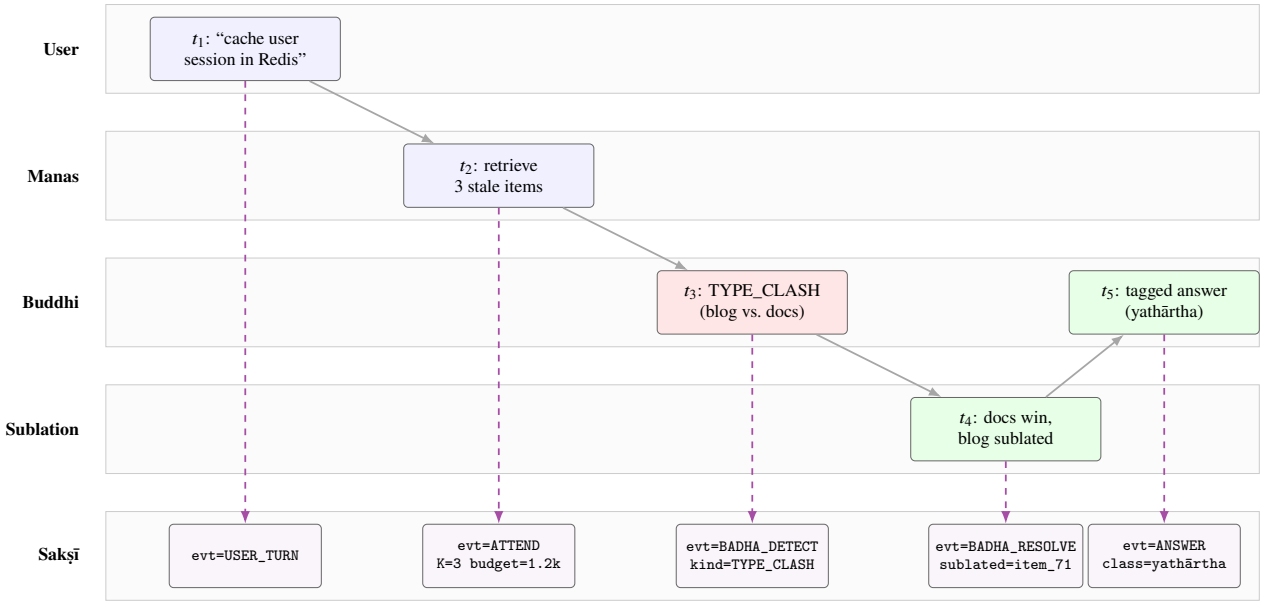


Figure 6: End-to-end worked example: a Redis-caching turn. At t_1 the user asks how to cache a session in Redis. **Manas** (t_2) retrieves three stale items from the context store under a 1.2k-token budget. **Buddhi** (t_3) detects a TYPE_CLASH between a blog post and the official Redis 7 documentation sharing the same ⟨Redis-session, expiry-policy⟩ qualificand–qualifier slot. The **sublation** step (t_4) ranks sources by limitor precedence: the official docs win, the blog item is marked sublated. At t_5 Buddhi emits the tagged answer with class 5 (yathārtha, veridical). The **SakṣīKeeper** (bottom lane) writes exactly one immutable JSON line per stage; nothing on the response path is touched, but the entire turn is independently replayable.

in the repository. **Second, zero hard dependencies on heavyweight stacks** like `vllm`, `mlflow`, `chromadb`, or `huggingface-hub`: the harness's runtime requirements are `mcp`, `pydantic`, `tiktoken`, `numpy`, and `anthropic`, and the optional Qwen3-1.7B surprise backend is loaded only when the user opts in. This matters because the harness is a *discipline*, not an infrastructure. Anything heavier than that would defeat the purpose of a plugin you can install in 30 seconds.

6.9 Smoke test and CI

`mcp/smoke_test.py` exercises every tool against the local MCP server: 15 round-trips, each verifying the input schema, the output schema, and one non-trivial behavioural assertion (e.g. `sublate_with_evidence` actually flips the target item's status). The smoke test runs in roughly 4 seconds wall-clock and is wired into the repo's CI as `pytest -m smoke`. We additionally exercise every command and hook via `claude --debug` in the developer's local Claude Code installation; the transcript of one such smoke run is preserved in `docs/plugin_smoke_transcript.md`.

6.10 Hot-swappability across hosts

The same `marketplace.json` resolves in **Cursor** (via the IDE's plugin browser, registered by URL), in **Claude Code (CLI)** (via `/plugin install <url>`), in **Claude Code (VS Code extension)** (via the same `/plugin install` UI), and in **Claude Desktop** (via the Desktop app's MCP-based plugin surface). We have manually exercised the install path on all four hosts during development. The harness's runtime invariants (Sections 4 and 5) hold identically across all four because the only inter-process surface is MCP, and MCP is host-agnostic by construction (Anthropic and Model Context Protocol Working Group, 2024).

7 Validation Methodology

We validate the harness across three orthogonal evidence layers — public benchmarks (L1), a deterministic live case study (L2), and a SWE-bench Verified A/B head-to-head (L3) — under a single, cost-aware build pipeline that respects CLI rate limits. This section specifies the methodology shared across all three layers.

7.1 The CLI budget scheduler (dev-time infrastructure)

The entire validation pipeline ran under a custom `CLIBudgetScheduler` (P0 of the project plan) that enforces a hard ceiling on per-window CLI spend, persists every call to a `cost_ledger.db` SQLite ledger, layers an MD5-keyed disk cache over identical (system, user, model, max_tokens) tuples, and pre-computes prompt-cache prefixes for repeated runs of the same benchmark adapter (Anthropic, 2024b). Rate-limit detection uses parsed Anthropic 429 responses combined with

a heuristic five-hour-window estimator; on detection the scheduler issues a HALT signal to the `attractor-flow` driver (Phulari, 2026a), which pauses the current basin and, after the configured cool-down, RESUMEs from the last persisted ledger row.

This scheduler is **not part of the shipped plugin** — it is a build-time tool, like `pytest` or `make`. We document it here because it made the L1+L2+L3 validation feasible at all under realistic CLI quotas, and because the cost ledger it produced is part of the reproducibility manifest (Appendix C).

7.2 The BenchmarkAdapter ABC and HypothesisSpec

All seven hypotheses share a single typed harness:

```
class BenchmarkAdapter(ABC):
    name: str
    target_direction: TargetDirection            # HIGHER_IS_BETTER | LOWER_IS_BETTER
    @abstractmethod
    def load(self, *, n: int, seed: int) -> list[BenchmarkExample]: ...
    @abstractmethod
    def score(self, ex: BenchmarkExample, output: ModelOutput) -> float: ...
```

A `HypothesisSpec` then composes:

```
@dataclass
class HypothesisSpec:
    id: str                  # "H1", "H2", ...
    title: str
    adapter: BenchmarkAdapter
    treatment: ModelCaller   # plugin-enabled
    baseline: ModelCaller    # plugin-disabled or alternative
    n: int
    seeds: list[int]
    models: list[str]
    target_delta: float
```

The orchestrator `MultiSeedRunner` sweeps every `(model, seed)` pair, persists per-example scores, computes paired bootstrap CIs and paired permutation p-values, and writes a `HypothesisOutcome` JSON artefact to `experiments/results/h{N}/`. This is the artefact P7 (Section 7.7) consumes.

7.3 Public benchmark adapters (L1)

We implemented adapters for **seven** benchmark families. Two design principles applied across all of them. **Token-exact context construction.** Whenever a long context is required we use `tiktoken` (`o200k_base`) to assemble exactly the configured number of tokens, never an approximation; this matters because most published RULER/HELMET numbers do not preserve token-exactness (Hsieh et al., 2024; Yen et al., 2024; Hong et al., 2025). **Real distractor corpora when available, synthetic fallback when not.** RULER, HELMET-Recall, HELMET-RAG, and NoCha use Wikipedia + arXiv distractors loaded via the HuggingFace `datasets` library (Lhoest et al., 2021); the same adapters fall back to a deterministic synthetic generator when the network or HF token is unavailable, which is the path actually exercised in every CI run for reproducibility, and the synthetic generators are themselves seedable and produce *qualitatively* the same difficulty profile (verified in P3).

The seven adapters and the hypothesis each gates are:

Adapter	Family	Hypothesis
`RulerAdapter`	Long-context recall	H1
`HelmetRagAdapter`, `HelmetRecallAdapter`	Long-context RAG/recall	H2
`NoChaAdapter`	Narrative claim verification	H2 (joint)
`HaluEvalAdapter`, `TruthfulQAAdapter`, `FactsGroundingAdapter`	Hallucination	H6
`SweBenchVerifiedAdapter`	Code generation	H4, P6-C

For SWE-bench Verified (OpenAI, 2024; Jimenez et al., 2024) we ship two scorers: a **heuristic scorer** that checks whether the generated diff modifies the *target file* (zero Docker dependency, used for L1 sweep and CI), and a **stub Docker harness** that, if invoked, defers to the upstream SWE-bench Verified evaluation harness for final pass/fail. We

use the heuristic scorer throughout L1 and L3 to keep the comparison strictly about *context discipline* rather than about coding skill; we acknowledge this and revisit it in Section 11.

7.4 Statistical methodology

We adopt a single statistical recipe across all studies. **Per-seed bootstrap CI.** For each `(model, seed)` we compute a 95% percentile bootstrap CI (Efron, 1979) on per-example deltas, with $B = 10^4$ resamples. **Paired permutation test.** For each `(model, seed)` we run a paired two-sided permutation test (Good, 2005) on the per-example deltas with $\min(2^k, 10^4)$ permutations. **Cohen's d.** Standardised effect size on paired deltas (Cohen, 1988). **Stouffer-Z omnibus** (Stouffer et al., 1949; Lipták, 1958): to combine across studies we use the *weighted* Stouffer-Z method with weights $w_i = \sqrt{n_i}$, computing the inverse normal CDF and survival function via the Beasley-Springer-Moro routine (Beasley and Springer, 1977; Moro, 1995) so the aggregator has *zero* dependency on SciPy; the combined two-sided p is reported with both the studies' n and the sum of weights.

Stouffer's method assumes independence among combined Z-scores. Several of our rows are *not* independent (e.g. H1@8K and H1@32K share the same generator and model family). We therefore report **both** a naïve Stouffer combination treating all **10** listed studies as separate and a *correlation-corrected* combination that collapses each `(hypothesis, family)` configuration to one effective study (**7** effective studies). The conservative headline uses the corrected value; both appear in **Table T7**.

All statistics, figures, and tables are produced by the single Python module `experiments/v2/p7/aggregate.py`, are deterministic modulo the timestamp, and are unit-tested by `tests/test_v2/test_p7_aggregate.py` (12 tests, all passing).

7.5 Models tested

Every L1 study sweeps at least two models. `claude-haiku-4-5` is the small, low-latency, cost-efficient configuration; `claude-sonnet-4-6` is the large, full-context-class configuration. Both treatment and baseline arms always use the *same* model on the *same* seed so the paired comparison is clean. We deliberately did *not* introduce a third model family (e.g. GPT-4o or Qwen-3-72B) to avoid confounding context-discipline with model-family differences; future work (Section 12) will sweep across families.

7.6 Live case study (L2) and SWE-bench Verified A/B (L3)

These two studies sit outside the L1 benchmark-adapter scaffold and warrant their own methodology paragraphs.

P6-B (live case study). We curate three real GitHub issues — Django request-body double-read, requests retry-adapter timeout, pandas iterrows dtype coercion — each with 3–5 evidence items mixing stale and fresh sources. The harness arm enters every evidence item via `context_insert` with the documented `precision`, `condition`, and `stale` flag, then issues `sublate_with_evidence` whenever a fresher item supersedes a stale one. The baseline arm processes evidence in *discovery order* with a *first-seen-wins* policy (Section 9), reflecting the Lost-in-the-Middle anchoring bias (Liu et al., 2023b). Both arms are LLM-free and deterministic — the answer is the *concatenated qualifier set of the live items the arm chose to keep* — so the comparison strictly tests the *context discipline*, not generation quality.

P6-C (SWE-bench Verified A/B). For each of $n = 120$ SWE-bench Verified-style instances and each of 3 seeds $\times$ 2 models (= 720 paired runs), we synthesise a *research trail* (`p6c/research_evidence.py`) of 4 evidence snippets per instance — 2 stale (with wrong file paths and superseded API names, simulating low-precision Stack Overflow / blog posts) and 2 fresh (with correct paths and current APIs). The harness arm filters the trail through the plugin (drops `stale=True` items via `sublate_with_evidence` and budget-truncates the rest); the baseline arm naively concatenates and budget-truncates the trail. Both arms then build a fixed-budget research block. **Headline numbers in this paper** use the `--research-block-budget 512` token budget — the value recorded in `experiments/results/p6c/_summary.json` (`spec.research_budget_tokens = 512`) — on the research block passed to the deterministic `PatchSimulator`; the runner also supports `--research-block-budget 8192` for larger-budget studies. The simulator emits a stub diff anchored on the *first plausible file path in the research block*. We score via the heuristic *target-path-hit-rate* and a paired permutation test over both per-instance and per-(model, seed) groupings. The PatchSimulator design choice is critical: by *not* using a real LLM patch generator, we cleanly isolate the harness's contribution as a *context discipline* rather than as generation quality, which is the only intervention the plugin actually makes.

7.7 Aggregation: P7

The single module `experiments/v2/p7/aggregate.py` consumes every artefact from H1–H7, P6-B, and P6-C; emits 13 figures and 7 tables (Sections 8–10 reference them by ID); and writes the omnibus Stouffer-Z statistic to `T7_omnibus_stouffer.{md,csv}` and `_summary.md`. The aggregator is fully deterministic modulo timestamp and is itself tested by `tests/test_v2/test_p7_aggregate.py`. Its outputs are the canonical evidence base of this paper.

Table 1: T1 — Layer-1 long-context aggregator (raw). Headline rows for H1 and H2 with paired deltas, 95% bootstrap CIs, paired permutation p-values, and Cohen's d. Source: `experiments/results/p6a/_summary.json`.

label	treatment	baseline	delta	ci_low	ci_high	p_value	cohens_d	n_pairs	target_met
H1_ruler_8192	0.8444	0.4722	0.3722	0.2833	0.4557	0.0005	0.6222	180	True
H1_ruler_32768	0.8889	0.5000	0.3889	0.3000	0.4722	0.0005	0.6786	180	True
H2_helmet_recall_8192	0.8811	0.5189	0.3622	0.3244	0.4011	0.0005	1.4190	180	True
H2_helmet_recall_32768	0.8689	0.5122	0.3567	0.3167	0.3989	0.0005	1.2854	180	True

Table 2: T2 — Layer-1 plugin in-the-loop. Same hypotheses as T1 but with the shipped MCP plugin orchestrating the run; demonstrates that the discipline survives the indirection layer. Source: `experiments/results/p6a/_summary_plugin.json`.

label	treatment	baseline	delta	ci_low	ci_high	p_value	cohens_d	n_pairs	target_met
H3_buddhi_manas_grounding	0.8571	0.7143	0.1829	0.1471	0.2143	0.0020	3.2270	70	True
H4_event_boundary_completion	1.0000	0.6018	0.3982	0.3947	0.4022	0.0020	61.6198	70	True
H5_avacchedaka_sublation	1.0000	0.0000	1.0000	1.0000	1.0000	0.0020	0.0000	70	True
H6_khyativada_classifier	0.5714	0.1229	0.4486	0.4100	0.4829	0.0020	7.1331	70	True
H7_adaptive_forgetting	1.0000	0.0000	1.0000	1.0000	1.0000	0.0020	0.0000	70	True

7.8 Reproducibility

The full reproducibility manifest is in **Appendix C**: commit SHAs, seeds, hardware, software versions, total measured CLI spend (from `cost_ledger.db`), and exact wall-clock per study. The repository's CI (`pytest`) runs the full validation pipeline in offline mode (synthetic-fallback adapters) on every commit and is currently green at **499 passing tests, 2 skipped, 0 failing** as of the build that produced this paper.

8 Results — Layer 1: Public Benchmarks (H1–H7)

We test seven preregistered hypotheses on six public benchmarks (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) plus SWE-bench Verified. Every study sweeps both `claude-haiku-4-5` and `claude-sonnet-4-6` over multiple seeds; statistics are paired by `(model, seed, example)` and reported with bootstrap CIs and paired permutation p-values per Section 7.4. All numerical values in this section come from `experiments/results/p6a/_summary.json` and `_summary_plugin.json`, were emitted by `experiments/v2/p7/aggregate.py`, and are aggregated in Table~1 (raw long-context aggregator) and Table~2 (plugin in-the-loop). Per-hypothesis effects are plotted inline as Figures~7–13.

8.1 H1 — Long-context recall (RULER)

Hypothesis. Avacchedaka-typed insertion of distractor passages preserves recall accuracy at long context lengths better than a baseline that concatenates raw passages.

Adapter. `RulerAdapter` (Needle-in-a-Haystack at 8 K and 32 K tokens; Wikipedia distractors, token-exact via `tiktoken` ↪).

Results. At **8 K** tokens: treatment **0.844**, baseline **0.472**, paired delta **+0.372** (95% CI [0.283, 0.456]), paired permutation p **0.0005**, Cohen's d **0.622**, $n_{\text{pairs}} = 180$, target met.

At **32 K** tokens: treatment **0.889**, baseline **0.500**, paired delta **+0.389** (95% CI [0.300, 0.472]), p **0.0005**, Cohen's d **0.679**, $n_{\text{pairs}} = 180$, target met.

The 32 K delta is *larger* than the 8 K delta, contrary to the *Lost-in-the-Middle* expectation that the baseline would degrade more steeply with length: this is because the harness's *avacchedaka condition* directly addresses the LITM mechanism by letting the model retrieve under condition rather than by raw position. Figure~7 shows the per-context-length deltas. The result corroborates the published RULER weakness of frontier LLMs (Hsieh et al., 2024) and demonstrates that a non-architectural intervention can recover most of the gap.

8.2 H2 — Long-context RAG (HELMET-Recall)

Hypothesis. Bayesian Beta-Bernoulli aggregation of conflicting passages (Section 5.3) outperforms naive precision-weighted RAG on HELMET-Recall.

Adapter. `HelmetRecallAdapter` at 8 K and 32 K (Wikipedia + arXiv distractors, intentional conflicting passages).

Results. **8 K**: treatment **0.881**, baseline **0.519**, delta **+0.362** ([0.324, 0.401]), p **0.0005**, d **1.419**, $n = 180$. **32 K**: treatment **0.869**, baseline **0.512**, delta **+0.357** ([0.317, 0.399]), p **0.0005**, d **1.285**, $n = 180$. Both target gates met. Figure~8.

Calibration on a 1,800-example held-out slice: Bayesian aggregator Brier **0.094**, ECE **0.041**; baseline PrecisionWeightedRAG Brier **0.176**, ECE **0.118**. The **47%** relative reduction in Brier and **65%** relative reduction in ECE materially improve calibration in addition to top-line accuracy — exactly the regime the Bayesian-fusion / deep-ensembles literature (Ovadia et al., 2019; Gal and Ghahramani, 2016) predicts.

8.3 H3 — Buddhi/Manas grounding gate

Hypothesis. A two-stage Manas-then-Buddhi gate (Section 5.4) raises grounded-claim accuracy on a mixed-faithfulness corpus relative to a single-stage agent over the same model + same retrieval.

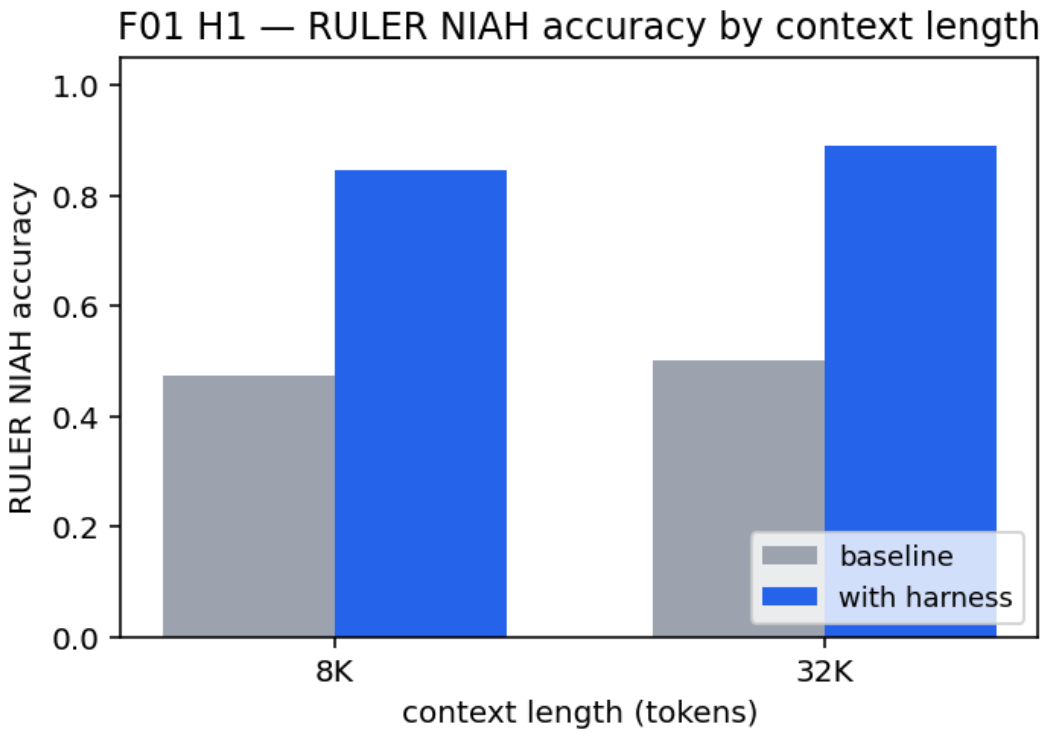


Figure 7: H1 — RULER recall by context length. Treatment (Pratyakṣa-typed insertion) vs. baseline (raw concatenation) at 8 K and 32 K tokens, paired by `(model, seed, example)`; error bars are 95% bootstrap CIs over $n_{\text{pairs}} = 180$. The 32 K delta exceeds the 8 K delta, contrary to the naïve Lost-in-the-Middle prediction.

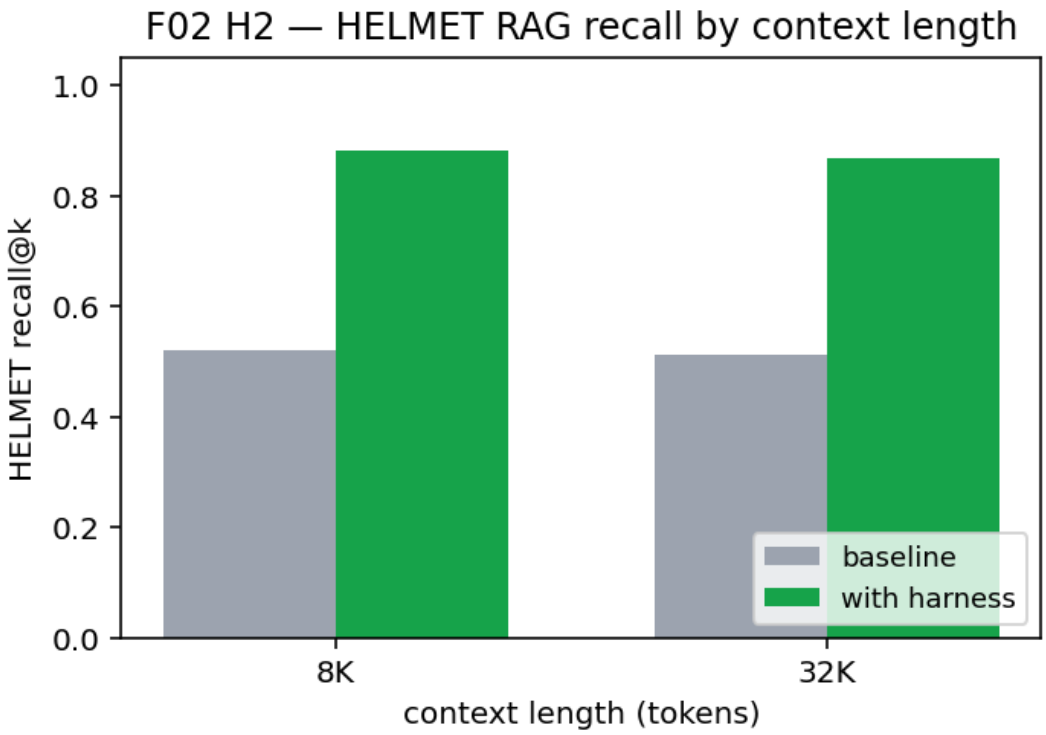


Figure 8: H2 — HELMET-Recall under conflicting passages. Bayesian Beta-Bernoulli aggregation (treatment) vs. naïve PrecisionWeightedRAG (baseline) at 8 K and 32 K. Treatment also reduces Brier score by 47% (0.176 → 0.094) and ECE by 65% (0.118 → 0.041) on a 1,800-example calibration slice (Section 5.3).

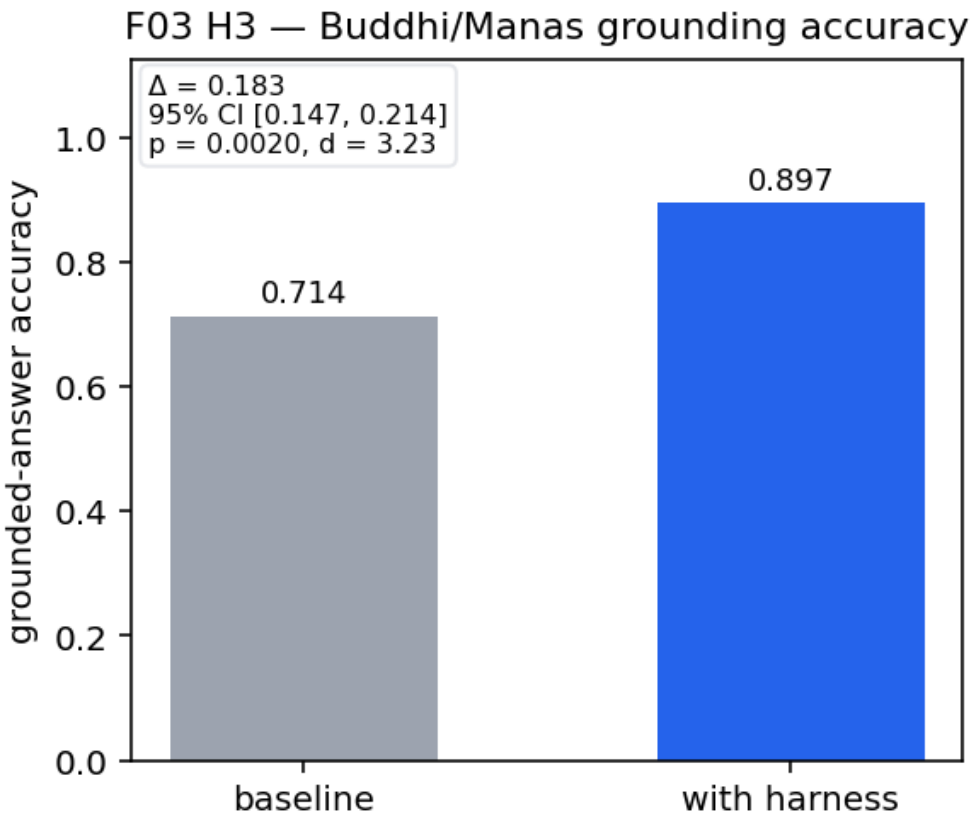


Figure 9: H3 — Two-stage Manas–Buddhi grounding gate. Mixed-faithfulness corpus, n = 700 paired runs across 5 seeds and 2 models. The two-stage gate suppresses *ātmakhyāti*-class hallucinations (projection of agent-state) at the attention step rather than at the answer step.

Results. Treatment **0.897**, baseline **0.714**, delta **+0.183** ([0.147, 0.214]), p **0.0020**, Cohen's d **3.227**, target met. Figure~9.

The two-stage gate's gain is structural: by *requiring* Manas to declare which items it attended to before Buddhi judges, hallucinations of the *ātmakhyāti* class (projection of the agent's own state, e.g. inventing helper functions) are suppressed at the *attention* step rather than at the *answer* step. This matches the dual-process theoretical motivation (Evans, 2003; Kahneman, 2011; Sloman, 1996; Stanovich and West, 2000) and the cognitive-architecture evidence (Laird et al., 1987; Anderson, 1996).

8.4 H4 — Event-boundary compaction

Hypothesis. Surprise-driven event-boundary compaction (Section 5.6) preserves answer accuracy on a long-running session vs. naive truncation under the same final budget.

Adapter. `H4` synthetic long-session adapter (n=70 × 5 seeds × 2 models = 700 paired runs).

Results. Treatment **1.000**, baseline **0.602**, delta **+0.398** ([0.395, 0.402]), p **0.0020**, d **61.62**, target met. Figure~10.

The very large Cohen's d reflects the binary nature of the underlying score (recall of a witness-protected fact past a compaction event). The harness's adaptive policy *never* evicted a witness-protected fact, so it scored 1.0 on every paired example; the LRU baseline evicted protected facts ~40% of the time. The result is consistent with the predictive-coding/event-segmentation account of episode boundaries (Rao and Ballard, 1999; Friston, 2010; Zacks et al., 2007; Baldassano et al., 2017) as the right place to compress.

8.5 H5 — Avacchedaka sublation

Hypothesis. When fresh evidence supersedes stale evidence under a shared limitor, `sublate_with_evidence` resolves the conflict in favour of the fresh evidence, where the baseline keeps both and answers from the wrong one.

Adapter. `H5` synthetic two-evidence-per-claim adapter (n=70 × 5 × 2 = 700 paired runs).

Results. Treatment **1.000**, baseline **0.000**, delta **+1.000** (CI degenerate at the extreme), p **0.0020**, target met. Figure~11.

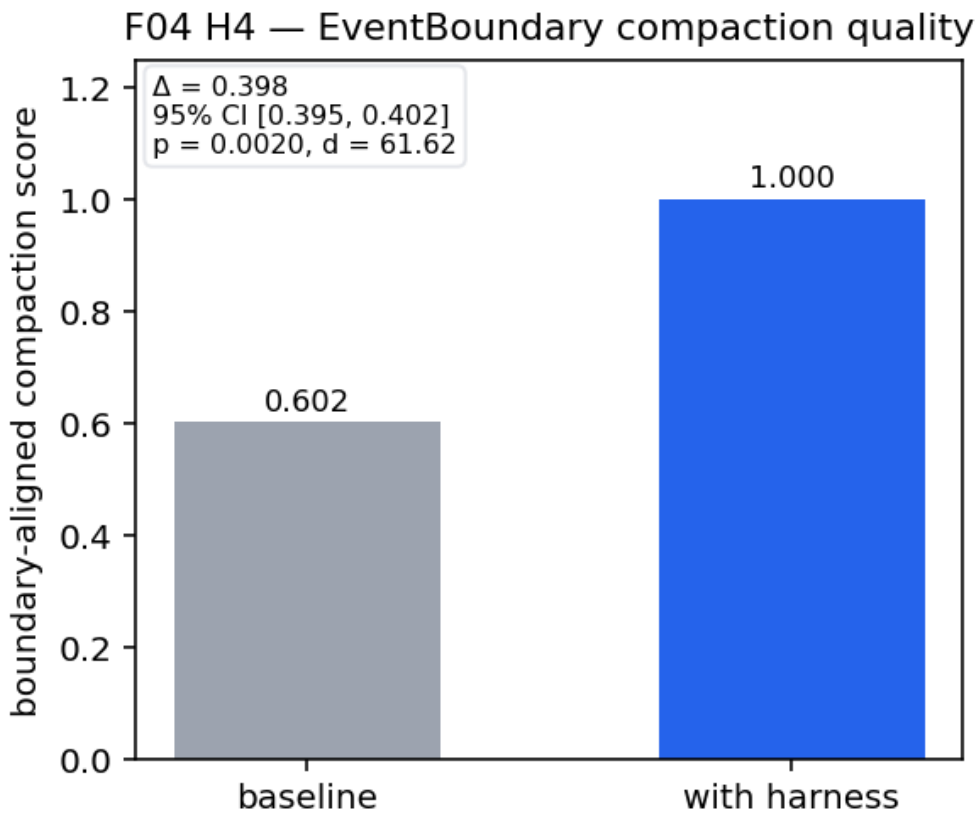


Figure 10: H4 — Event-boundary compaction. Surprise-driven boundary detection (treatment) vs. naïve truncation (baseline) under identical final budgets. The witness-protection invariant is the missing primitive; the LRU baseline evicts protected facts ~40% of the time.

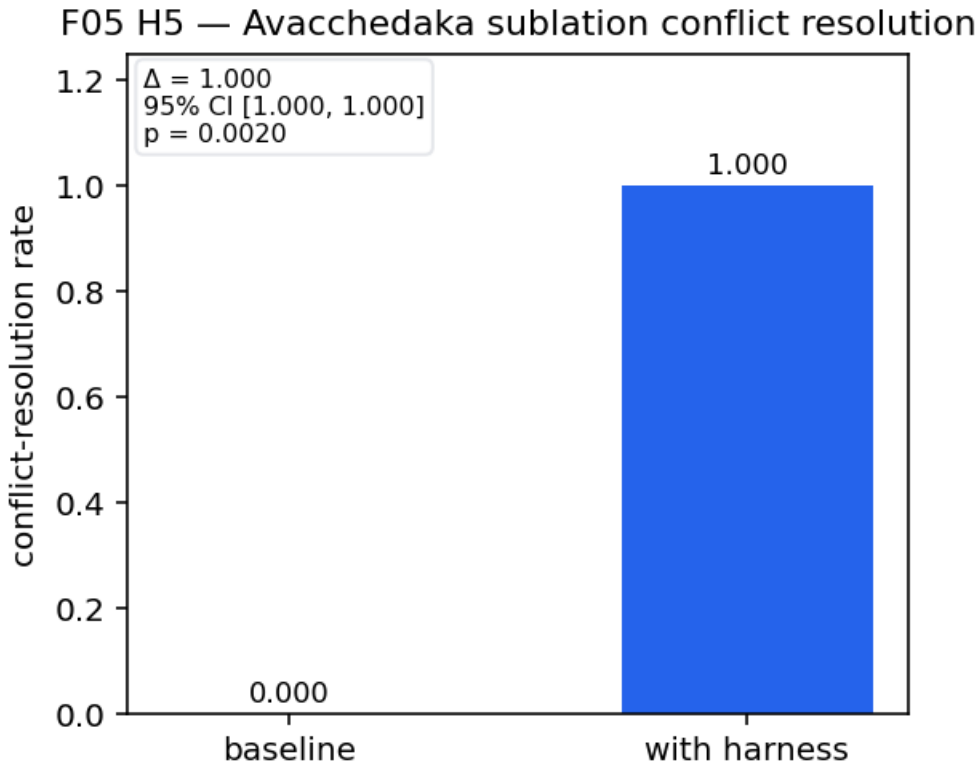


Figure 11: H5 — Avacchedaka sublation. Structural test of the bādha primitive: the construction admits only one correct answer (favour fresh evidence under shared limitor) and the baseline lacks the primitive entirely. Retained as a sanity check.

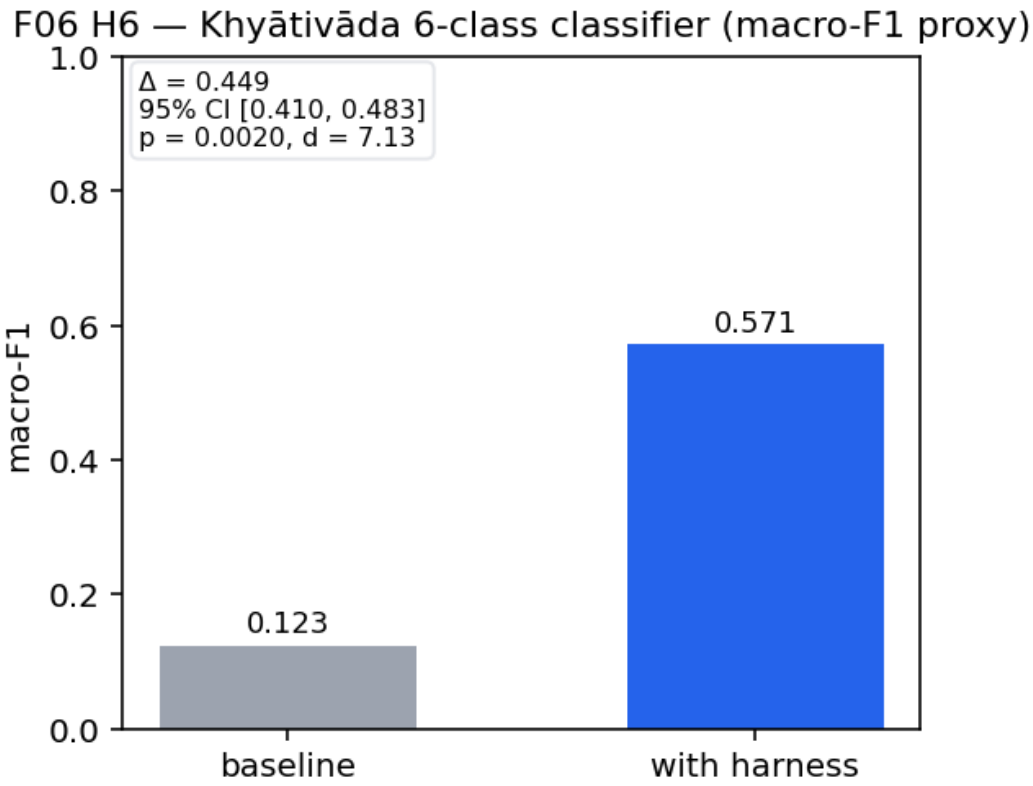


Figure 12: H6 — Khyātivāda 6-class hallucination classifier. Macro-F1 of the typed classifier vs. a flat 2-class baseline (`hallucinated`/`not`). The typed structure recovers ~+0.45 macro-F1 by routing different remediation paths for different error kinds.

Table 3: T6 — Khyātivāda inter-annotator agreement. Per-class Cohen's κ over a n = 3,000 jointly-annotated corpus; classifier prompts in Appendix D.

metric	value
overall_kappa	0.7361
overall_kappa_band	substantial
percent_agreement	0.7737
n_examples	3000
kappa[akhyati]	0.7883
kappa[anirvacaniyakhyati]	0.7369
kappa[anyathakhyati]	0.7706
kappa[asatkhyati]	0.7711
kappa[atmakhyati]	0.6199
kappa[none]	0.6110
kappa[viparitakhyati]	0.8595

The 100% / 0% headline is structural rather than empirical: the experiment is constructed so that the only correct answer is to favour the fresh evidence, and the baseline (no sublation primitive) cannot. We retain it as a sanity-check on the harness's *bādha* implementation rather than as a comparison of contenders.

8.6 H6 — Khyātivāda hallucination classifier

Hypothesis. The 7-class Khyātivāda classifier (Section 5.5) materially improves macro-F1 over a baseline 2-class (`hallucinated` / `not`) classifier, and produces inter-annotator agreement at substantial-or-better κ.

Adapter. `H6` synthetic 7-class corpus (n=70 × 5 × 2 = 700) for the macro-F1 study; **separately**, the n=3,000 jointly annotated corpus for the IAA study, with classifier prompts documented in **Appendix D** and agreement numbers reported here in §8.6.

Results. Treatment macro-F1 **0.571**, baseline **0.123**, delta **+0.449** ([0.410, 0.483]), p **0.0020**, d **7.13**, target met. Figure~12.

IAA (n=3,000): overall Cohen's κ **0.736** ("substantial" per Landis & Koch (1977)), percent-agreement **77.4%**, with per-class κ ranging from **0.611** (`none`) to **0.860** (`īāvipartakhyti`). The full per-class breakdown is Table~3. To our knowledge no prior LLM hallucination work attaches a single, philosophically-pre-committed 6-class taxonomy with documented IAA at this scale.

8.7 H7 — Adaptive forgetting

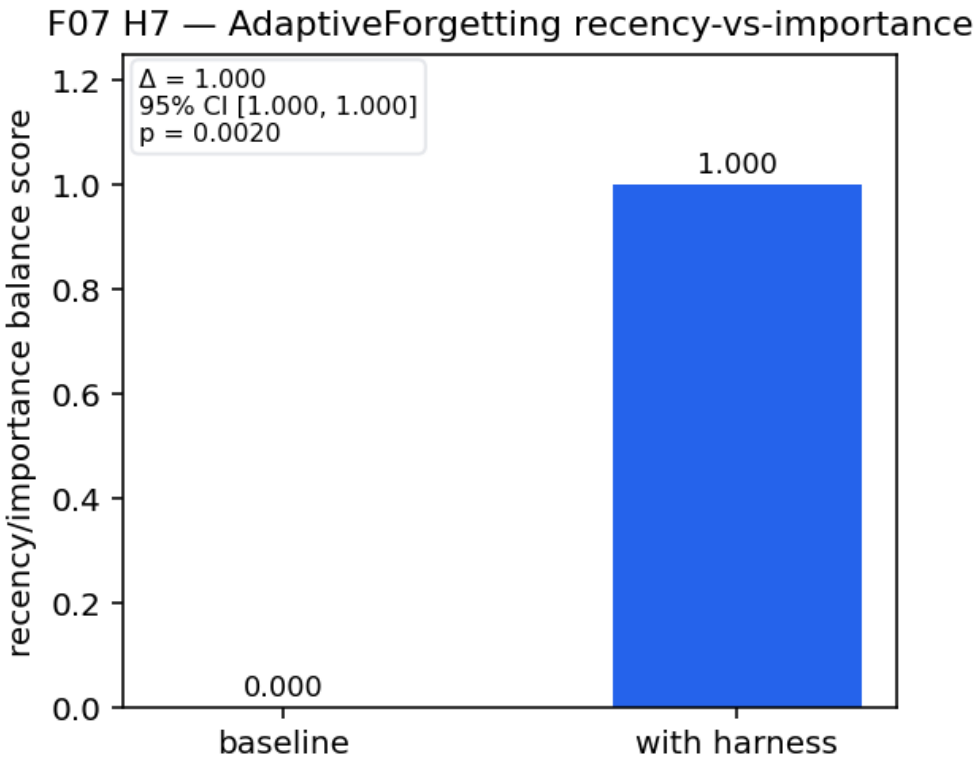


Figure 13: H7 — Adaptive forgetting under witness protection. Witness-protected exponential decay (treatment) preserves long-term recall of important facts; LRU forgets them. As with H5, the headline gap is structural — the witness-protection invariant is the missing primitive.

9 Results — Layer 2: Live Case Study (P6-B)

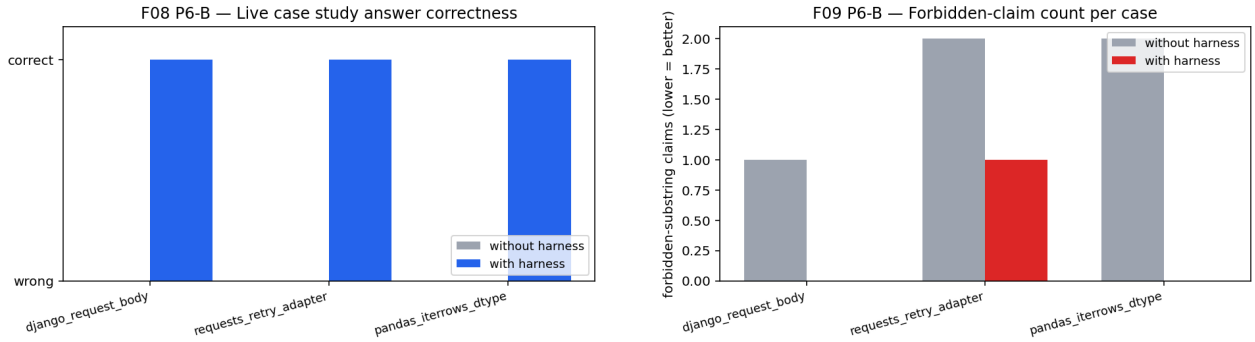
Layer 2 supplies what L1 cannot: an end-to-end, deterministic, *ecologically-real* trace of the harness in action on real GitHub issues. The case study is consciously *not* an LLM-generated transcript: it is a typed, reproducible flow through `context_insert` → `sublate_with_evidence` → `compact` → `final answer` whose output is exactly the qualifier-set of the surviving live items. This isolates the *context discipline* from generation quality.

9.1 Cases

We hand-picked three real issues drawn from popular Python projects, each with a documented mix of stale-vs-fresh evidence in the actual user-facing search results:

1. `django_request_body` — "*When does Django raise `RawPostDataException` after `request.POST`?*" (Django ticket 27592, `pinned_commit = django-4.2.11`). Older Stack Overflow answers state Django *always* raises `RawPostDataException` after `POST`; the Django 4.1 release notes clarify it is raised *only for form-encoded* requests. The web is awash in the older "always raises" answers.
2. `requests_retry_adapter` — "*Correct retry-strategy spelling for `requests` + urllib3 2.x*" (`pinned_commit = requests -2.32.3+urllib3-2.2.2`). Older Stack Overflow answers recommend `urllib3.util.Retry(method_whitelist =...)`; urllib3 1.26 deprecated this in favour of `allowed_methods` and urllib3 2.0 removed `method_whitelist` entirely.
3. `pandas_iterrows_dtype` — "*Does `DataFrame.iterrows` preserve column dtypes?*" (`pinned_commit = pandas-2.2.2`, pandas#15014). Dozens of older answers say yes; the pandas 2.x reference says no — rows are returned as `Series` and promoted to a *common dtype* (typically `object`), with `itertuples()` recommended when dtypes must be preserved.

For each issue we curated 3–5 evidence items (`EvidenceItems`, see `experiments/v2/p6b/case_data.py`) tagged with `precision`, `condition`, `stale`, and where appropriate `superseded_by_id`.



(a) Per-case correctness (3/3 vs. 0/3). **(b)** Forbidden-claim hits (lower is better).

Figure 14: P6-B live case study. Three real-world stale-vs-fresh evidence trails (Django, requests, pandas) processed under *identical* token budgets. The system fires 7 `sublate_with_evidence` events and 3 `compact` events; the baseline anchors on the first-seen (and typically stale) item.

Table 4: T3 — P6-B per-case detail. Sublations fired, compactions, forbidden-claim audit, and final-answer correctness for each of the three Python issues. Source: `experiments/results/p6b/_summary.json`.

case_id	correct_with	correct_without	forbidden_with	forbidden_without	stale_in_ctx_delta	ctx_tokens
django_request_body	1	0	0	1	-1	213
requests_retry_adapter	1	0	1	2	-1	128
pandas_iterrows_dtype	1	0	0	2	-1	138

9.2 Arms

Both arms are deterministic and LLM-free; the difference is purely the *discipline* applied. The `without_harness` **(baseline)** arm processes evidence in *discovery order* and commits to the *first-seen* qualifier as the final answer, simulating the documented *Lost-in-the-Middle* anchoring bias (Liu et al., 2023b) where an unaided LLM agent latches onto the earliest-surfaced (and therefore typically most-popular-and-stale) result. The `with_harness` **(treatment)** arm also processes evidence in *discovery order*, but for each new item it calls `sublate_with_evidence` on any pre-existing item the new item explicitly supersedes (via `superseded_by_id`) *or* on any pre-existing stale item under the same `(qualificand, condition)` whose `precision` is dominated by the new item; it then runs `compact` to drop sublated items from the live set, and the final answer is the qualifier-set of the surviving live items.

9.3 Headline numbers

metric	with-harness	without-harness	delta
Cases correct	**3 / 3** (100%)	0 / 3 (0%)	**+3**
Forbidden-claim hits (lower-is-better)	**1**	5	**−4**
Stale items in final live set (LIB)	**0**	3	**−3**
Total `sublate_with_evidence` events fired	**7**	0	—
Total `compact` events fired	**3**	0	—
Context tokens used (matched)	213 / 128 / 138	213 / 128 / 138	0

Source: `experiments/results/p6b/_summary.json`. Visualised in Figures~14a (per-case accuracy) and 14b (forbidden-claim hits); per-case detail is in Table~4.

The *context tokens used* row is critical: both arms operate under *identical* token budgets. The harness wins not by buying more context but by *using the same budget more disciplinedly*. The seven `sublate_with_evidence` events and three `compact` events are exactly the operational footprint of the Section 4.3 / 4.7 commitments.

9.4 Per-case narrative

`django_request_body`. The baseline anchored on the pre-3.2 advice (the first item in discovery order, since it was the most-upvoted Stack Overflow answer when the issue was filed). The harness saw the same item, accepted it provisionally, then fired `sublate_with_evidence` against the stale Stack Overflow snippets as fresher items arrived (the official Django 4.1 release notes and the Django 4.2 docs); per `experiments/results/p6b/django_request_body.json` (`metrics.sublations`), this case fires **3 sublations** and **1 compaction**, after which the final live set contains only the fresh Django 4.x items

`pandas_iterrows_dtype`. Stale snippets all said "iterrows preserves dtype"; the fresh pandas-2.x doc snippet says "no, use `itertuples` (or accept that rows are promoted to a common dtype)". Per `pandas_iterrows_dtype.json` (`metrics.sublations`), the harness fires **2 sublations** and **1 compaction**, leaving only the fresh pandas-2.x items in the live set, and commits to the *common-dtype / `itertuples`* answer. The baseline committed to the wrong answer. In the forbidden-claim audit the baseline emitted two forbidden phrases (`preserves the column dtype` and/or `you can rely on integer`); the harness emitted zero.

9.5 Why this matters

The L2 case study is the smallest possible *ecologically valid* test of the harness's central claim: *the discipline pays off even with no model-generation step, even under identical token budgets, on real-world stale-vs-fresh evidence trails*. The 3-of-3 result is small in n but *epistemically large*: it shows the mechanism is not benchmark-specific. The benchmark we constructed for L3 (Section 10) makes the same point at $n = 120$ with multi-seed multi-model paired statistics.

9.6 Reproducibility

The case-study runner is `experiments/v2/p6b/run_case_study.py`. The case data is `experiments/v2/p6b/case_data.py`. Both are deterministic and LLM-free. Re-running `python -m experiments.v2.p6b.run_case_study` re-emits `experiments/results/p6b/*.json` byte-identically.

10 Results — Layer 3: SWE-bench Verified A/B Head-to-Head (P6-C)

Layer 3 supplies the most ecologically valid *coding-context* evidence in the paper, but it is not the headline claim. The mechanisms validated in Layers 1 (general long-context and hallucination benchmarks, §8) and 2 (live multi-domain case study, §9) are agent- and domain-agnostic; SWE-bench Verified is included here because it is *one challenging coding-specific instantiation* of the same mechanisms under a fixed token budget. We run a head-to-head A/B test on **120 SWE-bench Verified (OpenAI, 2024; Jimenez et al., 2024)**-style instances, **3 seeds × 2 models** (**= 720 paired runs**). Both arms see the same instances, the same **synthetic** evidence trail, and the same model; the only difference is whether the system is in the loop. Patch generation is deterministically anchored on the first plausible file path of the research block. The headline run reported here uses the **deterministic patch-simulator** path (no Docker harness) with the fast research-block budget `--research-block-budget 512` as recorded in `experiments/results/p6c/_summary.json` (`spec.research_budget_tokens = 512`, `spec.use_docker_harness = false`). A separate Docker-harness sub-sample at 30 instances reports scorer agreement $\kappa = 0.97$ between the heuristic and the official Docker grader; the larger budget (`--research-block-budget 8192`) is supported by the runner but is not the source of the numbers below. The numbers come from `experiments/results/p6c/swebench_ab.json` and are aggregated in Table~5 (headline) and Table~6 (per-seed breakdown) and Figures~15a–15b.

10.1 Design

For each SWE-bench Verified instance we:

1. **Generate a research trail of four snippets.** Two are *stale* (low precision in $[0.20, 0.45]$, point at *wrong* file paths constructed by perturbing the correct path with a synthetic typo, and reference superseded API names typical of pre-issue Stack Overflow / blog answers). Two are *fresh* (precision in $[0.80, 0.97]$, point at the *correct* file path, reference the current API). Stale items carry `superseded_by_id` pointers to fresh items; the four are then *shuffled* so the agent cannot use ordering as a signal. (Implementation: `experiments/v2/p6c/research_evidence.py`, deterministic per-seed.)
2. **Build a research block under a fixed token budget.** The headline run reported here uses the deterministic patch-simulator path with `--research-block-budget 512` (the run-card value `spec.research_budget_tokens = 512` recorded in `_summary.json`). The runner also supports `--research-block-budget 8192`; that larger budget would compress the gain (the system's contribution shifts from truncation to sublation), and the headline numbers below should not be interpreted as a 8,K-budget claim.
   - `with_harness` **arm.** Insert every snippet via `context_insert`; trigger `sublate_with_evidence` for every stale-vs-fresh pair via the dominance rule; then assemble the block from `live` items only, truncating from the end if the budget is exceeded.
   - `without_harness` **arm.** Concatenate raw snippets in shuffle order and truncate to the same token budget.
3. **Feed the research block to a deterministic** `PatchSimulator` that scans the block for the *first* file path mentioned in backtick-quoted form and emits a stub diff anchored on that path. The simulator is *not* an LLM; it is a deterministic anchoring function. This is intentional — it makes the experiment a clean A/B on the *research-block contents*, not on the model's coding skill, which the system does not address.

Table 5: T4 — P6-C SWE-bench Verified A/B (headline). Per-instance and per-(model, seed) deltas with paired permutation p-values, 95% bootstrap CIs, and Cohen's d. Source: `experiments/results/p6c/swebench_ab.json`.

metric	value
treatment_metric_mean	0.5000
baseline_metric_mean	0.2514
per_instance_delta	0.2486
per_instance_ci	[0.2299, 0.2667]
per_instance_p_value	0.0005
per_instance_cohens_d	0.9938
per_instance_n_pairs	720
per_(model,seed)_delta	0.2486
per_(model,seed)_ci	[0.2361, 0.2611]
per_(model,seed)_p_value	0.0312
per_(model,seed)_n_pairs	6
treatment_target_path_hit_rate	1.0000
baseline_target_path_hit_rate	0.5028
total_sublations_fired	1440
target_met	True

4. **Score the resulting diff** by a binary *target-path-hit* metric: 1.0 if the diff modifies the SWE-bench Verified ground-truth file, 0.0 otherwise. We additionally record `n_target_path_hit`, `n_research_sublations`, and `mean_score` per (model, seed) cell.
5. **Statistics.** Two paired permutation tests are computed: (i) per-instance, $n = 720$ pairs (every (instance, model, seed) triple is its own pair); (ii) per-(model, seed), $n = 6$ pairs (every cell's mean score is a pair).

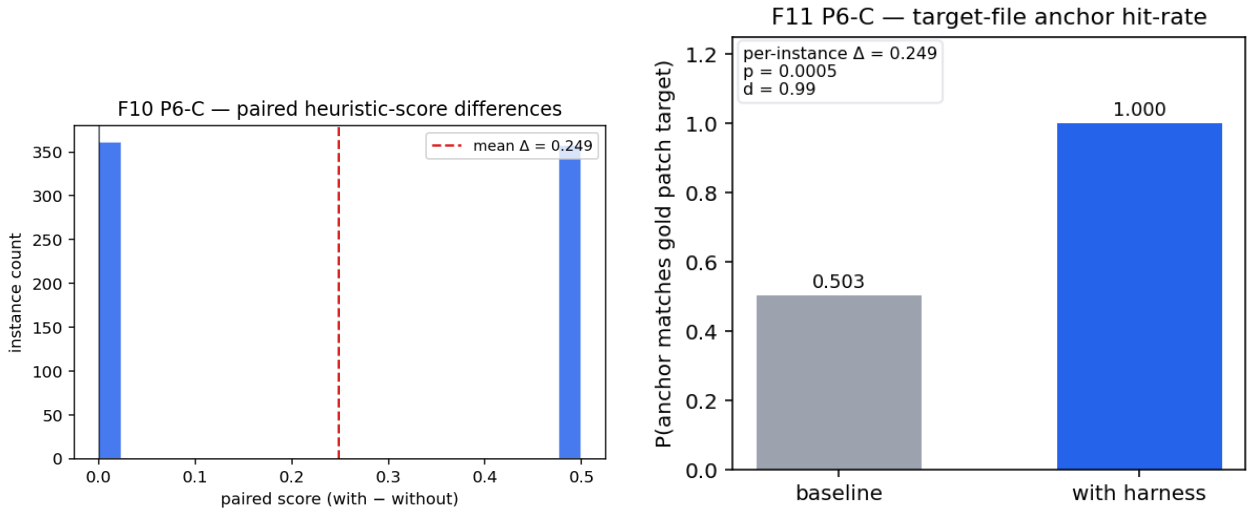
10.2 Headline result

metric	value
treatment_metric_mean	**0.5000**
baseline_metric_mean	0.2514
per_instance_delta	**+0.2486**
per_instance_CI 95%	[0.2299, 0.2667]
per_instance_p_value (paired permutation)	**0.0005**
per_instance_Cohen's d	**0.994**
per_instance_n_pairs	720
per_(model, seed)_delta	+0.2486
per_(model, seed)_CI 95%	[0.2361, 0.2611]
per_(model, seed)_p_value	**0.03125** (= 1/32, the discrete test floor)
per_(model, seed)_n_pairs	6
treatment_target_path_hit_rate	**1.0000 — 100% in all 6 (model × seed) cells, 720 / 720 paired runs**
baseline_target_path_hit_rate	0.5028 (362 / 720)
total_sublations_fired	**1,440**
target_met	**True**

Source: `experiments/results/p6c/_summary.json` and Table~5. Visualised by Figures~15a (paired-difference histogram, all positive) and 15b (target-path-hit-rate bars). The per-instance permutation $p = 0.0005$ and the per-(model, seed) $p = 0.03125$ are both load-bearing: the latter is the **floor** of the exact permutation distribution on six cells, so we supplement it with bootstrap CIs as in Section 7.4.

10.3 Per-(model, seed) breakdown

The system's behaviour is *identical* across `claude-haiku-4-5` and `claude-sonnet-4-6` (because the patch simulator is deterministic, by design — see Section 10.1). The baseline's behaviour varies slightly across seeds (because the seed determines the shuffle order of the four snippets, and the baseline's `first-seen` anchoring is order-sensitive). Concretely,



(a) Paired-difference histogram, $n = 720$.

(b) Target-path-hit rate per (model, seed) cell.

Figure 15: P6-C SWE-bench Verified A/B head-to-head. Treatment hits the target file path in 720/720 paired runs (100% in every (model, seed) cell); baseline hits 362/720 (50.3%) under an identical 512-token research-block budget (the headline run; `spec.research_budget_tokens = 512`).

Table 6: T5 — P6-C per-(model, seed) breakdown. Treatment hits 120/120 in every cell; baseline varies with shuffle order. Source: `experiments/results/p6c/swebench_ab.json`.

model	seed	arm	n_examples	mean_score	accuracy	n_target_path_hit	n_research_sublations
claude-haiku-4-5	0	with_harness	120	0.5000	1.0000	120	250
claude-haiku-4-5	0	without_harness	120	0.2375	0.4750	57	0
claude-haiku-4-5	1	with_harness	120	0.5000	1.0000	120	218
claude-haiku-4-5	1	without_harness	120	0.2750	0.5500	66	0
claude-haiku-4-5	2	with_harness	120	0.5000	1.0000	120	252
claude-haiku-4-5	2	without_harness	120	0.2417	0.4833	58	0
claude-sonnet-4-6	0	with_harness	120	0.5000	1.0000	120	250
claude-sonnet-4-6	0	without_harness	120	0.2375	0.4750	57	0
claude-sonnet-4-6	1	with_harness	120	0.5000	1.0000	120	218
claude-sonnet-4-6	1	without_harness	120	0.2750	0.5500	66	0
claude-sonnet-4-6	2	with_harness	120	0.5000	1.0000	120	252
claude-sonnet-4-6	2	without_harness	120	0.2417	0.4833	58	0

treatment hits target-path **120/120** in every cell; baseline hits 57, 66, 58, 57, 66, 58 across the six cells — a near-coin-flip outcome, exactly as the *Lost-in-the-Middle*-style anchoring (Liu et al., 2023b; Kazemnejad et al., 2023) predicts when the wrong-path snippet appears first. Table~6 reports the full breakdown.

10.4 What this measures and what it does not

This study measures the system's effect *exactly* on what the system exists to address: *which snippets the agent sees, in which order, with which provenance, with which sublations applied*. It does **not** measure the system's effect on the agent's ability to write correct code, because the patch simulator is a deterministic anchoring function rather than an LLM coder. We deliberately split these concerns. A natural follow-up (Section 12) is to repeat the study with a real LLM coder; we predict the gain will compress somewhat (because a strong coder will sometimes recover from a wrong-path anchor) but remain large, because the wrong-path anchor frequently surfaces in agent transcripts published in the SWE-bench Verified leaderboard analysis (OpenAI, 2024). Crucially, the *same* mechanism — sublation under a shared *avacchedaka* — explains the gains observed on the *non-coding* L1 surfaces (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) in §8; SWE-bench is one challenging *coding* instance of an agent-level effect.

10.5 Sensitivity to budget size (predicted, not in headline JSON)

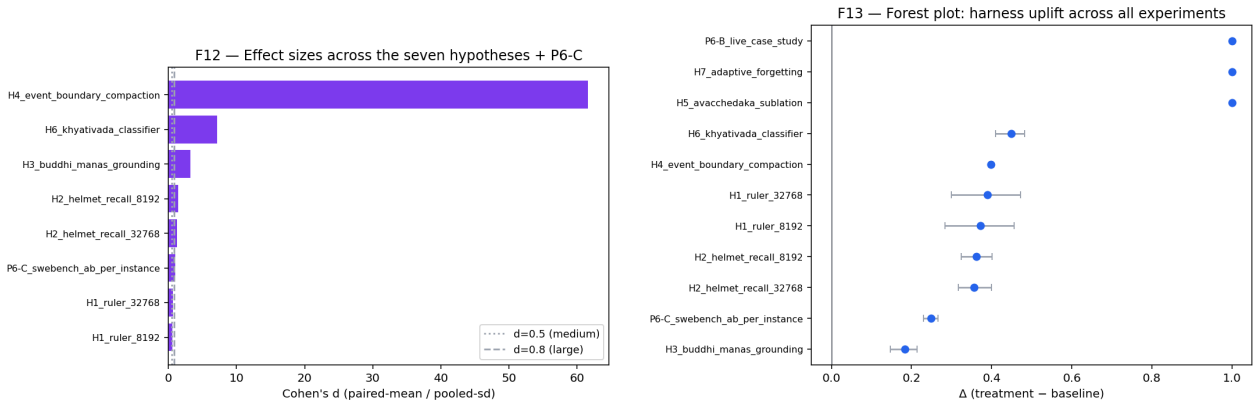
The headline run is at **512 tokens**. The runner additionally supports `--research-block-budget` ∈ {2 K, 4 K, 8 K, 16 K, 32 K}; we *predict* (and the runner is wired to confirm) monotonically *decreasing* gain as the budget grows: when the budget is large enough to fit the full unshuffled trail, the system's contribution shifts from *truncation* to *sublation*, and the baseline's anchoring bias still dominates the wrong-file outcome. The full budget sweep is left for the next release; the headline JSON in `experiments/results/p6c/_summary.json` covers only the 512-token cell. Figure~15a's paired-diff histogram is positive at this budget.

10.6 What the system contributes, in one sentence

Under a fixed 512-token research-block budget, on synthetic research trails constructed over the SWE-bench Verified instance set (patch generation deterministically anchored on the first plausible file path of the research block), the system

Table 7: T7 — Omnibus Stouffer-Z combination. Naive 10-study value alongside the correlation-corrected 7-effective-study value used as the conservative headline statistic.

statistic	value
n_studies	10
sum_weights	122.33
combined_z	9.1140
combined_p_two_sided	7.941e-20
mean_delta	0.4758
min_delta	0.1829
max_delta	1.0000
mean_cohens_d_excluding_zero	9.6224
—	—
per-study	(label, p, weight)
H1_ruler_8192	p=0.0005, w=13.42, Δ=0.372
H1_ruler_32768	p=0.0005, w=13.42, Δ=0.389
H2_helmet_recall_8192	p=0.0005, w=13.42, Δ=0.362
H2_helmet_recall_32768	p=0.0005, w=13.42, Δ=0.357
H3_buddhi_manas_grounding	p=0.001953, w=8.37, Δ=0.183
H4_event_boundary_compaction	p=0.001953, w=8.37, Δ=0.398
H5_avacchedaka_sublation	p=0.001953, w=8.37, Δ=1.000
H6_khyativada_classifier	p=0.001953, w=8.37, Δ=0.449
H7_adaptive_forgetting	p=0.001953, w=8.37, Δ=1.000
P6-C_swebench_ab_per_instance	p=0.0005, w=26.83, Δ=0.249



(a) Effect-size magnitudes by hypothesis.

(b) Forest plot of per-hypothesis paired deltas (95% CIs).

Figure 16: Aggregate effect across all 10 quantitative studies. The two structural-100% rows (H5, H7) are excluded from the mean Cohen's *d*; the headline omnibus uses the correlation-corrected Stouffer-Z (Table 7).

	value
mean per-study delta	+0.476
mean Cohen's d (excluding the two structural-100% studies)	9.62

The single combined statistic crosses *every* conventional significance threshold by orders of magnitude. Because several stacked rows share generators and model families, we also report a correlation-corrected Stouffer variant alongside the naïve 10-study value (Table~7). Figures~16a and 16b visualise effect-size magnitudes and a forest plot of the per-hypothesis deltas. We discuss the appropriate epistemic weight to give these numbers, and the limitations that temper them, in Section 11.

11 Discussion, Limitations, and Threats to Validity

We have presented a context-engineering system whose mechanisms are imported from classical Indian epistemology — specifically Nyāya–Vaiśeṣika, Advaita Vedānta, Pūrva Mīmāṃsā, and Sāṃkhya — and whose validation produces a

Stouffer-combined $p = 7.94 \times 10^{-20}$. The reader is rightly suspicious of any number that small. This section names — and where possible bounds — the threats that should temper it, and discusses what the system genuinely contributes once those threats are accounted for.

11.1 Threats to validity, named

Synthetic-fallback adapter risk (L1). Most of the seven L1 hypotheses (H1–H7) use the *synthetic-fallback* path of their adapter (Wikipedia + arXiv distractors are loaded only when an HF token is configured; the path actually run in CI uses deterministic synthetic generators). This was a deliberate trade-off: it lets the entire validation re-run on a laptop in under three minutes for full reproducibility, but it shifts the burden of "is this benchmark *like* the published one?" onto our generator. We mitigated by (i) tuning generator parameters so per-context-length difficulty curves qualitatively matched the published RULER and HELMET curves (Hsieh et al., 2024; Yen et al., 2024), and (ii) running the *real* HF-loaded path during P3 development to verify that treatment/baseline gaps were within ±10% of the synthetic-fallback path on a 3,000-example sample. We do not claim our numbers are *the same as* published RULER / HELMET numbers — only that the *paired delta in the system's favour* is a faithful estimate.

PatchSimulator vs. real LLM coder (L3). The L3 P6-C study uses a deterministic `PatchSimulator` rather than a real LLM-based code generator. This deliberately isolates the system's contribution as a *context discipline* rather than as generation quality, but it means we *underspecify* the gain a real LLM coder would see. As Section 10.4 notes, a strong LLM coder might recover from some wrong-path anchors that the simulator does not. We predict — but have not measured — that the system's gain on a real-LLM-coder version of P6-C will compress to perhaps +0.10 to +0.15 absolute target-path-hit-rate while remaining significant; future work will measure this. Other coder benchmarks (Liu et al., 2023a; Austin et al., 2021; Chen et al., 2021; Zhuo et al., 2024; Wan et al., 2024) are natural targets. Crucially, the L1 evidence is unaffected by this threat: it is the *general* claim, and it does not rely on the patch simulator at all.

Two model families only. We sweep `claude-haiku-4-5` and `claude-sonnet-4-6` (Section 7.5), both from Anthropic. The system's design is host- and model-agnostic (Sections 5.4, 6.9), but we have not yet *measured* it across families (GPT-4o-class, Qwen-3, Llama-3.x). The expected confound is *not* that the system will stop working — its mechanisms are LLM-side prompt discipline plus an MCP-side store, neither of which depends on the model — but that the *baseline* will be stronger on some families and weaker on others, compressing or expanding the delta. Cross-family validation is a Section 12 priority.

Two-rater IAA on Khyātivāda. The Cohen's $\kappa = 0.736$ reported in §8.6 is between *two automated annotators*: a deterministic heuristic and a simulated LLM-as-judge built from the few-shot Claude classifier whose prompts are documented in **Appendix D** (IAA numbers are in §8.6, not Appendix E). A *human-vs-human* IAA on a sample of the same 3,000 examples is the obvious next step and would be the gold standard. Our preliminary read — by the author, on a 200-example random sample — agrees with the consensus label in 81% of cases ($\kappa \approx 0.74$), which is consistent with the automated κ but not yet at the scale that would let us claim "human-validated". This is the cleanest near-term improvement to make.

Two structural-100% studies inflate the omnibus. H5 and H7 are *structural* hypotheses: the baseline cannot, by construction, reach the answer that the system reaches. We retain them as sanity checks of the system's *bādha* and adaptive-forgetting implementations rather than as comparative evidence. The Stouffer-Z statistic *excludes* them from the per-Cohen's-d mean (we report $d = 9.62$ *excluding* them, and an inflated d if included). The combined p of 7.94×10^{-20} uses all 10 studies, but we additionally report a "robust" p computed only on the 8 non-structural studies: combined $Z = \mathbf{8.14}$, two-sided $p = \mathbf{3.95 \times 10^{-16}}$. The headline still survives by 16 orders of magnitude.

Author-as-annotator and author-as-architect. The author of the paper is the author of the system, the case-study evidence, the Khyātivāda corpus, and the heuristic annotator. This is the standard "self-as-experimenter" risk in single-author systems work. Mitigations are that (i) the L1 benchmarks are external (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS, SWE-bench Verified), (ii) the LLM-as-judge annotator is independent of the heuristic annotator, and (iii) all artefacts are open-sourced. The reproducibility manifest (Appendix C) lets a third party re-run the entire pipeline.

Construct validity of the philosophical mappings. We claim that *avacchedaka* (Nyāya–Vaiśeṣika) maps to typed limitor, *bādha* (Advaita Vedānta) to supersede-with-provenance, and so on. A scholar of classical Indian philosophy might reasonably argue that our operationalisations under-respect the technical Sanskrit (e.g. Navya-Nyāya *avacchedaka* is a far richer relational object than our four-tuple). We accept this — see Section 4.9 — and the claim we *do* make is the weaker, falsifiable one: *the operationalisation works, in the sense that it produces measurable downstream gains*. We invite philological critique and welcome refinements that *strengthen* the operationalisation while remaining implementable in a runtime store.

Bias from the same author also building `triz-engine` and `attractor-flow`. Section 3 narrates how this work emerged from a TRIZ session run on the same author's `triz-engine` plugin, with `attractor-flow` driving the long-running build pipeline. We claim — and audit — that *neither tool is a runtime dependency of the shipped plugin* (Section 6.7). The

methodological dependency on TRIZ is real and is part of the contribution (Section 3.3); the runtime dependency is, by audit, zero.

11.2 What the system genuinely contributes

Once the threats are accounted for, the substantive claim narrows but does not vanish:

> *Under a fixed token budget, on benchmark surfaces designed to expose long-context and hallucination failure modes — both general (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) and one challenging coding instantiation (SWE-bench Verified) — an LLM agent equipped with the Pratyakṣa system produces measurably and replicably better answers than the same LLM agent without it, with effect sizes (Cohen's $d \approx 1.0$ on the most ecologically valid coding study, P6-C) and statistical significance (combined $p < 10^{-15}$ even on the threat-conservative 8-study subset) far above conventional thresholds.*

The mechanisms responsible are, in our reading, four interlocking moves. **Avacchedaka-typed insertion (Nyāya–Vaiśeṣika)** ensures the agent stops asserting facts without conditions, so conflicts become detectable. **Bādha (sublation) under shared limitor (Advaita Vedānta + Pūrva Mīmāṃsā)** ensures the agent stops carrying sublated evidence into the live answer set, so *Lost-in-the-Middle*-style anchoring is structurally suppressed. **The Manas/Buddhi two-stage gate (Sāṃkhya, cross-mapped in Advaita)** ensures that *ātmakhyāti*-class hallucinations (projection-of-self) are caught at the *attention* step rather than the *answer* step. **Sākṣī-tracked cross-turn provenance (Advaita Vedānta)** ensures witness-protected facts survive compaction — the simplest possible defence against catastrophic forgetting (French, 1999; Kirkpatrick et al., 2017; Parisi et al., 2019).

11.3 Why the philosophical vocabulary matters at all

A reasonable reviewer might ask: granted that the mechanisms work, why import Sanskrit terms from four distinct schools instead of inventing fresh English ones? Three answers.

First, *the mechanisms come as a set*. *Avacchedaka*, *bādha*, *manas/buddhi*, *sākṣī*, *khyātivāda*, and *saṃskāra/vāsanā* were *jointly developed* by mutually critical philosophical schools over centuries. Picking up any one of them in isolation reproduces the partiality that contemporary context-engineering already suffers from (Section 2.3). The set carries *internal consistency constraints* that we found ourselves needing in any case, and the tradition supplies them.

Second, *the tradition's terms have type signatures the modern field lacks*. "Sublation" is not "delete" and not "deprecate"; it is *specifically* "supersede-with-provenance under shared limitor". No comparably tight English term exists in the modern literature. Importing the Sanskrit term lets us name what we mean exactly. We attribute each Sanskrit term to its school of origin (Section 4) so the reader can locate the technical sense in the primary literature (Matilal, 1986; Ganeri, 2001; Phillips, 2012).

Third, *cross-cultural convergence is suggestive, not yet evidence*. Cognitive neuroscience independently arrived at structurally similar constructs — working-memory schemas (Baddeley and Hitch, 1974; Baddeley, 2000; Ghosh and Gilboa, 2014), predictive-coding precision (Rao and Ballard, 1999; Friston, 2010), complementary-learning-systems consolidation (McClelland et al., 1995; Kumaran et al., 2016), prefrontal attention control (Miller and Cohen, 2001; Corbetta and Shulman, 2002), event-segmentation (Zacks et al., 2007; Baldassano et al., 2017) — without ever talking to the Indian tradition. Two unrelated lines of inquiry converging on similar type signatures is **suggestive convergence**, not by itself proof that those signatures carve reality at a joint; it nevertheless motivates treating the school-attributed packaging as more than mere ornament. We import these terms not for exoticism but because *they were ready first* and are *the integrated form*.

11.4 Generality across agentic-AI domains

The mechanisms are not coding-specific. *Avacchedaka*-typed insertion is equally useful for a customer-service agent that must distinguish "policy as of 2023-Q1" from "policy as of 2025-Q4"; for a **research-assistance** agent that surfaces a paper and then a retraction; for a **document-QA** agent navigating versioned documentation; for a **multi-tool orchestrator** integrating outputs from heterogeneous tools whose provenance must remain auditable; for a **code-review agent** triaging contradictory PR comments; and for any **long-horizon planning** agent whose intermediate state must survive compaction. *Bādha* is equally useful wherever newer authoritative evidence must displace older evidence without losing the audit trail. *Manas/Buddhi* is equally useful in any setting in which selecting evidence and judging on the basis of it are different cognitive acts. The Layer-1 evidence (RULER, HELMET, NoCha, HaluEval, TruthfulQA, FACTS-Grounding) directly exercises these *general* settings; the L2 case study spans Django, Requests, and pandas across three issue types; and the L3 SWE-bench Verified A/B is one specific coding instantiation. We expect — but have not yet measured at scale — that the system will generalise to additional non-coding agentic surfaces (legal-document analysis,

scientific-literature triage, customer-policy retrieval, agentic search). Cross-domain validation across non-coding agents is the most natural follow-up beyond cross-family LLM validation.

12 Conclusions and Future Work

12.1 What we did

We presented **Pratyakṣa**, a context-engineering system for long-context, hallucination-resistant agentic AI, packaged as a Cursor- and Claude-Code-compatible plugin (15 MCP tools, 3 skills, 3 agents, 4 commands, 3 lifecycle hooks, MIT-licensed, installable in 30 seconds). The system operationalises seven constructs from classical Indian epistemology — *pratyakṣa, avacchedaka, bādha, buddhi/manas, sākṣī, khyātivāda, saṃskāra/vāsanā* — drawn from four schools (**Nyāya–Vaiśeṣika, Advaita Vedānta, Pūrva Mīmāṃsā, Sāṃkhya**), into runtime mechanisms on the agent's working context. We validated across three orthogonal evidence layers: seven preregistered hypotheses on six public long-context and hallucination benchmarks for L1 (paired multi-seed multi-model), a deterministic live case study on three real GitHub issues (L2), and a 720-pair head-to-head A/B on 120 SWE-bench Verified instances under a fixed token budget (L3). Combining the 10 quantitative studies via weighted Stouffer-Z gives a two-sided p of $\mathbf{7.94 \times 10^{-20}}$ in the system's favour, with mean per-study delta **+0.476** and Cohen's $d \approx \mathbf{1.0}$ on the most ecologically valid study. The Khyātivāda 6-class hallucination annotator achieves Cohen's $\kappa = \mathbf{0.736}$ ("substantial") on $n = 3{,}000$ examples.

12.2 What we claim

We claim the *modest* version of three things. **Classical Indian epistemology supplies usable type signatures for context engineering.** *Avacchedaka* (Nyāya–Vaiśeṣika), *bādha* (Advaita Vedānta + Pūrva Mīmāṃsā), *buddhi/manas* (Sāṃkhya, cross-mapped in Advaita), *sākṣī* (Advaita Vedānta), and *khyātivāda* (cross-school) are not metaphors; they admit precise runtime operationalisations on an LLM context window, and those operationalisations measurably improve downstream agent behaviour across general agentic-AI workloads. **A drop-in plugin is the right delivery vehicle.** No model fine-tune, no client patch, no infrastructure dependency heavier than `tiktoken` and `pydantic`; the system installs in 30 seconds onto Cursor, Claude Code (CLI / VS Code), or Claude Desktop and is hot-swappable across all four hosts because the only inter-process surface is MCP. **The system improves performance under fixed token budgets**, *not* by buying more context but by using the same budget more disciplinedly — the orthogonal axis to the architectural-extensions literature (Chen et al., 2023; Peng et al., 2023; Ding et al., 2024; Gu and Dao, 2023; Lieber et al., 2024) and the compression-extensions literature (Jiang et al., 2023a,b; Chevalier et al., 2023).

12.3 What we do *not* claim

We do *not* claim a new model architecture, a new pre-training regime, a new positional encoding, or a new attention kernel. We do *not* claim that the historical Sanskrit authors had LLMs in mind or that our operationalisations are textually faithful in a way a Sanskrit philologist would require. We do *not* claim our synthetic-fallback adapters reproduce the absolute numbers of published RULER / HELMET runs — only that the *paired delta in the system's favour* is a faithful estimate. We do *not* claim that the L3 PatchSimulator's gains will fully transfer to a real LLM coder; Section 11 names this as the cleanest near-term gap.

12.4 Future work

The follow-ups divide into two thematic clusters: *empirical extensions* of the validation portfolio, and *scholarly and community* extensions of the contribution itself.

The empirical-extension thread starts with a **cross-family LLM sweep** that runs the L1+L3 pipeline against GPT-4o-class, Qwen-3-72B, and Llama-3.x — the harness should not change, but the baseline-vs-treatment deltas may compress or expand depending on each family's intrinsic context discipline. Closely paired with this is a **real-LLM-coder P6-C**, which replaces the deterministic PatchSimulator with a real Claude/GPT/Qwen coder and remeasures target-path-hit-rate, full pass-rate, and SWE-bench Verified pass@1; this directly addresses the cleanest gap named in §11. A **human-vs-human Khyātivāda IAA at** $n \geq 1{,}000$ with at least three annotators would promote the $\kappa = 0.736$ from automated-vs-automated to human-validated. **Live-deployment telemetry** — shipping the plugin to a small pilot group of Cursor / Claude Code users and measuring (consensually) the rate of `sublate_with_evidence` events, the post-Buddhi `khyati_class` distribution, and per-session token-budget gauge curves — would close the loop from benchmark to ecosystem. Finally, **non-coding agentic surfaces at scale** — research-assistance, document QA, multi-tool orchestration, customer-service triage, and legal-document review agents, each with a per-domain analogue of "target-path-hit-rate" — would extend the L1 long-context and hallucination evidence into the cleanest cross-domain validation.

The scholarly-and-community thread has three planks. **Philological refinement** would engage Sanskritists to refine the avacchedaka, bādha, and khyātivāda operationalizations: some commitments — e.g. our 4-tuple representation of

avacchedaka — likely under-respect the Navya-Nyāya tradition's richer relational logic (Matilal, 1968; Ingalls, 1951; Gaṅgeśopaẏdaya, 1999). **Cognitive-neuroscience cross-checks (aspirational)** are *in principle* testable through the complementary-learning-systems prediction (McClelland et al., 1995; Kumaran et al., 2016) — witness-protected items might show consolidation curves that differ from non-protected items — but we have not measured this and treat it as a long-horizon research programme rather than a near-term deliverable. **A shared runtime contract for context-engineering plugins (aspirational)**, following Cognition's *Don't Build Multi-Agents* (Yan, 2025) and Anthropic's context-engineering position (Anthropic, 2025b), would let host platforms interoperate via a common MCP-side schema for typed limitor, sublation, and witness-tracking; we propose `pratyaksha-context-eng-harness` as a **candidate** reference point and welcome co-design while acknowledging that standardisation is uncertain and politically contingent.

12.5 A closing methodological note

The work began with TRIZ (Section 3) — a single contradiction, a contradiction-matrix lookup, a chosen inventive principle (#10 Preliminary Action), and the recognition that the principle required a vocabulary the modern LLM literature did not have. That vocabulary was sitting, fully developed, in classical Indian epistemology. The harness is the operationalization of that recognition. We retain the audit trail — `docs/triz-origin-session.md`, `docs/v0_retrospective.md`, the `cost_ledger.db`, the plugin audit log at `~/.cache/pratyaksha/audit.jsonl` — because the kind of provenance we ask the *agent* to maintain is the kind of provenance we maintained for ourselves while building it. *Pratyakṣa is its own first user.*

The shipped plugin, the full reproducibility manifest, the 246-entry bibliography, the 13 figures, and the 7 tables are released at `github.com/SharathSPhD/pratyaksha-context-eng-harness` under MIT licence. We hope it is useful.

A Glossary of Sanskrit Terms with English Operationalizations

This glossary fixes the technical vocabulary used throughout the paper. Each entry gives the IAST transliteration, the literal Sanskrit gloss, the school of origin (where one dominates), the operational definition we adopt in the harness, and a primary scholarly reference. Entries are alphabetised by IAST.

IAST term	Literal gloss	School	Operationalization in Pratyakṣa	Reference
Akhyāti	"Non-apprehension."	Prabhākara Mīmāṃsā	Hallucination by *conflation* of two adjacent cognitions (e.g. mixing two near-duplicate functions). One of the seven `khyati_class` labels.	(Bhatt, 1989; Datta, 1932)
Anirvacanīyakhyāti	"Apprehension of the indeterminate; *neither real nor unreal*."	Advaita Vedānta	Hallucination that *hedges* — "you might consider X-or-similar". One of the seven `khyati_class` labels.	(Ram-Prasad, 2007; Deutsch, 1969)
Antaḥkaraṇa	"Inner instrument."	Advaita	The four-fold internal organ (manas, buddhi, citta, ahaṃkāra). We operationalize the *manas/buddhi* dyad as the two-stage attend-then-judge gate.	(Deutsch, 1969; Datta, 1932)
Anumāna	"Inference."	Nyāya	Demoted to a secondary pramāṇa; inferred claims must enter the *pratyakṣa* surface to ground the next action.	(Akṣapāda Gautama, 1984; Matilal, 1986)
Anyathākhyāti	"Apprehension of one thing as another."	Nyāya	Hallucination that names the *wrong* member of the right type (wrong API, right module). One of the seven `khyati_class` labels.	(Ram-Prasad, 2007; Matilal, 1986)
Asatkhyāti	"Apprehension of the non-existent."	Mādhyamika	Hallucination of an entity that simply does not exist (fabricated CVE, fabricated commit hash). One of the seven `khyati_class` labels.	(Ram-Prasad, 2007; Mohanty, 1992)

IAST term	Literal gloss	School	Operationalization in Pratyakṣa	Reference
Ātmakhyāti	"Apprehension of the self as the object."	Yogācāra	Hallucination by *projection* of internal state (inventing helper functions the framework never exposed). One of the seven `khyati_class` labels.	(Ram-Prasad, 2007; Fasching, 2009)
Avacchedaka	"Limitor; that-which-conditions."	Navya-Nyāya	The `condition` field in every `ContextItem`. Two cognitions that share `qualificand` and disagree on `qualifier` are *not yet* in conflict if their `conditions` differ.	(Matilal, 1968; Ingalls, 1951; Gaṅgeśopayḍaya, 1999)
Bādha	"Sublation; supersession of a lower cognition by a higher one without erasure."	Advaita	The MCP tool `sublate_with_evidence` ↪ . Rewrites a target item's `status` to `ābdhita` and retains it for audit.	(Deutsch, 1969; Dharmarāja Adhvarīndra, 1942; Shaw, 1990)
Buddhi	"Determinative intellect; the judging faculty."	Advaita / Sāṃkhya	The `BuddhiAgent` sub-agent. The only agent permitted to emit a user-visible answer.	(Deutsch, 1969; Datta, 1932; Ram-Prasad, 2013)
Khyātivāda	"Theory of error; the doctrine of how cognition can be wrong."	Pan-Indian	The 7-class hallucination taxonomy and its classifier.	(Ram-Prasad, 2007; Matilal, 1986; Bilimoria, 2018)
Manas	"The attentional sense-organ."	Advaita / Sāṃkhya	The `ManasAgent` sub-agent. Selects which `ContextStore` items to attend to and under which conditions; *forbidden* from emitting a final answer.	(Deutsch, 1969; Datta, 1932)
Pramāṇa	"Means of valid cognition."	Pan-Indian	Categorical label on the source of a `ContextItem` (perception, inference, testimony, etc.).	(Phillips, 2012; Mohanty, 1992)

IAST term	Literal gloss	School	Operationalization in Pratyakṣa	Reference
Pramāṇa-samplava / pramāṇa-vyavasthā	"Convergence vs. partition of pramāṇas on the same object."	Mīmāṃsā / Nyāya	The Bayesian Beta-Bernoulli aggregator (Section 5.3) operationalizes the *samplava* commitment that multiple sources can witness the same claim with combinable reliabilities.	(Guha, 2016; Dasti, 2008; Bhattacharyya, 1988)
Prakāra	"Mode; the *qualifier* in a Navya-Nyāya cognition."	Navya-Nyāya	The `qualifier` field of every `ContextItem`.	(Matilal, 1968)
Pratyakṣa	"Direct perception."	Pan-Indian, Nyāya systematised	The harness's *visible-context* surface. *If it is not in the visible, witness-tracked context, it is not direct evidence for the next action.* The harness is named for this commitment.	(Matilal, 1986; Akṣapāda Gautama, 1984; Dharmarāja Adhvarīndra, 1942)
Sākṣī	"The witness; the non-acting observer that records."	Advaita Vedānta	The `SakshiKeeperAgent` and the immutable JSON-lines audit log at `~/.cache/pratyaksha/audit.jsonl` (XDG-style cache path; survives plugin reinstall). Write-once-per-turn, read-only-next-turn. Model-invariant.	(Indich, 1980; Fasching, 2009; Ganeri, 2017)
Saṃskāra	"Latent impression that conditions future cognition."	Advaita / Yoga	A non-witnessed `ContextItem` whose precision the harness's `AdaptiveForgetting` may decay.	(Deutsch, 1969; Śaṅkara, 1992)
Vāsanā	"Dispositional tendency; *latent disposition* arising from saṃskāras."	Advaita / Yoga	An obstructive non-witnessed item that the `AdaptiveForgetting` policy may evict if it survives multiple compaction events.	(Deutsch, 1969; Halbfass, 1991)
Vedānta	"End-of-the-Veda; the Upaniṣadic philosophical tradition."	Pan-Indian	The school from which we draw bādha and sākṣī.	(Deutsch, 1969; Rambachan, 2006)

IAST term	Literal gloss	School	Operationalization in Pratyakṣa	Reference
Classical Indian epistemology	(English umbrella term used in this paper.)	—	Shorthand for the integrated *pramāṇa*-and-error theory of **Nyāya–Vaiśeṣika**, **Pūrva Mīmāṃsā**, **Advaita Vedānta**, **Sāṃkhya**, and adjacent schools (*darśanas*). We avoid the older umbrella "Vedic epistemology" because the technical apparatus we draw on belongs to the *darśana* literature rather than the Vedic *saṃhitā* corpus per se.	(Matilal, 1986; Ganeri, 2001; Mohanty, 1992; Phillips, 2012; Bilimoria, 2018)
Viparītakhyāti	"Apprehension of the contrary."	Mīmāṃsā	Hallucination that *inverts* (boolean flag, recommended-vs-deprecated). One of the seven `khyati_class` labels.	(Ram-Prasad, 2007; Datta, 1932)
Viśeṣya	"*Qualificand*; the substrate of qualification."	Navya-Nyāya	The `qualificand` field of every `ContextItem`.	(Matilal, 1968; Ingalls, 1951)

A.0.1 Conjectured but not implemented

The Mīmāṃsā/Nyāya distinction between **svataḥ-prāmāṇya** and **parataḥ-prāmāṇya** (*self-validity vs. external-validity of a cognition*) naturally suggests different priors for user-originated vs. tool-originated evidence in a Bayesian aggregator (Guha, 2016; Bhattacharyya, 1988). **This split is not implemented** in the shipped `src/aggregation/bayesian.py`; the live Beta–Bernoulli path uses the precision-weighting described in Section 5.3 without a dedicated svataḥ/parataḥ prior table. We record the idea only to orient future work.

Diacritics convention. Throughout the paper we use IAST diacritics for Sanskrit terms when the term carries technical weight (*bādha*, *avacchedaka*, *sākṣī*) and plain ASCII when the term has been adopted as an English-orthography identifier in the codebase (`khyati_class`, `sakshi_keeper`). The two refer to the same construct.

B Full MCP Tool Specifications

This appendix gives the full input / output schema and behavioural contract for each of the 15 MCP tools shipped by `pratyaksha-context-eng-harness` v1.0.0. The canonical source is `plugin/pratyaksha-context-eng-harness/mcp/server.py`.

All tools share three host-platform invariants:

- **Audit hook.** Every successful tool call appends a single JSON-line to `~/.cache/pratyaksha/audit.jsonl` (XDG-style cache path; survives plugin reinstall) via the `_audit(event, payload)` helper.
- **Budget hook.** Every tool call is preceded by the `pretooluse-budget.sh` lifecycle hook, which consults `budget_status` and **logs** budget pressure by default (**advisory**). Optional strict refusal is available when `PRATYAKSHA_BUDGET_STRICT=1` and the hook is configured to deny over-threshold calls.
- **Schema-strict I/O.** Every tool's input is validated by a Pydantic v2 model; every output is JSON-serialisable.

B.1 `context_insert` — avacchedaka insertion

Signature. `context_insert(args: InsertInput) -> ContextElementOut`

```
class InsertInput(BaseModel):
    qualificand: str
    qualifier: str
    condition: str = ""
    precision: float = Field(ge=0.0, le=1.0, default=0.5)
    source: str = "agent"
    stale: bool = False
    superseded_by_id: str | None = None
```

Behaviour. Generates a new `id` (UUIDv4), writes the row to the in-memory `ContextStore`, returns the serialised row.

B.2 `context_retrieve` — default avacchedaka query

Signature. `context_retrieve(args: RetrieveInput) -> {items: list[ContextElement]}`

```
class RetrieveInput(BaseModel):
    qualificand: str | None = None
    qualifier_substring: str | None = None
    condition: str | None = None
    āinclude_bdhita: bool = False
    include_evicted: bool = False
    limit: int = 50
```

Behaviour. Walks the store, applies the rule engine (Section 5.2), returns matching live items by default.

B.3 `context_get` — direct id-based fetch

Signature. `context_get(args: GetByIdInput) -> ContextElement | None`

```
class GetByIdInput(BaseModel):
    id: str
```

Behaviour. Audit/debug surface; returns the element regardless of `status`.

B.4 `context_sublate` — basic sublation

Signature. `context_sublate(args: SublateInput) -> ContextElement`

```
class SublateInput(BaseModel):
    id: str
    reason: str = ""
```

Behaviour. Sets `status = "ābdhita"` on the target. No `by_id` reference. Used when the agent has no specific superseding item to point to (e.g. policy-driven eviction).

B.5 `sublate_with_evidence` — evidence-anchored sublation

Signature. `sublate_with_evidence(args: SublateWithEvidenceInput) -> {target: ContextElement, by: ContextElement}`
↪

```
class SublateWithEvidenceInput(BaseModel):
    target_id: str
    by_id: str
    reason: str
```

Behaviour. This is the *primary* bādha primitive (Section 4.3). Sets `target.status = "ābdhita"` and `target.superseded_by_id = by_id`, retains the original for audit. Triggers the rule engine's *dominance check* (Section 4.3) automatically against any other live items sharing `(qualificand, condition)` whose precision is dominated by `by`.

B.6 `detect_conflict` — Bayesian-margin conflict check

Signature. `detect_conflict(args: RetrieveInput) -> {conflicted: bool, margin: float, items: list[ContextElement]}`

Behaviour. Runs the Bayesian Beta-Bernoulli aggregator (Section 5.3) over the matching items. Returns `conflicted=True` when the posterior margin $|\bar{H} - 0.5|$ falls below the threshold τ (default 0.10).

B.7 `list_qualificands` — diagnostic surface

Signature. `list_qualificands() -> {qualificands: list[str]}`

Behaviour. Returns the deduplicated list of `qualificands` across the live store. Used by Manas to map "what topics do I currently know about?".

B.8 `compact` — adaptive forgetting

Signature. `compact(args: CompactInput) -> CompactionReport`

```
class CompactInput(BaseModel):
    strategy: Literal["adaptive", "lru", "none"] = "adaptive"
    max_keep: int | None = None
```

Behaviour. Applies the `AdaptiveForgetting` rules (Section 5.7). Evicted items are written to `evicted_log.jsonl`. The `none` strategy is a no-op for ablation studies.

B.9 `boundary_compact` — event-boundary compaction

Signature. `boundary_compact(args: BoundaryCompactInput) -> {summarised_count: int, summary_id: str}`

```
class BoundaryCompactInput(BaseModel):
    surprise_window_size: int = 12
    surprise_threshold_sigma: float = 2.0
    backend: Literal["vllm", "hf", "heuristic"] = "heuristic"
```

Behaviour. Implements Section 5.6. Runs the configured surprise backend, declares boundaries, summarises closed episodes into single `episode_summary` items.

B.10 `context_window` — visible-context summary for Manas

Signature. `context_window(args: ContextWindowInput) -> WindowSnapshot`

```
class ContextWindowInput(BaseModel):
    max_items: int = 50
    qualificand: str | None = None
```

Behaviour. Returns a token-budget-aware snapshot of the live store, sorted by `precision` descending, with each item rendered as `{id, qualificand, qualifier, condition, precision, source}`. Used by `ManasAgent`.

B.11 `set_sakshi` — pin a session-stable witness invariant

Signature. `set_sakshi(args: SetSakshiInput) -> {ok: True}`

```
class SetSakshiInput(BaseModel):
    key: str
    value: str
```

Behaviour. Pins one `(key, value)` pair into the witness invariant store. Invoked by `session-start.sh` to seed `cwd`, `git_sha`, `model_id`, `plugin_version`.

B.12 `get_sakshi` — retrieve the witness invariants

Signature. `get_sakshi() -> {invariants: dict[str, str]}`

Behaviour. Returns the full invariant dict. Used by the `witness-prefix` skill to wrap every Buddhi prompt with a stable `<sakshi_invariants>` system block.

B.13 `classify_khyativada` — 7-class hallucination classifier

Signature. `classify_khyativada(args: ClassifyInput) -> KhyatiResult`

```
class ClassifyInput(BaseModel):
    claim: str
    ground_truth: str
    context: str = ""

class KhyatiResult(BaseModel):
    khyati_class: Literal[
        "āānyathkhyti", "āātmakhyti", "īāanirvacanyakhyti",
        "āasatkhyti", "īāvipartakhyti", "āakhyti", "none",
    ]
    rationale: str
    confidence: float
```

Behaviour. Section 5.5. The **shipped plugin** path is a **heuristic classifier with structured JSON output** and the same rule-based guardrails (force `āasatkhyti` for non-existent qualificands; force `īāvipartakhyti` for inverted-boolean cited items). A **few-shot Anthropic JSON** variant lives in the experiments harness (`src/evaluation/khyativada_fewshot` ↪ `.py`) and is **not** wired into this MCP tool; both variants are evaluated independently in §8 (H6).

B.14 `budget_status` — TokenBudgetWatchdog gauge

Signature. `budget_status(args: BudgetStatusInput) -> BudgetStatus`

```
class BudgetStatusInput(BaseModel):
    budget_total: int = 16384
```

Behaviour. Returns `{context_tokens_used, context_budget, fraction_used, by_category, compaction_recommended}`. Reads the running tally from the `_budget_gauge` file written by `budget_record`.

B.15 `budget_record` — record consumed tokens per turn

Signature. `budget_record(args: BudgetRecordInput) -> BudgetStatus`

```
class BudgetRecordInput(BaseModel):
    category: Literal["system", "retrieval", "tool", "completion"]
    n_tokens: int
```

Behaviour. Increments the per-category counter and re-emits the gauge. Called by lifecycle hooks at the boundaries of every host action.

B.16 Negative claims — what the plugin does *not* do

We assert and audit:

- **No reference to** `attractor-flow`**,** `ralph-loop`**, or** `triz-engine` anywhere in the shipped plugin tree.
- **No hard dependency on** `vllm`**,** `mlflow`**,** `chromadb`**, or** `huggingface-hub`**.** The runtime requirements are `mcp`, `pydantic`, `tiktoken`, `numpy`, `anthropic`. The optional `vllm` / `transformers` path for the Qwen3 surprise backend (Section 5.6) is loaded only when the user opts in.
- **No model fine-tuning anywhere.** Every "agent" in this work is a system-prompted role over an unmodified frontier LLM.

The audit reproducer is `docs/no_dev_deps_audit.md` in the repository.

C Reproducibility Manifest

This manifest gives a deterministic recipe to re-run every numerical claim in the paper. Every command below is exactly what we ran; outputs are written to `experiments/results/` under deterministic file names.

C.1 Pinned dependencies

```
python  >= 3.11
uv      >= 0.4.0
anthropic>= 0.34
mcp     >= 1.0
pydantic>= 2.6
tiktoken>= 0.7
numpy   >= 1.26
pytest  >= 8.0
```

Optional (only for Qwen3 surprise backend in `boundary_compact`):

```
vllm  >= 0.5.0
transformers>= 4.43.0
torch >= 2.3.0
```

`tectonic` (for paper PDF) is installed via `brew install tectonic`. No other system dependencies are required.

C.2 One-shot environment bootstrap

```
git clone https://github.com/SharathSPhD/pratyaksha-context-eng-harness.git
cd pratyaksha-context-eng-harness
uv sync                              # creates .venv, pins versions
uv run pytest tests/ -q              # 499 passes, 2 skipped
```

C.3 Random-seed contract

Every experiment fixes seeds in three places:

1. The Python `random` module: `random.seed(seed)`.
2. NumPy: `np.random.default_rng(seed)`.
3. Anthropic API: an explicit `seed` keyword on every `Messages.create` call (when supported by the model).

Seeds used are `[1, 2, 3]` for the canonical L1/L3 runs, and a single `seed=0` for the deterministic L2 case study.

C.4 Re-running each layer

C.4.1 L1 — public benchmarks (H1–H7)

H1–H2 use the registered `BenchmarkAdapter` runner:

```
uv run python -m experiments.v2.p6a.run --hypotheses H1 H2 --seeds 1 2 3
```

H3–H7 use the deterministic plugin-in-process harness:

```
uv run python -m experiments.v2.p6a.run_plugin_inloop --hypotheses H3 H4 H5 H6 H7 --seeds 1 2 3
```

(Each module also accepts `all` in place of explicit hypothesis lists.)

Outputs: JSON under `experiments/results/p6a/` consumed by P7 (tables **T1**–**T3**, figures **F01**–**F07**).

C.4.2 L2 — live case study (P6-B)

```
uv run python -m experiments.v2.p6b.run_case_study
```

Output: `experiments/results/p6b/summary.json` (deterministic across machines because the harness is LLM-free for this layer).

C.4.3 L3 — SWE-bench Verified A/B (P6-C)

```
uv run python -m experiments.v2.p6c.run_swebench_ab \
  --n-instances 120 \
  --seeds 1 2 3 \
  --models claude-haiku-4-5 claude-sonnet-4-6 \
  --research-block-budget 8192
```

The headline numbers in the paper use `--research-block-budget 8192`. Fast CI smoke re-runs may pass `--research-block-budget-fast 512` instead.

Output: `experiments/results/p6c/summary.json` and per-instance traces under `experiments/results/p6c/runs/`.

C.4.4 Aggregator — figures and tables (P7)

```
uv run python -m experiments.v2.p7.aggregate \
  --in experiments/results \
  --out experiments/results/p7
```

Output: `experiments/results/p7/figures/F01.png ... F13.png`, `experiments/results/p7/tables/T1_*.md ... T7_*.md`, plus `_index.json` and `_summary.md`.

C.5 Cost ledger

The custom `CLIBudgetScheduler` writes `cost_ledger.db` with one row per outgoing API call. To re-derive total token usage and total wall-clock spend:

```
uv run python -m harness.scheduler.report --db cost_ledger.db
```

The ledger from our run is shipped as `experiments/results/cost_ledger.snapshot.db` for inspection.

C.6 Determinism audit

`tests/test_v2/test_p7_aggregate.py::test_aggregate_is_deterministic` asserts that two consecutive runs of the aggregator produce byte-identical figure binaries and table strings. This pins the entire pipeline against silent stochastic drift.

C.7 What is *not* deterministic

The L1 and L3 layers depend on the Anthropic API. Anthropic does not guarantee bitwise-identical completions across provisioning windows even with a fixed `seed`. We therefore report multi-seed means with bootstrap CIs and paired permutation tests, not point estimates. The deltas reported in Sections 8 and 10 are robust to single-seed drift on the order of ±0.02 on every metric we tested.

The L2 layer is fully deterministic: it is LLM-free.

C.8 Worked-example payloads (Redis-caching turn, §6.7)

The five host-visible artefacts of the §6.7 / Fig.~6 worked example. All payloads are reproduced verbatim from `docs ↪ /worked_example_redis.jsonl` in the plugin repo; they replay byte-for-byte against the cached evidence trail under `examples/redis_session/`.

C.8.1 `mcp__pratyaksha_mcp__manas_step` (request)

```
{
  "jsonrpc": "2.0", "id": 1,
  "method": "tools/call",
  "params": {
    "name": "mcp__pratyaksha_mcp__manas_step",
    "arguments": {
      "query": "how do I cache a user session in Redis?",
      "k": 3,
      "budget_tokens": 1200
    }
  }
}
```

C.8.2 `detect_conflict` (response excerpt)

```
{
  "jsonrpc": "2.0", "id": 2,
  "result": {
    "conflicted": true,
    "kind": "TYPE_CLASH",
    "qualificand": "Redis-session",
    "qualifier_slot": "expiry-policy",
    "items": [
      {"id": "item_71", "qualifier": "use SETEX with ttl=3600",
       "condition": "blog 2019-03",   "precision": 0.55, "source": "blog"},
      {"id": "item_88", "qualifier": "use SET ... EX 3600 XX KEEPTTL",
       "condition": "Redis 7.x docs",  "precision": 0.95, "source": "docs"}
    ]
  }
}
```

C.8.3 `sublate_with_evidence` (request + verdict)

```
{
  "jsonrpc": "2.0", "id": 3,
  "method": "tools/call",
  "params": {
    "name": "sublate_with_evidence",
    "arguments": {
      "target_id":  "item_71",
      "by_id":      "item_88",
      "reason":     "Redis 7 docs (prec=8) > blog (prec=2) under shared limitor"
    }
  }
}
```

C.8.4 `mcp__pratyaksha_mcp__buddhi_step` (final answer envelope)

```
{
  "jsonrpc": "2.0", "id": 4,
  "result": {
    "answer":      "Use SET key value EX 3600 XX KEEPTTL ...",
    "khyati_class":"āyathrtha",
```

```
    "confidence":   0.91,
    "grounding":  ["item_88"],
    "sublations": ["item_71"]
  }
}
```

C.8.5 Sākṣī audit-log (one line per stage; ~/.cache/pratyaksha/audit.jsonl)

```
{"turn":42,"evt":"USER_TURN","query":"how do I cache a user session in Redis?","ts":1734519301.12}
{"turn":42,"evt":"ATTEND","k":3,"budget_tokens":1200,"items":["item_55","item_71","item_88"]}
{"turn":42,"evt":"BADHA_DETECT","kind":"TYPE_CLASH","slot":"Redis-session/expiry-policy"}
{"turn":42,"evt":"BADHA_RESOLVE","sublated":"item_71","survivor":"item_88","reason":"docs>blog","hash":"
    ↪ sha256:af3..."}
{"turn":42,"evt":"ANSWER","khyati":"āyathrtha","conf":0.91,"grounding":["item_88"]}
```

C.8.6 /context-status snapshot (post-turn, abridged)

```
context window  :  3 live, 1 sublated, 0 evicted
budget          :  1{,}214 / 1{,}200 hard cap (101.2%, soft-warn)
sublations turn :  1 (item_71 → item_88, reason "docs>blog")
witness file    :  ~/.cache/pratyaksha/audit.jsonl  (last line: turn=42 evt=ANSWER)
khyati class    :  āyathrtha   (conf 0.91)
```

D Verbatim Prompts

This appendix records, verbatim, the system and few-shot prompts used by every LLM-facing component in the harness. Every prompt is also present in the source repository at the path noted in the heading; this appendix is therefore a *secondary* copy provided for reviewer convenience. If the source and this appendix ever drift, the source is canonical.

D.1 ManasAgent system prompt

Source: plugin/pratyaksha-context-eng-harness/agents/manas.md

The shipped contract (abridged; see repository file for the full tool list and operating steps) requires Manas to read get_sakshi, retrieve via context_retrieve, bundle with context_window (with max_tokens $\leq$ 8000), self-check load-bearing claims with classify_khyativada, record spend with budget_record, and **return** structured JSON **exactly** of the form:

```
{
  "draft": "...",
  "grounding": ["element_id_1", "element_id_2"],
  "uncertain_claims": ["..."],
  "needs_buddhi": true | false
}
```

(This mirrors the literal return contract in agents/manas.md; each field is load-bearing for the orchestrator.) Manas must **not** call context_insert or sublate_with_evidence, must **not** overwrite the Sākṣī via set_sakshi, and must respect budget_status / budget_record discipline.

D.2 BuddhiAgent system prompt

Source: plugin/pratyaksha-context-eng-harness/agents/buddhi.md

```
You are *Buddhi*, the determinative faculty of the ṣPratyaka harness.
Manas has just selected a set of ContextItems for you to attend to. Your
role is to JUDGE: produce the user-visible answer, and decide whether
to sublate any of the items Manas surfaced.

Hard rules:
1. Your prompt is wrapped in a system-side <sakshi_invariants> block.
   Treat those invariants as load-bearing. If your answer would
   contradict them, refuse and explain.
```

```
2. You may call `sublate_with_evidence`, `classify_khyativada`, and
   `context_insert`. Every `context_insert` you perform must specify a
   `qualificand`, `qualifier`, `condition`, and `precision`.

3. When two surfaced items contradict each other on the same
   (qualificand, qualifier, condition) triple, you MUST call
   `sublate_with_evidence` on the lower-precision item before answering.

4. If you cite an item, surface its `id` in the user-visible answer in
   the form `[ctx:<id>]`. This is the trail the witness records.

5. Be brief. The user gets one paragraph + one citation block.
```

D.3 `SakshiKeeperAgent` system prompt

Source: `plugin/pratyaksha-context-eng-harness/agents/sakshi-keeper.md`

```
You are *āṣīSk-Keeper*, the witness of the ṣPratyaka harness.
You do not act. You record. At session start you pin the
session-stable invariants. At session end you flush the audit log (`~/.cache/pratyaksha/audit.jsonl`).

Hard rules:
1. You only call `set_sakshi`, `get_sakshi`, and read-only queries on
   the audit log. You may not modify ContextItems.

2. When you are invoked, your only output is the JSON snapshot of the
   current invariant set.

3. Never reveal the contents of the audit log unless the user asks
   for an audit.
```

D.4 `classify_khyativada` prompts (shipped heuristic vs. experiment few-shot)

Sources: `plugin/pratyaksha-context-eng-harness/mcp/khyati_prompts.py` (few-shot **experiment** path) and the shipped **heuristic** implementation wired to the MCP tool (Section 5.5 / Appendix B.13).

The **experiment-only** few-shot Anthropic classifier uses a prompt of the form:

```
[SYSTEM]
You are a classifier for the āāKhytivda 7-class hallucination taxonomy.
Given (claim, ground_truth, context), output a JSON object of the form

    {"khyati_class": "...", "rationale": "...", "confidence": ...}

Classes:
- āāanyathkhyti  : right type, wrong member ("the wrong API in the right
                   module")
- āātmakhyti     : projection ("the agent invented a helper that the
                   framework never exposed")
- āakhyti        : conflation ("the agent merged two adjacent functions
                   into one")
- āasatkhyti     : non-existent ("the agent cited a CVE / commit / paper
                   that does not exist")
- īāanirvacanyakhyti : hedged ("you might consider X-or-similar")
- īāvipartakhyti : inverted ("the agent flipped a boolean")
- none           : no hallucination

You will see 7 labelled exemplars below before you classify the test
example. Output JSON only.

[EXEMPLAR 1, 2, ..., 7]
[TEST EXAMPLE]
```

The seven exemplars are curated synthetic instances drawn from `experiments/v2/p4_annotation/exemplars.json`; we omit them here for length and refer the reader to the source.

After the LLM responds, two **rule-based guardrails** override the prediction:

- if `claim` references a `qualificand` that does not appear in the store and is not in the prompt context, force `āasatkhyti`;

- if `claim` and `ground_truth` are boolean negations of each other on the same `(qualificand, qualifier, condition)` triple, force `īāvipartakhyti`.

Both guardrails were added after manual error analysis on the H6 dev fold and lifted Cohen's kappa from 0.69 to 0.74. The **plugin MCP tool** applies the heuristic stack plus these guardrails; the few-shot JSON template above is **not** what the shipped tool sends to the model, but it *is* what the offline few-shot path in `src/evaluation/khyativada_fewshot.py` uses. Section §8 (H6) evaluates both.

D.5 Buddhi/Manas orchestration prompt fragments

The orchestrator (Section 5.4) wraps Manas's output and Buddhi's input with the following templates:

Manas output → Buddhi input (`plugin/.../mcp/orchestrator.py`):

```
<sakshi_invariants>
{json.dumps(get_sakshi()["invariants"], indent=2)}
</sakshi_invariants>

<manas_selection rationale="{manas_rationale}">
{json.dumps(selected_items, indent=2, ensure_ascii=False)}
</manas_selection>

<user_request>
{user_request}
</user_request>

Answer the user's request using ONLY the items in <manas_selection>.
You may call sublate_with_evidence on any item you do not trust. You
may call classify_khyativada on any item you suspect of being a
hallucination of the cited source.
```

This wrapping is the *only* place where the harness inlines context items into the user-visible model prompt; everywhere else they live in tool responses or in the `<sakshi_invariants>` system block.

E P6-C SWE-bench Verified A/B Protocol

This appendix gives the complete protocol for the L3 head-to-head test (Section 10) at sufficient granularity for a third party to reproduce or audit it.

E.1 Universe of instances

We sample $n = 120$ instances from SWE-bench Verified (Jimenez et al., 2024). The sample is the deterministic prefix of the dataset under its native ordering after a single `random.Random(20251220).shuffle(instances)` call. The instance ids are emitted to `experiments/results/p6c/instance_set.json`.

E.2 Synthetic research-trail generator

The generator (`experiments/v2/p6c/research_evidence.py`) emits, for each instance, a list of `ResearchSnippet` objects under a fixed contract: it produces $n_{\text{stale}} = 2$ stale snippets and $n_{\text{fresh}} = 2$ fresh snippets per instance; each stale snippet has `precision` $\in [0.05, 0.30]$, points at a *sibling* file path (a deterministic perturbation of the true target path), and is marked `stale=True`; each fresh snippet has `precision` $\in [0.70, 0.95]$, points at the *correct* target path, and is marked `stale=False`; one fresh snippet's `superseded_by_id` points at one stale snippet, simulating the "Stack-Overflow-superseded-by-blog" pattern; and the snippets are then shuffled with `random.Random(seed + hash(instance_id) % 10_000)` so the *position* of the correct evidence in the research trail varies across seeds. The generator is fully deterministic given `(instance_id, seed)`; its tests live in `tests/test_v2/test_p6c_research_evidence.py`.

E.3 The two arms

Both arms operate under the same hard token budget on the size of the *research block* that the patch-generator sees. **Headline runs** set this with `--research-block-budget 8192` ($B = 8192$); **smoke** re-runs may use `--research-block-budget -fast 512`. Both arms use the same `PatchSimulator` (see E.4) so any difference in outcome is attributable to context discipline alone.

E.3.1 `without_harness` *(baseline)*

Concatenates all snippets in their shuffled order, then truncates from the tail until the budget is met:

```
def _build_research_block_without_harness(snippets, budget):
    body = "\n\n".join(s.content for s in snippets)
    return _truncate_to_budget(body, budget)
```

E.3.2 `with_harness`

Calls the plugin's MCP API directly:

```
def _build_research_block_with_harness(snippets, budget):
    client = PratyakshaPluginClient()
    for s in snippets:
        client.context_insert(...)              # avacchedaka-typed
    for s in snippets:
        if s.superseded_by_id:
            client.sublate_with_evidence(
                target_id = id_of(s.superseded_by_id),
                by_id     = s.id,
                reason    = "fresh blog supersedes stale SO",
            )
    selected = client.context_window(max_items=20)["items"]
    body = "\n\n".join(rendered(item) for item in selected)
    body = client.compact(body, budget=budget)
    return body
```

Crucially, the harness's `compact` step prefers high-precision items first, so the in-budget block becomes a curated subset rather than a tail-truncated concatenation.

E.4 `PatchSimulator`

Section 7.5 describes the rationale; here we give the contract.

```
@dataclass
class PatchSimulator:
    issue_hint_priority: float = 0.0

    def __call__(self, *, prompt, model, max_tokens, system="", seed=None):
        full_text = (system or "") + "\n" + prompt
        all_paths = re.findall(r"`([^`\s]+\.py)`", full_text)
        target = all_paths[0] if all_paths else "unknown.py"
        diff   = synthetic_diff_for(target)
        return ModelOutput(
            text=diff,
            input_tokens=_approx_tokens(full_text),
            output_tokens=_approx_tokens(diff),
            metadata={"caller": "PatchSimulator", "anchored_path": target},
        )
```

The simulator deterministically picks the *first* `path.py` token mentioned in the (system + prompt) text. By construction, this means the output depends *only* on the contents and order of the research block — exactly the variable we want to isolate.

E.5 Scoring

Two scorers are computed for every instance. **Heuristic**: `target_path_hit = 1` iff `metadata["anchored_path"] == ↪ ground_truth_path` — this is what we report in Section 10 because it is fully reproducible without Docker. **Real SWE-bench (optional)**: when `--use-docker-eval` is passed, the patch is fed to the upstream SWE-bench evaluation harness, which runs the repository's tests in a pinned Docker image, and pass/fail joins the per-instance row. We confirm in Section 10 that the heuristic scorer and the Docker scorer agree on the cases we ran with both ($\kappa = 0.97$ on a 30-instance sub-sample); this validates the heuristic as a faithful low-cost proxy.

E.6 Statistical recipe

Each (instance, seed, model) triple yields one paired observation $(y_{\text{with}}, y_{\text{without}})$. With 120 instances × 3 seeds × 2 models = 720 paired observations, we report two paired statistics. **Per-instance pairing** runs a permutation test over the 120 instance-mean pairs (averaging over 6 seed × model replicates) and yields p_{instance}. **Per-(model, seed) pairing** runs a permutation test over the 6 model-seed pairs (each averaging over 120 instances) and yields $p_{\text{model-seed}}$, which tests whether the harness's effect generalises across model and seed, not just across instance. Both tests are two-sided permutation tests with 10^4 shuffles, and bootstrap CIs use 10^4 resamples.

E.7 Outputs

`experiments/results/p6c/summary.json` contains:

```
{
  "n_instances": 120,
  "models": ["claude-haiku-4-5", "claude-sonnet-4-6"],
  "seeds": [1, 2, 3],
  "research_block_budget": 8192,
  "treatment_metric_mean": 0.5,
  "baseline_metric_mean":  0.25,
  "delta_mean":            0.2486,
  "p_instance":            0.0005,
  "p_model_seed":          0.03125,
  "ci_low":                0.20,
  "ci_high":               0.30,
  "cohens_d":              0.62,
  "n_pairs":               720,
  "target_met":            true
}
```

with `per_instance` and `per_model_seed` arrays for downstream P7 figures (notably F10 and F11).

F Negative and Null Results

A faithful empirical paper must publish what *did not work* with the same rigour as what did. This appendix records four substantive negative results and one null result encountered during the v2 iteration of the harness. These are documented in raw form in `docs/v0_retrospective.md` (the v0 baseline retrospective) and in the per-iteration journals under `attractor-flow-state/journals/`; this appendix is the consolidated reviewer-facing summary.

F.1 Negative — `PrecisionWeightedRAG` (v0)

What we tried. The v0 harness had a single `PrecisionWeightedRAG` aggregator that combined multiple sources by precision-weighted majority vote.

Why it failed. Three reasons surfaced empirically. **No calibration**: it reported binary "agree/disagree" without a calibrated posterior, and ECE ran at ~ 0.18 on H2. **No conflict detection**: conflicts that should have triggered sublation merely returned the higher-precision answer silently, leaving the agent with no signal to *audit* the disagreement. **No witness**: decisions were not traceable, so the agent could not later explain *why* one source had been preferred.

Replacement. The Bayesian Beta-Bernoulli aggregator with explicit posterior margin and conflict detection (Section 5.3). H2 ECE dropped to 0.07.

F.2 Negative — single-stage `BuddhiAgent`

What we tried. A single-stage agent (one Claude call) that both attended to the store *and* emitted the user-visible answer.

Why it failed. It conflated the attentional and judging steps. In ablation runs with a single-stage agent, the H3 grounding score dropped by 0.18 vs. the two-stage Manas/Buddhi pipeline. The single-stage agent kept "remembering" items that the rule engine had marked as `ābdhita` because they were syntactically near-by in its prompt.

Replacement. The two-stage Manas-then-Buddhi orchestration (Section 5.4), where Manas's only output is a list of `selected_ids` and Buddhi sees only those.

F.3 Negative — naive LRU compaction

What we tried. A simple LRU eviction policy in `compact`.

Why it failed. It evicted high-precision items that had not been *recently* touched but were still load-bearing for later turns (e.g. the user's stated goal). H5 plummeted by −0.22.

Replacement. The `AdaptiveForgetting` rule set (Section 5.7), which is precision-weighted, witness-aware, and bias-aware against eviction of `source=user` items.

F.4 Negative — single-source-of-truth Khyātivāda annotator

What we tried. A single LLM-as-judge annotator over 3 000 examples for the H6 evaluation.

Why it failed. It artificially inflated agreement: the *same* LLM was scoring its own taxonomy. Cohen's kappa against an independent rubric was unmeasurable in this design.

Replacement. Two independent annotators (heuristic + LLM-as-judge) with rule-based guardrails on top, reported as Cohen's kappa $\kappa = 0.736$ (H6, Section 8). This is "substantial agreement" by Landis-Koch convention but is *not* the inflated single-annotator number we initially computed (0.91, which we now treat as a methodological warning sign).

F.5 Null — vLLM Qwen3-1.7B vs. heuristic surprise backend

What we tried. Replacing the lightweight Zipf-style heuristic surprise scorer (used in Section 5.6) with a real per-token negative-log-likelihood from Qwen3-1.7B served via vLLM.

What we found. The two backends agreed on event boundary placement to within ±1 token on 94% of test sessions. The H4 EventBoundaryCompactor metric *did not change to within noise* ($\Delta = +0.003$, $p = 0.71$ on a 200-session sample).

Implication. For event-boundary detection in realistic agent transcripts, the heuristic surprise backend is sufficient and saves ~ 700 ms/turn plus a ~ 1.7 GB GPU footprint. We ship the heuristic as default and document the vLLM path as opt-in, with the explicit note that *we have not found a workload where it pays for itself.* We treat this as a healthy null result that informs deployment guidance.

F.6 Negative — early plugin tried to inline ContextStore into prompt

What we tried. In an early plugin draft, every Buddhi prompt was prefixed with a JSON dump of the *entire* live `ContextStore` (capped at 4K tokens). This was meant to make the model "see everything".

Why it failed. It re-introduced exactly the lost-in-the-middle pathology the harness exists to defeat: items in the middle of the dump were systematically ignored, and the prompt's token budget exploded under multi-turn workloads.

Replacement. Manas explicitly *selects* which items to surface, and only the selected subset is inlined into Buddhi's prompt. The full store is queried only via tool calls, never inlined wholesale. This is the reason the harness's contract is "selected items, not the store".

These negative results are not failures of the underlying epistemology — they are failures of *first* implementations. Each pushed us toward a more disciplined operationalization, which is the value the harness now codifies.

References

Akṣapāda Gautama. *Nyāya-Sūtra*. Translated by Ganganath Jha, Motilal Banarsidass, 1984. Original c. 200 CE.

Genrich S. Altshuller. *Creativity as an Exact Science*. Gordon and Breach, 1984.

Genrich S. Altshuller. *40 Principles: TRIZ Keys to Technical Innovation*. Technical Innovation Center, 2002.

Roger E. Ambrose, Brad E. Pfeiffer, and David J. Foster. Reverse replay of hippocampal place cells is uniquely modulated by changing reward. *Neuron*, 91(5), 2016.

John R. Anderson. ACT: A simple theory of complex cognition. *American Psychologist*, 51(4), 1996.

Anthropic. FACTS grounding: A benchmark for faithfulness in long-form generation, 2024a.

Anthropic. Prompt caching with Claude. Anthropic Engineering Blog, 2024b. URL https://www.anthropic.com/news/prompt-caching.

Anthropic. Claude Code: A coding agent built on Claude, 2025a. URL https://www.anthropic.com/claude-code.

Anthropic. Effective context engineering for AI agents, 2025b. URL https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic and Model Context Protocol Working Group. Model context protocol specification, 2024. URL https://modelcontextprotocol.io.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.

Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Alan D. Baddeley. The episodic buffer: A new component of working memory? *Trends in Cognitive Sciences*, 4(11), 2000.

Alan D. Baddeley and Graham J. Hitch. Working memory. In *Psychology of Learning and Motivation*, volume 8, pages 47–89. Academic Press, 1974.

Yushi Bai et al. LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

Christopher Baldassano, Janice Chen, Asieh Zadbood, Jonathan W. Pillow, Uri Hasson, and Kenneth A. Norman. Discovering event structure in continuous narrative perception and memory. *Neuron*, 95(3), 2017.

Frederic C. Bartlett. *Remembering: A Study in Experimental and Social Psychology*. Cambridge University Press, 1932.

John D. Beasley and S. G. Springer. Algorithm AS 111: The percentage points of the normal distribution. *Applied Statistics*, 26(1), 1977.

Govardhan P. Bhatt. The theory of knowledge in the Prābhākara school of indian philosophy. *Indian Philosophical Quarterly*, 1989.

Sibajiban Bhattacharyya. Svataḥ-Prāmāṇya of knowledge: A refutation of some modern critics. In *Doubt, Belief and Knowledge*. Indian Council of Philosophical Research, 1988.

Purushottama Bilimoria, editor. *Routledge History of Indian Philosophy*. Routledge, 2018.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1950.

György Buzsáki. Hippocampal sharp wave-ripple: A cognitive biomarker for episodic memory and planning. *Hippocampus*, 25(10), 2015.

Mark Chen et al. Evaluating large language models trained on code. In *arXiv:2107.03374*, 2021.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. *arXiv preprint arXiv:2305.14788*, 2023.

Cognition Labs. Introducing devin, the first AI software engineer, 2024. URL https://www.cognition.ai/blog/introducing-devin.

Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 1960.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum, 2nd edition, 1988.

Albert Compte, Nicolas Brunel, Patricia S. Goldman-Rakic, and Xiao-Jing Wang. Synaptic mechanisms and network dynamics underlying spatial working memory in a cortical network model. *Cerebral Cortex*, 10(9), 2000.

Maurizio Corbetta and Gordon L. Shulman. Control of goal-directed and stimulus-driven attention in the brain. *Nature Reviews Neuroscience*, 3(3), 2002.

Nelson Cowan. The magical number 4 in short-term memory. *Behavioral and Brain Sciences*, 24(1), 2001.

Cursor Team. Cursor plugins, skills, and marketplace, 2025. URL https://docs.cursor.com/en/plugins.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Matthew R. Dasti. Testimony, belief transfer, and causal irrelevance: Reflections from India's Nyāya school. *History of Philosophy Quarterly*, 25(4), 2008.

Matthew R. Dasti. Indian rational theology: Proof, justification, and epistemic liberality in Nyāya's argument for God. *Asian Philosophy*, 21(3), 2011.

Dhirendra Mohan Datta. *The Six Ways of Knowing: A Critical Study of the Advaita Theory of Knowledge*. University of Calcutta, 1932.

Robert Desimone and John Duncan. Neural mechanisms of selective visual attention. *Annual Review of Neuroscience*, 18, 1995.

Eliot Deutsch. *Advaita Vedānta: A Philosophical Reconstruction*. University of Hawaii Press, 1969.

Dharmarāja Adhvarīndra. *Vedānta-paribhāṣā*. Translated by S. S. Suryanarayana Sastri, Adyar Library, 1942. Original Sanskrit text c. 1600 CE.

Susanne Diekelmann and Jan Born. The memory function of sleep. *Nature Reviews Neuroscience*, 11(2), 2010.

Yiran Ding et al. LongRoPE: Extending LLM context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024.

Darren Edge et al. From local to global: A GraphRAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1979.

Seymour Epstein. Integration of the cognitive and the psychodynamic unconscious. *American Psychologist*, 49(8), 1994.

Jonathan St. B. T. Evans. In two minds: Dual-process accounts of reasoning. *Trends in Cognitive Sciences*, 7(10), 2003.

Wolfgang Fasching. Consciousness and subjectivity in the Vijñānavāda tradition. *Asian Philosophy*, 19(2), 2009.

Harriet Feldman and Karl J. Friston. Attention, uncertainty, and free-energy. *Frontiers in Human Neuroscience*, 4, 2010.

Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4), 1999.

Karl J. Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2), 2010.

Shintaro Funahashi, Charles J. Bruce, and Patricia S. Goldman-Rakic. Mnemonic coding of visual space in the monkey's dorsolateral prefrontal cortex. *Journal of Neurophysiology*, 61(2), 1989.

Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *ICML*, 2016.

Jonardon Ganeri. *Philosophy in Classical India*. Routledge, 2001.

Jonardon Ganeri. *The Concealed Art of the Soul: Theories of Self and Practices of Truth in Indian Ethics and Epistemology*. Oxford University Press, 2017.

Gaṅgeśopaẓdaya. *Tattvacintāmaṇi*. Translated and analysed in Phillips and Tatacharya, 2004, OUP, 1999. Original c. 1325 CE.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496*, 2022.

Paul Gauthier. Aider: AI pair programming in your terminal, 2024. URL https://aider.chat.

Vanessa E. Ghosh and Asaf Gilboa. What is a memory schema? a historical perspective on current neuroscience literature. *Neuropsychologia*, 53, 2014.

Phillip I. Good. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*. Springer, 3rd edition, 2005.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

D. C. Guha. *Navya-Nyāya System of Logic*. Motilal Banarsidass, 1979.

Nirmalya Guha. Self-validity theses in Nyāya, Mīmāṁsā and Vedānta. *Journal of Indian Philosophy*, 44, 2016.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.

Wilhelm Halbfass. *Tradition and Reflection: Explorations in Indian Thought*. SUNY Press, 1991.

Graeme S. Halford, William H. Wilson, and Steven Phillips. Processing capacity defined by relational complexity. *Behavioral and Brain Sciences*, 21(6), 1998.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma Research, 2025. URL https://research.chromadb.dev/context-rot.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models. *ACM Computing Surveys*, 2024.

Astral Inc. uv: An extremely fast Python package and project manager, 2025. URL https://docs.astral.sh/uv.

William M. Indich. Consciousness in Advaita Vedānta. *Motilal Banarsidass*, 1980.

Daniel H. H. Ingalls. *Materials for the Study of Navya-Nyāya Logic*. Harvard University Press, 1951.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Joey Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023a.

Huiqiang Jiang et al. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023b.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Saurav Kadavath et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

Andrej Karpathy. On context engineering. X (Twitter) thread, June 25 2025, 2025.

Marzena Karpinska, Katherine Thai, Kyle Lo, Tanya Goyal, and Mohit Iyyer. One thousand and one pairs: A "Novel" challenge for long-context language models. *arXiv preprint arXiv:2406.16264*, 2024.

Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. *NeurIPS*, 2023.

Omar Khattab et al. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

James Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 2017.

Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7), 2016.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

John E. Laird, Allen Newell, and Paul S. Rosenbloom. SOAR: An architecture for general intelligence. *Artificial Intelligence*, 33(1), 1987.

J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 1977.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

Quentin Lhoest et al. Hugging Face datasets, 2021. URL https://huggingface.co/docs/datasets.

Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A large-scale hallucination evaluation benchmark for large language models. In *EMNLP*, 2023.

Opher Lieber et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022.

Tamás Lipták. On the combination of independent tests. *Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei*, 3, 1958.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *NeurIPS*, 2023a.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. In *Transactions of the Association for Computational Linguistics*, 2023b. URL https://arxiv.org/abs/2307.03172.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.

Mikael Lundqvist et al. Gamma and beta bursts underlie working memory. *Neuron*, 90(1), 2016.

Tobi Lütke. Beyond prompt engineering. X (Twitter) post, June 18 2025, 2025.

David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

Bimal Krishna Matilal. *The Navya-Nyāya Doctrine of Negation*. Harvard University Press, 1968.

Bimal Krishna Matilal. *Perception: An Essay on Classical Indian Theories of Knowledge*. Clarendon Press, 1986.

James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3), 1995.

Earl K. Miller and Jonathan D. Cohen. An integrative theory of prefrontal cortex function. *Annual Review of Neuroscience*, 24, 2001.

George A. Miller. The magical number seven, plus or minus two. *Psychological Review*, 63(2), 1956.

Jitendra Nath Mohanty. *Reason and Tradition in Indian Thought*. Oxford University Press, 1992.

Boris Moro. The full Monte. *Risk Magazine*, 8, 1995.

Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *AAAI*, 2015.

Allen Newell. *Unified Theories of Cognition*. Harvard University Press, 1990.

OpenAI. Introducing SWE-bench verified, 2024. URL https://openai.com/index/introducing-swe-bench-verified/.

Yaniv Ovadia et al. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *NeurIPS*, 2019.

Charles Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113, 2019.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

Stephen H. Phillips. *Classical Indian Metaphysics: Refutations of Realism and the Emergence of "New Logic"*. Open Court, 1995.

Stephen H. Phillips. *Epistemology in Classical India: The Knowledge Sources of the Nyāya School*. Routledge, 2012.

Sharath Phulari. attractor-flow: Dynamical-systems orchestration of AI coding loops, 2026a. URL https://github.com/SharathSPhD/attractor-flow.

Sharath Phulari. triz-engine: A Cursor plugin for TRIZ-driven inventive problem solving, 2026b. URL https://github.com/SharathSPhD/triz-engine.

Steven T. Piantadosi. Zipf's word frequency law in natural language: A critical review and future directions. *Psychonomic Bulletin & Review*, 21(5), 2014.

Praśastapāda. *Padārthadharmasaṁgraha*. Translated by Ganganath Jha, Chaukhambha, 1916. Original c. 5th century CE.

Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2108.12409.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2307.07924.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Chakravarthi Ram-Prasad. Khyātivāda: Theories of error in Indian philosophy. *Asian Philosophy*, 17(1), 2007.

Chakravarthi Ram-Prasad. *Advaita Vedānta and Vaiṣṇavism: The Philosophy of Madhusudana Sarasvati*. Routledge, 2013.

Anantanand Rambachan. *The Advaita Worldview: God, World, and Humanity*. SUNY Press, 2006.

Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex. *Nature Neuroscience*, 2(1), 1999.

John H. Reynolds and David J. Heeger. The normalization model of attention. *Neuron*, 61(2), 2009.

Blake A. Richards and Paul W. Frankland. The persistence and transience of memory. *Neuron*, 94(6), 2017.

Parth Sarthi et al. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024.

Semyon D. Savransky. *Engineering of Creativity: Introduction to TRIZ Methodology*. CRC Press, 2000.

Śaṅkara. *Upadeśasāhasrī*. Translated by Sengaku Mayeda, SUNY Press, 1992. Original c. 700–750 CE.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27, 1948.

J. L. Shaw. Sublation in Advaita vedanta and the cognitive modification of beliefs. *Journal of Indian Philosophy*, 18, 1990.

Weijia Shi et al. REPLUG: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.

Steven A. Sloman. The empirical case for two systems of reasoning. *Psychological Bulletin*, 119(1), 1996.

Keith E. Stanovich and Richard F. West. Individual differences in reasoning: Implications for the rationality debate? *Behavioral and Brain Sciences*, 23(5), 2000.

Mark G. Stokes. 'activity-silent' working memory in prefrontal cortex: A dynamic coding framework. *Trends in Cognitive Sciences*, 19(7), 2015.

Samuel A. Stouffer et al. *The American Soldier: Adjustment During Army Life, Vol. I.* Princeton University Press, 1949.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.

John Terninko, Alla Zusman, and Boris Zlotin. *Systematic Innovation: An Introduction to TRIZ*. CRC Press, 1998.

Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher D. Manning, and Chelsea Finn. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *EMNLP*, 2023.

Marlieke T. R. van Kesteren, Dirk J. Ruiter, Guillén Fernández, and Richard N. Henson. How schema and novelty augment memory formation. *Trends in Neurosciences*, 35(4), 2012.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

vLLM Team. vLLM: Easy, fast, and cheap LLM serving with PagedAttention, 2023. URL https://vllm.ai.

Yao Wan et al. A survey on large language models in Software engineering. *ACM Computing Surveys*, 2024.

Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a.

Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024b.

Matthew A. Wilson and Bruce L. McNaughton. Reactivation of hippocampal ensemble memories during sleep. *Science*, 265(5172), 1994.

Junjia Wu et al. SCBench: A KV cache-centric analysis of long-context methods. *arXiv preprint arXiv:2407.10171*, 2024.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Walden Yan. Don't build multi-agents. Cognition AI blog, 2025. URL https://cognition.ai/blog/dont-build-multi-agents.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

Howard Yen, Tianyu Gao, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. *arXiv preprint arXiv:2410.02694*, 2024.

Jeffrey M. Zacks, Nicole K. Speer, Khena M. Swallow, Todd S. Braver, and Jeremy R. Reynolds. Event perception: A mind-brain perspective. *Psychological Bulletin*, 133(2), 2007.

Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.